



The spelled-out intro to neural networks and backpropagation: building micrograd

从零实现 micrograd：神经网络与反向传播

时长 2:25:51

句数 1363

术语 159

章节 20

摘要

观看后，学习者将通过从零构建 micrograd（一个基于 Value 对象表达式图、用链式法则自动求梯度的微型自动求导引擎）理解反向传播的底层原理。他们还会看到如何用这些梯度实现神经元、多层感知机、损失函数和梯度下降训练循环，并了解 micrograd 与 PyTorch 的异同。

After watching, learners will understand how backpropagation works under the hood by building micrograd, a tiny autograd engine that constructs expression graphs of Value objects and computes gradients automatically using the chain rule. They will also see how those gradients power a neuron, MLP, loss function, and a gradient-descent training loop, and how micrograd compares to PyTorch.

微梯度与自动求导介绍 00:00

- 00:00 Hello, my name is Andrej and I've been training deep **neural networks** for a bit more than a decade and in this lecture

你好，我叫 Andrej，我已经训练深度神经网络十多年了，而在本讲中

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- 00:05 I'd like to show you what **neural network** training looks like under the hood. So in particular 我想向你展示神经网络训练在底层究竟是什么样的。具体来说

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- 00:10 we are going to start with a blank Jupyter notebook and 我们将从一个空白的 Jupyter notebook 开始，并且

- ▶ 00:13 by the end of this lecture we will define and train your own net and you'll get to see
到本讲结束时，我们将定义并训练你自己的网络，你会看到
- ▶ 00:17 everything that goes on under the hood and exactly
底层发生的所有事情，以及确切地
- ▶ 00:20 sort of how that works on an intuitive level.
从直观层面理解它是如何工作的。
- ▶ 00:22 Now specifically what I would like to do is I would like to take you through
具体来说，我想做的事情是带你完整走一遍
- ▶ 00:26 building of `micrograd`. Now `micrograd` is this library that I released on GitHub about two years ago
构建 `micrograd` 的过程。`micrograd` 是我大约两年前在 GitHub 上发布的一个库

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 00:31 but at the time I only uploaded the source code and you'd have to go in by yourself and really
但当时我只上传了源代码，你必须自己深入进去，真正
- ▶ 00:37 figure out how it works. So in this lecture I will take you through it step by step and kind of comment
on all the pieces of it.
弄清楚它是如何工作的。所以在本讲中，我会一步一步带你走一遍，并对其中的各个部分做一些讲解。
- ▶ 00:43 So what is `micrograd` and why is it interesting?
那么 `micrograd` 是什么，它为什么有趣？

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 00:47 Good.
好。

- ▶ 00:49 Micrograd is basically an autograd engine. Autograd is short for automatic gradient and really what it does is it implements back propagation.

Micrograd 基本上就是一个自动微分引擎。Autograd 是 automatic gradient 的缩写，实际上它实现的就是反向传播。

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

autograd engine

自动微分引擎 能够根据计算图自动计算表达式中各变量梯度的引擎。

反向传播与梯度

- ▶ 00:57 Now back propagation is this algorithm that allows you to efficiently evaluate the gradient of 反向传播是这样一个算法：它让你能够高效地计算

back propagation

反向传播 利用链式法则从输出向输入高效计算损失函数梯度的算法。

反向传播与梯度

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 01:03 some kind of a loss function with respect to the weights of a neural network. 某个损失函数相对于神经网络权重的梯度。

loss function

损失函数 衡量模型预测与目标之间差异、训练中需要最小化的函数。

训练与优化

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

- ▶ 01:07 And what that allows us to do then is we can iteratively tune the weights of that neural network to minimize the loss function and

这样一来，我们就能迭代地调整该神经网络的权重，以最小化损失函数，并

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- ▶ 01:13 therefore improve the accuracy of the network.
因此提高网络的准确率。
- ▶ 01:16 So back propagation would be at the mathematical core of any modern deep neural network library like say PyTorch or JAX.

所以反向传播是任何现代深度神经网络库（比如 PyTorch 或 JAX）的数学核心。

back propagation

反向传播 利用链式法则从输出向输入高效计算损失函数梯度的算法。

反向传播与梯度

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

表达式图与反向传播示例 00:01

- ▶ 01:23 So the functionality of micrograd is I think best illustrated by an example. So if we just scroll down here
我认为用例子最能说明 micrograd 的功能。如果我们往下滚动一下

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 01:29 you'll see that micrograd basically allows you to build out mathematical expressions and
你会看到 micrograd 基本上允许你构建出数学表达式，并且

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 01:35 here what we are doing is we have an expression that we're building out where you have two inputs a and b and

这里我们做的是：我们正在构建一个表达式，其中有两个输入 a 和 b，然后

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 01:41 you'll see that a and b are negative 4 and 2, but we are wrapping those
你会看到 a 和 b 分别是 -4 和 2，但我们把这些

- ▶ 01:46 values into this `value object` that we are going to build out as part of `micrograd`.
值包装进这个 `value` 对象里，而这个 `value` 对象是我们作为 `micrograd` 的一部分要构建的。

value object

Value 对象 `micrograd` 中包装标量数值并记录计算图关系的对象。

代码与工程

micrograd

`micrograd` 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 01:51 So this `value object` will wrap the numbers themselves
所以这个 `value` 对象会包装数字本身

value object

Value 对象 `micrograd` 中包装标量数值并记录计算图关系的对象。

代码与工程

- ▶ 01:55 and then we are going to build out a mathematical `expression` here where `a` and `b` are
然后我们会在这里构建一个数学表达式，其中 `a` 和 `b` 被

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 01:59 transformed into `c`, `d` and eventually `e`, `f` and `g`.
转换成 `c`、`d`，最终变成 `e`、`f` 和 `g`。

- ▶ 02:04 And I'm showing some of the functionality of `micrograd` and the operations that it supports.
我这里展示的是 `micrograd` 的一些功能以及它支持的操作。

micrograd

`micrograd` 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 02:08 So you can add two `value` objects. You can multiply them. You can raise them to a `constant` power.
所以你可以把两个 `value` 对象相加。你可以把它们相乘。你可以把它们提升到一个常数幂。

value

Value 对象 `micrograd` 中表示标量及其梯度、参与计算图的基本类。

代码与工程

constant

常数 在表达式中固定的数值，如整数或浮点数。

数学与微积分

▶ 02:14 You can also subtract by 1, negate, squash at 0,
你也可以减 1、取负、在 0 处截断,

▶ 02:18 square, divide by constant, divide by it, etc.
平方、除以常数、除以它, 等等。

constant

常数 在表达式中固定的数值, 如整数或浮点数。

数学与微积分

▶ 02:22 And so we're building out an expression graph with these two inputs a and b and we're creating an output value of g

于是我们正在用这两个输入 a 和 b 构建一个表达式图, 并创建一个输出值 g

expression graph

表达式图 由运算节点和输入变量组成的数学表达式图, 用于描述计算过程。

反向传播与梯度

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

▶ 02:30 and micrograd will in the background build out this entire mathematical expression .
而 micrograd 会在后台构建出这整个数学表达式。

micrograd

micrograd 一个轻量级自动微分库, 用 Python 从零实现标量计算图与反向传播。

代码与工程

expression

表达式 由多个运算组合而成的计算式, 在计算图中对应多个节点。

代码与工程

▶ 02:35 So it will for example know that c is also a value .
所以它会例如知道 c 也是一个 value。

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

▶ 02:38 c was a result of an addition operation and the child nodes of c are a and b because the
c 是一次加法运算的结果, 而 c 的子节点是 a 和 b, 因为

- ▶ 02:46 and it will maintain pointers to a and b `value` objects. So we'll basically know exactly how all of 并且它会维护指向 a 和 b 这两个 `value` 对象的指针。所以我们基本上会确切知道所有

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 02:51 this is laid out. And then not only can we do what we call the `forward pass` where we actually look 这些是如何排布的。然后我们不仅可以做我们所说的前向传播，也就是实际查看

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

- ▶ 02:57 at the `value` of g of course that's pretty straightforward. We will access that using the g 的值——当然这很直接。我们会通过

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 03:01 `dot data attribute`. And so the output of the `forward pass` the `value` of g is 24.7 it turns out. `.data` 属性来访问它。所以前向传播的输出，也就是 g 的值，结果是 24.7。

dot data attribute

`.data` 属性 Value 对象上的 `.data` 属性，保存该节点包装的原始数值。

代码与工程

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

- ▶ 03:08 But the big deal is that we can also take this g `value object` and we can call dot `backward`. 但关键在于，我们还可以拿到这个 g value 对象，并调用 `.backward`。

value object

Value 对象 micrograd 中包装标量数值并记录计算图关系的对象。

代码与工程

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 03:14 And this will basically initialize **back propagation** at the **node g**.
而这基本上会在节点 g 上初始化反向传播。

back propagation

反向传播 利用链式法则从输出向输入高效计算损失函数梯度的算法。

反向传播与梯度

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

- ▶ 03:19 And what **backpropagation** is going to do is it's going to start at g and it's going to go
反向传播要做的是，它从 g 开始，然后会

backpropagation

反向传播 从输出节点开始，沿计算图反向递归应用链式法则，计算每个中间节点和输入对输出的导数。

反向传播与梯度

- ▶ 03:24 backwards through that **expression graph** and it's going to recursively apply the **chain rule** from
沿着那个表达式图反向传播，并递归地应用来自

expression graph

表达式图 由运算节点和输入变量组成的数学表达式图，用于描述计算过程。

反向传播与梯度

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 03:28 **calculus**. And what that allows us to do then is we're going to evaluate basically the **derivative**
微积分的链式法则。然后这让我们能够做的是，我们基本上要计算导数

calculus

微积分 研究变化率与累积量的数学分支。

数学与微积分

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 03:34 of g with respect to all the internal nodes like e d and c but also with respect to the inputs a and b.
g 对所有内部节点（比如 e、d 和 c）的导数，但也会对输入 a 和 b 求导。

- ▶ 03:43 And then we can actually query this **derivative** of g with respect to a for example that's a
然后我们实际上可以查询 g 对 a 的导数，比如那就是 a

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 03:48 **grad** in this case it happens to be 138 and the **derivative** of g with respect to b which
的 **grad**，在这个例子里它恰好是 138，还有 g 对 b 的导数，它

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 03:54 also happens to be here 645. And this **derivative** we'll see soon is very important information
也恰好在这里是 645。我们很快就会看到，这个导数是非常重要的信息

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 04:00 because it's telling us how a and b are affecting g through this mathematical **expression**. So in
因为它告诉我们 a 和 b 是如何通过这个数学表达式影响 g 的。所以

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 04:07 particular a dot **grad** is 138. So if we slightly **nudge** a and make it slightly larger 138 is telling
具体来说， $a.grad$ 是 138。所以如果我们稍微推动 a ，让它稍微大一点，138 是在告诉

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

nudge

微调 对输入做极小扰动，用来看函数输出如何变化。

数学与微积分

- ▶ 04:15 us that g will grow and the **slope** of that growth is going to be 138. And the **slope** of growth 我们 g 会增长，而这个增长的斜率将是 138。而增长的斜率

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

- ▶ 04:22 of b is going to be 645. So that's going to tell us about how g will respond if a and b get tweaked 对于 b 来说将是 645。所以这会告诉我们，如果 a 和 b 被微调

- ▶ 04:28 a tiny amount in a positive direction. Okay. Now you might be confused about what this 在正方向上极小的量， g 会如何响应。好了。现在你可能会困惑，这个

- ▶ 04:35 **expression** is that we built out here. And this **expression** by the way is completely meaningless. 我们在这里构建的表达式是什么。顺便说一句，这个表达式完全没有意义。

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 04:39 I just made it up. I'm just flexing about the kinds of operations that are supported by 我只是随便编的。我只是在炫耀那些被支持的操作种类，由

- ▶ 04:43 **micrograd**. What we actually really care about are **neural networks**. But it turns out that **micrograd** 支持。我们真正关心的是神经网络。但事实证明

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- ▶ 04:48 **neural networks** are just mathematical expressions just like this one, but actually slightly bit less 神经网络只是像这样的数学表达式，但实际上稍微没那么

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- ▶ 04:53 crazy even **neural networks** are just a mathematical **expression**. They take the input data as an input
疯狂，甚至神经网络也只是一个数学表达式。它们把输入数据作为输入

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 04:59 and they take the **weights** of a **neural network** as an input and it's a mathematical **expression**.
它们把神经网络的权重作为输入，而它是一个数学表达式。

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- ▶ 05:03 And the output are your predictions of your **neural net** or the **loss function**. We'll see this
输出则是你神经网络的预测结果或损失函数。我们稍后会看到

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

loss function

损失函数 衡量模型预测与目标之间差异、训练中需要最小化的函数。

训练与优化

- ▶ 05:08 in a bit. But basically **neural networks** just happen to be a certain class of mathematical
这一点。但基本上，神经网络恰好是一类数学

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- ▶ 05:12 expressions. But **backpropagation** is actually significantly more general. It doesn't actually
表达式。但反向传播实际上要通用得多。它其实并不

backpropagation

反向传播 从输出节点开始，沿计算图反向递归应用链式法则，计算每个中间节点和输入对输出的导数。

反向传播与梯度

- ▶ 05:17 care about **neural networks** at all. It only tells about arbitrary mathematical expressions.
关心神经网络。它只涉及任意数学表达式。

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- ▶ 05:22 And then we happen to use that machinery for training of **neural networks**. Now one more note I
然后我们恰好把这一机制用于训练神经网络。现在我还想再补充一点，我

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- ▶ 05:27 would like to make at this stage is that as you see here **micrograd** is a **scalar**-valued
想在这个阶段指出的是，如你在这里所见，micrograd 是一个标量值的

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

scalar

标量 单个数值，区别于向量、矩阵或更高维张量。

数学与微积分

- ▶ 05:31 **autograd engine**. So it's working on the, you know, level of individual scalars like negative
自动微分引擎。所以它处理的是，你知道，单个标量的层面，比如负

autograd engine

自动微分引擎 能够根据计算图自动计算表达式中各变量梯度的引擎。

反向传播与梯度

- ▶ 05:35 four and two. And we're taking **neural nets** and we're breaking them down all the way to these
四和二。我们把神经网络拿过来，一直拆解到这些

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 05:39 atoms of individual scalars and all the little pluses and times. And it's just excessive.
单个标量的原子，以及所有小小的加法和乘法。这实在是多余。

- ▶ 05:44 And so obviously you would never be doing any of this in production. It's really just for them
所以显然你在生产环境里永远不会做这些。这真的只是为了

▶ 05:48 for pedagogical reasons because it allows us to not have to deal with these n-dimensional
教学目的，因为它让我们不必处理这些 n 维的

▶ 05:53 tensors that you would use in modern deep **neural network** libraries. So this is really
张量，也就是你在现代深度神经网络库中会用到那种。所以这真的是

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

▶ 05:58 done so that you understand and factor out **backpropagation** and **chain rule** and understanding
为了让你理解并分离出反向传播、链式法则，以及理解

backpropagation

反向传播 从输出节点开始，沿计算图反向递归应用链式法则，计算每个中间节点和输入对输出的导数。

反向传播与梯度

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

▶ 06:03 of your training. And then if you actually want to train bigger networks, you have to be
你的训练过程。然后如果你真的想训练更大的网络，你就必须

▶ 06:07 using these tensors, but none of the math changes. This is done purely for efficiency.
使用这些张量，但数学本身没有任何变化。这样做纯粹是为了效率。

▶ 06:12 We are basically taking **scalar** values, all the **scalar** values, and packaging them up into tensors,
我们基本上就是取标量值，把所有标量值打包成张量，

scalar

标量 单个数值，区别于向量、矩阵或更高维张量。

数学与微积分

▶ 06:17 which are just arrays of these scalars. And then because we have these large arrays,
而张量就是这些标量的数组。然后因为我们有了这些大数组，

▶ 06:21 we're making operations on those large arrays that allows us to take advantage of the
我们在这些大数组上做运算，这让我们能够利用

▶ 06:26 **parallelism** in a computer. And all those operations can be done in parallel. And then the whole
计算机中的并行性。所有这些运算都可以并行完成。然后整个

parallelism

并行性 计算任务可同时执行以提升速度的能力，张量运算可利用它加速。

代码与工程

- ▶ 06:30 thing runs faster. But really none of the math changes and that's done purely for efficiency. 过程运行得更快。但实际上数学没有任何变化，这纯粹是为了效率。
- ▶ 06:35 So I don't think that it's pedagogically useful to be dealing with tensors from scratch. 所以我认为从零开始处理张量在教学上并没有太大用处。
- ▶ 06:39 And I think, and that's why I fundamentally wrote `micrograd` because you can understand how 而且我认为，这正是我写 `micrograd` 的根本原因，因为你可以理解如何

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 06:43 things work at the fundamental level. And then you can speed it up later. Okay, so here's the 这些东西在根本层面上是如何工作的。然后你之后可以加速它。好，那么接下来是
- ▶ 06:48 fun part. My claim is that `micrograd` is what you need to train your networks and everything 有趣的部分。我的主张是，`micrograd` 就是你训练网络所需要的东西，而其他

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 06:53 else is just efficiency. So you'd think that `micrograd` would be a very complex piece of code. 一切都只是效率问题。所以你会以为 `micrograd` 会是非常复杂的代码。

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 06:58 And that turns out to not be the case. So if we just go to `micrograd`, and you'll see 但事实证明并非如此。所以如果我们直接去看 `micrograd`，你会看到

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 07:04 that there's only two files here in `micrograd`. This is the actual engine. It doesn't know 这里 `micrograd` 里只有两个文件。这是实际的引擎。它并不知道

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 07:08 anything about `neural nets`. And this is the entire `neural nets` library on top of `micrograd`.
任何关于神经网络的东西。而这是建立在 `micrograd` 之上的整个神经网络库。

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 07:14 So engine and `nn.py`. So the actual `backpropagation` `autograd engine` that gives you the power of
所以是 engine 和 `nn.py`。所以这个实际的反向传播自动梯度引擎，赋给你

backpropagation

反向传播 从输出节点开始，沿计算图反向递归应用链式法则，计算每个中间节点和输入对输出的导数。

反向传播与梯度

autograd engine

自动微分引擎 能够根据计算图自动计算表达式中各变量梯度的引擎。

反向传播与梯度

- ▶ 07:22 `neural networks` is literally 100 lines of code of like very simple Python,
神经网络的能力，实际上就是 100 行非常简单的 Python 代码，

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- ▶ 07:30 which we'll understand by the end of this lecture. And then `nn.py`, this `neural network`
我们会在本讲结束时理解它。然后是 `nn.py`，这个神经网络

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- ▶ 07:34 library built on top of the `autograd engine` is like a joke. It's like we have to define what is a
库建立在自动梯度引擎之上，简直像个笑话。就好像我们得定义什么是

autograd engine

自动微分引擎 能够根据计算图自动计算表达式中各变量梯度的引擎。

反向传播与梯度

- ▶ 07:41 **neuron** . And then we have to define what is a **layer** of neurons. And then we define what is a multilayer

神经元。然后我们得定义什么是神经元的一层。然后我们定义什么是多层

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

- ▶ 07:46 **perceptron**, which is just a sequence of layers of neurons. And so it's just a total joke. So 感知机，它其实就是一个神经元层的序列。所以这完全是个笑话。所以

- ▶ 07:52 basically, there's a lot of power that comes from only 150 lines of code. And that's all you 基本上，只有 150 行代码，却能带来巨大的能力。而这就是你需要

- ▶ 07:58 need to understand to understand **neural network** training and everything else is just efficiency. 理解神经网络训练所需要理解的全部，其他一切都只是效率。

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- ▶ 08:02 And of course, there's a lot to efficiency. But fundamentally, that's all that's happening. 当然，效率方面有很多可讲。但从根本上说，发生的就只有这些。

- ▶ 08:07 Okay, so now let's dive right in and implement **micrograd** step by step. The first thing I'd like 好，那么现在让我们直接开始，一步步实现 micrograd。我想做的第一件事

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

数值导数与函数斜率 00:08

- ▶ 08:12 to do is I'd like to make sure that you have a very good understanding intuitively of what a 是，我想确保你对什么是

- ▶ 08:16 **derivative** is and exactly what information it gives you. So let's start with some basic 导数, 以及它究竟给你什么信息, 有一个非常好的直觉理解。所以让我们从一些基本

derivative

导数 描述函数在某一点的变化率, 即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 08:21 imports that I copy paste in every Jupyter notebook always. And let's define the function, 导入开始, 这些是我每个 Jupyter notebook 里总会复制粘贴的。然后让我们定义函数,

- ▶ 08:27 **scalar** **value** function f of x as follows. So I just made this up randomly, I just wanted a 标量值函数 $f(x)$ 如下。所以我只是随便编的, 我只是想要一个

scalar

标量 单个数值, 区别于向量、矩阵或更高维张量。

数学与微积分

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 08:33 **scalar-valued function** that takes a single **scalar** x and returns a single **scalar** y . And we can call 一个标量值函数, 它接收单个标量 x , 并返回单个标量 y 。而且我们可以调用

scalar-valued function

标量值函数 输入和输出都是单个标量 (实数) 的函数。

数学与微积分

scalar

标量 单个数值, 区别于向量、矩阵或更高维张量。

数学与微积分

- ▶ 08:39 this function of course, so we can pass in say 3.0 and get 20 back. Now, we can also plot this 这个函数, 比如传入 3.0, 就会得到 20。现在, 我们也可以绘制这个

- ▶ 08:45 function to get a sense of its shape. You can tell from the mathematical **expression** that this 函数, 来大致了解它的形状。从数学表达式可以看出, 这

expression

表达式 由多个运算组合而成的计算式, 在计算图中对应多个节点。

代码与工程

- ▶ 08:49 is probably a parabola, it's a `quadratic`. And so if we just create a set of `scalar` values that we can 很可能是一条抛物线，也就是二次函数。所以如果我们创建一组标量值，我们可以

quadratic

二次函数 最高次为二次的函数，图像为抛物线，例如 $3x^2 - 4x + 5$ 。

数学与微积分

scalar

标量 单个数值，区别于向量、矩阵或更高维张量。

数学与微积分

- ▶ 08:58 feed in using for example, `arange` from negative five to five and steps of 0.25. So this is, 例如用 `arange` 从 -5 到 5、步长为 0.25 来生成输入。所以这是，

arange

`arange` NumPy 中按起点、终点和步长生成等间隔数组的函数。

代码与工程

- ▶ 09:04 so `x` is just from negative five to five, not including five, in steps of 0.25. And we 所以 `x` 就是从 -5 到 5，不包含 5，步长为 0.25。然后我们

- ▶ 09:11 can actually call this function on this `NumPy array` as well. So we get a set of `y`'s if we 实际上也可以在这个 NumPy 数组上调用这个函数。所以如果我们

numpy array

NumPy 数组 NumPy 中的多维数组对象，可对整组标量做向量化运算。

代码与工程

- ▶ 09:15 call `f` on `x`. And these `y`'s are basically also applying function on every one of these elements 对 `x` 调用 `f`，就会得到一组 `y`。这些 `y` 基本上也就是把函数分别作用在每一个元素上，

- ▶ 09:24 independently. And we can plot this using `matplotlib`. So `plt.plot`, `x` and `y`'s, and we get 独立地。我们可以用 `matplotlib` 来绘制它。也就是 `plt.plot(x, y's)`，然后得到

matplotlib

`matplotlib` Python 中常用的绘图库，用于画函数曲线等。

代码与工程

- ▶ 09:30 nice parabola. So previously here we fed in 3.0 somewhere here, and we received 20 back, 一条漂亮的抛物线。所以之前我们在这里输入了 3.0，大概在这个位置，然后得到 20，

▶ 09:36 which is here the y coordinate. So now I'd like to think through what is the **derivative**
也就是这里的 y 坐标。现在我想仔细想一想，导数

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 09:42 of this function at any single input point x, right? So what is the **derivative** at different
这个函数在任意单个输入点 x 处的导数是什么，对吧？那么在不同

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 09:47 points x of this function? Now, if you remember back to your **calculus** class, you've probably derived
点 x 处，这个函数的导数是什么？现在，如果你还记得微积分课，你可能已经求过

calculus

微积分 研究变化率与累积量的数学分支。

数学与微积分

▶ 09:52 derivatives. So we take this mathematical **expression**, $3x^2$ minus $4x$ plus five, and you would write
导数。我们会取出这个数学表达式， $3x^2 - 4x + 5$ ，然后你会

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

▶ 09:57 out on a piece of paper and you would, you know, apply the product rule and all the other rules
在纸上把它写出来，然后，你知道，应用乘积法则以及其他各种法则，

▶ 10:01 and derive the mathematical **expression** of the **derivative** of the original function. And
并推导出原函数导数的数学表达式。然后

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 10:06 then you could plug in different x's and see what the **derivative** is. We're not going to 你就可以代入不同的 x，看看导数是多少。我们不会

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 10:10 actually do that because no one in **neural networks** actually writes out the **expression** for the neural 真的这么做，因为在神经网络中，没有人会真的写出神经

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

▶ 10:15 net. It would be a massive **expression**. It would be, you know, thousands, tens of thousands of terms. 网络的表达式。它会是一个非常庞大的表达式。它会有，你知道，成千上万、甚至几万项。

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

▶ 10:20 No one actually derives the **derivative**, of course. And so we're not going to take this kind of 当然，没有人真的会去推导导数。所以我们不会采用这种

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 10:24 I don't like the **symbolic approach**. Instead, what I'd like to do is I'd like to look at the definition 我不喜欢符号方法。相反，我想做的是，我想看看

symbolic approach

符号方法 直接对数学表达式进行符号运算来求导，而不是用数值近似。

数学与微积分

▶ 10:28 of **derivative** and just make sure that we really understand what **derivative** is measuring, 导数的定义，并确保我们真正理解导数在衡量什么，

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 10:32 what it's telling you about the function. And so if we just look at **derivative**, 它在告诉你关于函数的什么信息。所以如果我们只看导数,

derivative

导数 描述函数在某一点的变化率, 即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 10:42 we see that, okay, so this is not a very good definition of **derivative**. This is a definition 我们会看到, 好吧, 这并不是一个很好的导数定义。这是一个

derivative

导数 描述函数在某一点的变化率, 即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 10:46 of what it means to be **differentiable**. But if you remember from your **calculus**, it is the **limit**, 关于“可微”意味着什么的定义。但如果你还记得微积分, 它是极限,

differentiable

可微的 指函数在某一点存在导数, 即在该点可以用线性函数很好地近似。

数学与微积分

calculus

微积分 研究变化率与累积量的数学分支。

数学与微积分

- ▶ 10:50 as h goes to zero, of f of x plus h minus f of x over h . So basically what it's saying is, if you 当 h 趋于 0 时, $[f(x+h)-f(x)]/h$ 。所以基本上它说的是, 如果你

- ▶ 10:58 slightly bump up, you're at some point x that you're interested in or hey, and if you slightly 稍微增加一点, 你在某个感兴趣的点 x 处, 或者说嘿, 如果你稍微

- ▶ 11:03 bump up, you know, you slightly increase it by small number h , how does the function respond? 增加一点, 你知道, 你把它增加一个很小的数 h , 函数会如何响应?

- ▶ 11:09 With what **sensitivity** does it respond? What is the **slope** at that point? Does the function go up 它以多大的敏感度响应? 那一点的斜率是多少? 函数是上升

sensitivity

敏感度 函数输出对输入微小变化的响应程度, 在单变量中与导数直接相关。

数学与微积分

slope

斜率 函数图像在某一点切线的倾斜程度, 数值上等于该点的导数。

数学与微积分

- ▶ 11:13 or does it go down and by how much? And that's the **slope** of that function, the **slope** of that response

还是下降，以及变化多少？这就是函数在那一点的斜率，那个响应在那一点的斜率。

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

- ▶ 11:20 at that point. And so we can basically evaluate the **derivative** here numerically by taking a very
在那一点。所以我们可以取一个非常

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 11:26 small h . Of course, the definition would ask us to take h to zero. We're just going to pick
小的 h ，在这里数值地计算导数。当然，定义要求我们让 h 趋于 0。我们只是要选

- ▶ 11:31 a very small h , 0.001. And let's say we're interested in 3.0. So we can look at f of x ,
一个非常小的 h , 0.001。假设我们感兴趣的是 3.0。所以我们可以看 $f(x)$,

- ▶ 11:37 of course, as 20. And now f of x plus h , so if we slightly **nudge** x in a positive direction,
当然，是 20。现在看 $f(x+h)$ ，所以如果我们把 x 朝正方向稍微挪动一点，

nudge

微调 对输入做微小扰动，用来看函数输出如何变化。

数学与微积分

- ▶ 11:43 how is the function going to respond? And just looking at this, do you expect f of x plus h to
函数会如何响应？只看这个，你期望 $f(x+h)$

- ▶ 11:49 be slightly greater than 20? Or do you expect to be slightly lower than 20? And so since
略大于 20？还是期望它略低于 20？那么既然

- ▶ 11:55 3 is here and this is 20, if we slightly go positively, the function will respond positively.
3 在这里，而这是 20，如果我们稍微朝正方向走，函数会正向响应。

- ▶ 12:01 So you'd expect this to be slightly greater than 20. And now by how much is telling you the
所以你会期望它略大于 20。而现在，大多少就告诉你

- ▶ 12:06 sort of the strength of that **slope**, the size of the **slope**? So $f(x+h)$ minus $f(x)$, 这个斜率的强度, 斜率的大小? 所以 $f(x+h)$ 减去 $f(x)$,

slope

斜率 函数图像在某一点切线的倾斜程度, 数值上等于该点的导数。

数学与微积分

- ▶ 12:13 this is how much the function responded in a positive direction. And we have to normalize by 这就是函数在正方向上响应了多少。然后我们要把它归一化, 除以

- ▶ 12:18 the run. So we have the **rise over run** to get the **slope**. So this, of course, is just a numerical 水平变化量。所以我们用 **rise over run** 来得到斜率。当然, 这只是一个数值

rise over run

纵向变化除以横向变化 用两点间 $\Delta y / \Delta x$ 近似斜率, 是导数的数值计算起点。

数学与微积分

slope

斜率 函数图像在某一点切线的倾斜程度, 数值上等于该点的导数。

数学与微积分

- ▶ 12:25 approximation of the **slope**, because we have to make h very, very small to **converge** to the 斜率的近似, 因为必须让 h 非常非常小, 才能收敛到

slope

斜率 函数图像在某一点切线的倾斜程度, 数值上等于该点的导数。

数学与微积分

converge

收敛 训练过程中损失逐渐趋于稳定、不再明显下降的状态。

训练与优化

- ▶ 12:30 exact amount. Now if I'm doing too many zeros, at some point, I'm going to get an incorrect answer 精确值。现在如果我取太多零, 到了某个点, 就会得到不正确的答案,

- ▶ 12:38 because we're using **floating point arithmetic** and the representations of all these numbers 因为我们使用的是浮点运算, 而且所有这些数字的表示

floating point arithmetic

浮点运算 计算机以有限精度表示实数进行运算, 步长过小会因舍入误差导致结果不准确。

代码与工程

- ▶ 12:42 in computer memory is finite. And at some point, we get into trouble. So we can converge towards
在计算机内存中都是有限的。到了某个点，我们就会遇到麻烦。所以我们可以收敛到

converge

收敛 训练过程中损失逐渐趋于稳定、不再明显下降的状态。

训练与优化

- ▶ 12:47 the right answer with this approach. But basically, at three, the slope is 14. And you can see
用这种方法得到正确答案。但基本上，在 $x=3$ 处，斜率是 14。你可以看到

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

- ▶ 12:55 that by taking three x square minus four x plus five and differentiating it in our head. So three x
把 $3x^2 - 4x + 5$ 在脑子里求导就能看出来。所以 $3x$
- ▶ 13:01 square would be six x minus four. And then we plug in x equals three. So that's 18 minus four is 14.
平方项求导得到 $6x - 4$ 。然后代入 $x=3$ ，就是 $18 - 4 = 14$ 。
- ▶ 13:08 So this is correct. So that's at three. Now how about the slope at say negative three?
所以这是对的。所以那是在 $x=3$ 处。那么比如在 $x=-3$ 处的斜率呢？

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

- ▶ 13:17 Would you expect, would you expect for the slope? Now telling the exact value is really hard,
你会预期，你会预期斜率是多少？现在要说出精确值真的很难，

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 13:22 but what is the sign of that slope? So negative three, if we slightly go in the positive direction
但那个斜率的符号是什么？在 $x=-3$ 处，如果我们稍微往正方向移动

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

- ▶ 13:29 at x , the function would actually go down. And so that tells you that the **slope** would be negative.
x 稍微增大，函数实际上会下降。这就告诉你斜率会是负的。

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

- ▶ 13:33 So we'll get a **slope** number below, below -20 . And so if we take the **slope**, we expect something
所以我们会得到一个低于 -20 的斜率数值。那么如果看斜率，我们预期会得到

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

- ▶ 13:39 negative, negative 22 . Okay. And at some point here, of course, the **slope** would be zero.
负的， -22 。好的。当然，在这里某个点，斜率会变成零。

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

- ▶ 13:47 Now for this specific function, I looked it up previously, and it's at two over three.
对于这个具体的函数，我之前查过，它在 $2/3$ 处。

- ▶ 13:52 So at roughly two over three, that's somewhere here, this **derivative** would be zero. So basically at
所以大约在 $2/3$ 处，大概在这里，这个导数会是零。所以基本上在

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 14:00 that precise point, yeah, at that precise point, if we **nudge** in a positive direction, the function
那个精确的点，对，在那个精确的点，如果我们往正方向轻轻推一下，函数

nudge

微调 对输入做微小扰动，用来看函数输出如何变化。

数学与微积分

- ▶ 14:07 doesn't respond. This stays the same almost. And so that's why the **slope** is zero. Okay,
几乎不响应。它几乎保持不变。这就是为什么斜率是零。好的，

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

- ▶ 14:11 now let's look at a bit more complex case. So we're going to start, you know, 现在我们来查看一个稍微复杂一点的例子。所以我们要开始，你知道，
- ▶ 14:15 complexifying a bit. So now we have a function here with output variable D, that is a function 稍微把它复杂化。现在这里有一个函数，输出变量是 D，它是一个函数
- ▶ 14:23 of three scalar inputs, a, b, and c. So a, b, and c are some specific values, three inputs into our 由三个标量输入 a、b、c 组成。所以 a、b、c 是一些具体的值，是进入我们的

scalar inputs

标量输入 单个数值形式的输入，而不是向量或张量。

神经网络基础

- ▶ 14:29 expression graph, and a single output d. And so if we just print d, we get four. And now 表达式图，并产生单个输出 d。所以如果我们直接打印 d，就会得到 4。现在

expression graph

表达式图 由运算节点和输入变量组成的数学表达式图，用于描述计算过程。

反向传播与梯度

手动推导链式法则 00:14

- ▶ 14:36 what I'd like to do is I'd like to again look at the derivatives of d with respect to a, b, 我想做的是，再次来看 d 对 a、b、
- ▶ 14:41 and c. And think through, again, just the intuition of what this derivative is telling us. 和 c 的导数。并再次想一想，这个导数在直观上告诉我们什么。

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 14:47 So in order to evaluate this derivative, we're going to get a bit hacky here. We're going to again have 所以为了计算这个导数，我们这里会用一个有点取巧的办法。我们又会会有一个

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 14:53 a very small **value** of h . And then we're going to fix the inputs at some values that we're 非常小的 h 值。然后我们会把输入固定在一些我们

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 14:59 interested in. So these are the, this is the point a, b, c at which we're going to be evaluating 感兴趣的值上。所以这些就是，这就是点 a, b, c ，我们要在这个点上计算

- ▶ 15:04 the **derivative** of d with respect to all a, b , and c at that point. So there are the inputs. 在该点上 d 对 a, b, c 全部的导数。这些就是输入。

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 15:10 And now we have $d1$ is that **expression**. And then we're going to, for example, look at the **derivative** 现在我们有 $d1$ 是这个表达式。然后我们会，比如说，看

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 15:15 of d with respect to a . So we'll take a , and we'll bump it by h , and then we'll get $d2$ to be the d 对 a 的导数。所以我们会取 a ，把它增加 h ，然后我们得到 $d2$ 是

- ▶ 15:21 exact same function. And now we're going to print, you know, $d1$ is $d1$, $d2$ is $d2$, and print 完全相同的函数。现在我们要打印，你知道， $d1$ 是 $d1$ ， $d2$ 是 $d2$ ，并打印

- ▶ 15:33 **slope**. So the **derivative** or **slope** here will be, of course, $d2$ minus $d1$ divided by h . So $d2$ minus $d1$ is 斜率。所以这里的导数或斜率当然就是 $d2$ 减 $d1$ 再除以 h 。所以 $d2$ 减 $d1$ 就是

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 15:45 how much the function increased when we bumped the, the specific input that we're interested in
当我们把感兴趣的那个具体输入增加时，函数增加了多少

▶ 15:54 by a tiny amount. And this is then normalized by h to get the **slope**. So, yeah. So this, so I just found
一个微小的量。然后用 h 归一化，就得到斜率。所以，嗯。所以这个，所以我刚发现

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

▶ 16:08 this, we're going to print $d1$, which we know is four. Now $d2$ will be bumped, a will be bumped by h .
这个，我们要打印 $d1$ ，我们知道它是 4。现在 $d2$ 会变化， a 会被增加 h 。

▶ 16:20 So let's just think through a little bit what $d2$ will be printed out here. In particular, $d1$ will be four.
所以我们稍微想一下，这里打印出来的 $d2$ 会是多少。特别地， $d1$ 会是 4。

▶ 16:31 Will $d2$ be a number slightly greater than four or slightly lower than four? And it's going to
 $d2$ 会是一个略大于 4 的数，还是略小于 4 的数？这将

▶ 16:36 tell us the **sign of the derivative**. So we're bumping a by h , b is minus three, c is 10.
告诉我们导数的符号。所以我们将 a 增加 h ， b 是负三， c 是 10。

sign of the derivative

导数的符号 表示函数随输入增大而增大还是减小。

数学与微积分

▶ 16:47 So you can just in particular think through this **derivative** and what it's doing. a will be
所以你可以特别地想一想这个导数以及它在做什么。 a 会

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 16:52 slightly more positive. And, but b is a negative number. So if a is slightly more positive,
稍微更偏正一些。而且，不过 b 是一个负数。所以如果 a 稍微更偏正一些，

▶ 17:00 because b is negative three, we're actually going to be adding less to D .
因为 b 是负三，所以我们实际上会给 D 加得更少。

▶ 17:08 So you'd actually expect that the **value** of the function will go down. So let's just see this.
所以你其实会预期函数值会下降。那我们来看看。

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 17:15 Yeah. And so we went from four to 3.9996. And that tells you that the **slope** will be negative. 是的。于是我们从 4 变到了 3.9996。这就告诉你斜率会是负的。

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

- ▶ 17:23 And then will be a negative number because we went down. And then the exact number of **slope** 然后它会是一个负数，因为我们下降了。然后斜率的精确数值

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

- ▶ 17:30 will be exact amount of **slope** is negative three. And you can also convince yourself that negative 会是，斜率的精确数值是负三。你也可以说服自己，负

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

- ▶ 17:35 three is the right answer mathematically and analytically. Because if you have a times b 三是数学上和分析上正确的答案。因为如果你有 a 乘 b

- ▶ 17:39 plus c and you are, you know, you have **calculus**, then differentiating a times b plus c with respect 加 c，并且你，你知道，你学过微积分，那么对 a 乘 b 加 c 关于

calculus

微积分 研究变化率与累积量的数学分支。

数学与微积分

- ▶ 17:46 to a gives you just b. And indeed the **value** of b is negative three, which is the **derivative** that we a 求导，结果就是 b。而 b 的值确实是负三，这正是我们得到的

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 17:52 have. So you can tell that that's correct. So now if we do this with b, so if we bump b by a 导数。所以你可以说这是正确的。那么现在如果我们对 b 做这件事，如果我们把 b

- ▶ 17:58 little bit in a positive direction, we'd get different slopes. So what is the influence of b
往正方向稍微增大一点，我们会得到不同的斜率。那么 b 对输出 D 的影响是什么？
- ▶ 18:04 on the output D? So if we bump b by a tiny amount in a positive direction, then because a is positive,
所以如果我们把 b 往正方向增大一点点，那么因为 a 是正的，
- ▶ 18:11 we'll be adding more to D. Right? So, and now what is the, what is the sensitivity? What is the slope
我们就会给 D 加得更多。对吧？那么现在，敏感度是什么？那个加法的

sensitivity

敏感度 函数输出对输入微小变化的响应程度，在单变量中与导数直接相关。

数学与微积分

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

- ▶ 18:18 of that addition? And it might not surprise you that this should be two. And why is it two?
斜率是什么？你可能不会惊讶，这应该是 2。为什么是 2？
- ▶ 18:25 Because dD/db , differentiating with respect to b would be, would give us a and the value
因为 dD/db ，对 b 求导，会得到 a，而 a 的

differentiating with respect to

对.....求导 计算函数关于某个指定变量的导数。

数学与微积分

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 18:32 of a is two. So that's also working well. And then if c gets bumped a tiny amount by h,
值是 2。所以这也成立。然后如果 c 被增大一点点，增量为 h，
- ▶ 18:39 then of course, a times b is unaffected. And now c becomes slightly bit higher. What does that do
那么当然，a 乘 b 不受影响。而现在 c 变得稍微高一点。这对函数
- ▶ 18:44 to the function? It makes it slightly bit higher because we're simply adding c. And it makes it
有什么影响？它会让函数稍微高一点，因为我们只是加了 c。并且它会让函数
- ▶ 18:49 slightly bit higher by the exact same amount that we added to c. And so that tells you that the
稍微高一点，增加的量正好等于我们加到 c 上的量。所以这告诉你

▶ 18:54 **slope** is one. That will be the **rate** at which D will increase as we scale c. Okay, so we now have some 斜率是 1。这将是 D 随 c 缩放而增加的速率。好，我们现在对

slope

斜率 函数图像在某一点切线的倾斜程度，数值上等于该点的导数。

数学与微积分

rate

变化率 一个量随另一个量变化而变化的快慢。

数学与微积分

▶ 19:06 intuitive sense of what this **derivative** is telling you about the function. And we'd like to move to 这个导数在告诉你函数的什么有了一些直观理解。接下来我们想转到

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 19:10 **neural networks**. Now, as I mentioned, **neural networks** will be pretty massive expressions, 神经网络。现在，正如我提到的，神经网络会是相当庞大的表达式，

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

▶ 19:13 mathematical expressions. So we need some data structures that maintain these expressions. And 数学表达式。所以我们需要一些数据结构来维护这些表达式。而

▶ 19:17 that's what we're going to start to build out now. So we're going to build out this **Value** 这就是我们现在要开始构建的东西。所以我们要构建这个 Value

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

▶ 19:23 object that I showed you in the README page of **micrograd**. So let me copy paste a skeleton 对象，我在 micrograd 的 README 页面里给你们展示过。让我复制粘贴一个骨架

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

定义标量值类与运算 00:19

- ▶ 19:30 of the first very simple `Value object`. So class `Value` takes a single `scalar value` that it wraps
也就是第一个非常简单的 Value 对象。class Value 接收一个它包装并跟踪的单个标量值

value object

Value 对象 micrograd 中包装标量数值并记录计算图关系的对象。

代码与工程

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 19:37 and keeps track of. And that's it. So we can, for example, do `Value` of 2.0, and then we can
并跟踪这个标量值。就这样。比如我们可以执行 `Value(2.0)`，然后我们可以

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 19:45 get we can look at its content. And Python will internally use the `wrapper function`
获取、我们可以查看它的内容。Python 会在内部使用包装函数

wrapper function

包装函数 对底层方法做一层包装的函数，这里用来生成更易读的表达式字符串。

代码与工程

- ▶ 19:52 to return this string. So this is a `Value object` with data equals 2 that we're creating here.
来返回这个字符串。所以这里我们创建的是一个 data 等于 2 的 Value 对象。

value object

Value 对象 micrograd 中包装标量数值并记录计算图关系的对象。

代码与工程

- ▶ 20:03 Now we'd like to do is like, we'd like to be able to
现在我们想做的是，我们希望能够

- ▶ 20:07 have not just like two values, but we'd like to do A plus B, right? We'd like to add them.
不只是有两个值，而是想做 A 加 B，对吧？我们想把它们相加。

- ▶ 20:13 So currently, you would get an error because Python doesn't know how to add two **Value**
所以目前你会得到一个错误，因为 Python 不知道如何把两个 Value

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 20:18 objects. So we have to tell it. So here's addition. So you have to basically use these special
对象相加。所以我们必须告诉它。这里是加法。所以基本上你必须使用这些特殊的

- ▶ 20:28 **double underscore methods** in Python to define these **operators** for these objects. So if we call
Python 双下划线方法来为这些对象定义这些运算符。所以如果我们调用

double underscore methods

双下划线方法 Python 中以 `_` 开头和结尾的特殊方法，用于定制对象的内置行为。

代码与工程

operators

运算符 对数值或对象执行运算的符号，例如 `+`、`-`、`*`、`/`。

代码与工程

- ▶ 20:35 the, if we use this plus operator, Python will internally call `a.__add__(b)`. That's what will
这个，如果我们使用这个加号运算符，Python 会在内部调用 `a.__add__(b)`。这就是内部将会

- ▶ 20:44 happen internally. And so B will be the other and self will be a. And so we see that what we're
发生的事情。所以 B 会是 other，而 self 会是 a。于是我们看到，我们

- ▶ 20:51 going to return is a new **Value object**. And it's just, it's going to be wrapping the plus of
将要返回的是一个新 Value 对象。而且它只是、它将会包装它们的

value object

Value 对象 micrograd 中包装标量数值并记录计算图关系的对象。

代码与工程

- ▶ 20:58 their data. But remember now, because data is the actual Python number. So this
data 的加法。但现在要记住，因为 data 就是实际的 Python 数字。所以这个

- ▶ 21:04 operator here is just the typical floating point addition now, it's not an addition of Value
运算符现在只是典型的浮点加法，它不再是 Value

floating point addition

浮点加法 按浮点数表示规则执行的加法运算，与整数或精确实数加法不同。

数学与微积分

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 21:10 objects. And we'll return a new Value. So now a plus B should work. And it should print Value of
对象的加法。我们会返回一个新的 Value。现在 A 加 B 应该能工作了。而且它应该打印 Value，值为

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 21:17 negative one, because that's two plus minus three. There we go. Okay, let's now implement multiply,
负一，因为那是 2 加上 -3。好了。好，现在让我们实现乘法，

- ▶ 21:24 just so we can recreate this expression here. So multiply, I think it won't surprise you,
只是为了能在这里复现这个表达式。那么乘法，我想不会让你意外，

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 21:29 will be fairly similar. So instead of add, we're going to be using mul. And then here,
会非常相似。所以这里不用 add，而是用 mul。然后这里，

- ▶ 21:34 of course, we want to do times. And so now we can create a C value object, which will be 10.0.
当然，我们要做 times。现在可以创建一个 C 的 Value 对象，值为 10.0。

value object

Value 对象 micrograd 中包装标量数值并记录计算图关系的对象。

代码与工程

- ▶ 21:40 And now we should be able to do a times B. Well, let's just do a times B first.
现在应该能做 a times B。好吧，先做一下 a times B。

- ▶ 21:48 That's **value** of negative six now. And by the way, I skipped over this a little bit.
现在它的值是负六。顺便说一下，我刚刚跳过了一点。

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 21:52 Suppose that I didn't have the **wrapper function** here, then it's just that you'll get some kind
假设我这里没有这个包装函数，那么你会得到某种

wrapper function

包装函数 对底层方法做一层包装的函数，这里用来生成更易读的表达式字符串。

代码与工程

- ▶ 21:57 of an ugly **expression**. So what wrapper is doing is it's providing us a way to print out like
难看的表达式。所以 wrapper 所做的，就是给我们提供一种打印出

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 22:02 a nicer looking **expression** in Python. So we don't just have something cryptic, we actually are,
更好看表达式的方式，在 Python 里。这样我们就不会只看到某种晦涩的东西，实际上我们看到的是，

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 22:08 you know, it's **value** of negative six. So this gives us a times. And then this, we should now be able
你知道，它的值是负六。所以这给了我们 a times。然后这个，我们现在应该可以

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 22:16 to add C to it, because we've defined and told the Python how to do mul and add. And so this
把 C 加进去，因为我们已经定义并告诉 Python 如何做 mul 和 add。所以这

- ▶ 22:21 will call this will basically be equivalent to a dot mul of B. And then this new **value object**
会调用.....这基本上等价于 a dot mul of B。然后这个新的 Value 对象

value object

Value 对象 micrograd 中包装标量数值并记录计算图关系的对象。

代码与工程

- ▶ 22:29 will be dot add of C. And so let's see if that worked. Yep, so that worked well, that gave us four, 会做 dot add of C。让我们看看行不行。是的，成功了，得到了四，
- ▶ 22:36 which is what we expect from before. And I believe you can just call them manually as well. There 和我们之前预期的一样。而且我相信你也可以手动调用它们。
- ▶ 22:42 we go. So yeah. Okay, so now what we are missing is the connective tissue of this `expression`. As I 好了。所以现在缺少的是这个表达式的连接组织。正如我

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 22:49 mentioned, we want to keep these `expression` graphs. So we need to know and keep pointers 提到的，我们想保留这些表达式图。所以我们需要知道并保留指针，

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 22:53 about what values produce what other values. So here, for example, we are going to introduce a new 记录哪些值产生了哪些其他值。所以这里，比如，我们要引入一个新

- ▶ 22:59 variable, which we'll call `children`. And by default, it will be an empty tuple. And then we're actually 变量，我们叫它 `children`。默认情况下，它是一个空元组。然后我们实际上

children

子节点 产生当前值的输入值集合，用来建立计算图中的连接关系。

反向传播与梯度

- ▶ 23:04 going to keep a slightly different variable in the class, which we'll call `underscore prev`, 会在类里保存一个稍微不同的变量，我们叫它 `underscore prev`，

- ▶ 23:08 which will be the set of `children`. This is how I did it in the original `micrograd` looking at my code here. I can't remember exactly the reason I believe it was efficiency. But this 它会是 `children` 的集合。这就是我在原始 `micrograd` 里看着我的代码这么做的。我记不清确切原因了，我觉得是为了效率。但这个

children

子节点 产生当前值的输入值集合，用来建立计算图中的连接关系。

反向传播与梯度

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

▶ 23:14 at my code here. I can't remember exactly the reason I believe it was efficiency. But this 我记不清为什么我相信它效率。

▶ 23:19 `_children` will be a tuple for convenience. But then when we actually maintain `_children` 为了方便会是一个元组。但当我们实际维护

children

子节点 产生当前值的输入值集合，用来建立计算图中的连接关系。

反向传播与梯度

▶ 23:22 it in the class, it will be just a set. I believe for efficiency. So now, when we are creating a 它在类里时，它只会是一个集合。我相信这是为了效率。所以现在，当我们创建一个

▶ 23:31 `value` like this with a `constructor`, `children` will be empty, and `_prev` will be the empty set. 像这样用构造函数创建的值时，`children` 会是空的，`_prev` 也会是空集。

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

constructor

构造函数 创建对象时自动调用的初始化方法，用于设置实例属性。

代码与工程

▶ 23:35 But when we're creating a `value` through addition or multiplication, we're going to feed in the 但当我们通过加法或乘法创建一个值时，我们会传入这个值的

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

▶ 23:40 `children` of this `value`, which in this case is self and other. So those are the `children` `children`，在这个例子中就是 self 和 other。所以这些就是 `children`

children

子节点 产生当前值的输入值集合，用来建立计算图中的连接关系。

反向传播与梯度

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 23:48 here. So now we can do `d._prev`. And we'll see that the `children` of `d` we now know are this 在这里。所以现在我们可以做 `d._prev`。我们会看到，我们现在知道的 `d` 的 `children` 是这个

`_prev`

前驱集合 当前值所依赖的子节点集合，反向传播时沿该集合向上追溯。

反向传播与梯度

`children`

子节点 产生当前值的输入值集合，用来建立计算图中的连接关系。

反向传播与梯度

- ▶ 23:57 `value` of negative six and `value` of 10. And this of course is the `value` resulting from `a times b` 值为负六的值和值为 10 的值。当然，这是 `a` 乘以 `b` 得到的结果，

`value`

Value 对象 `micrograd` 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 24:03 and the `c value`, which is 10. Now, the last piece of information we don't know. So we know 以及 `c` 值，也就是 10。现在，最后一块我们不知道的信息。所以我们现在

`value`

Value 对象 `micrograd` 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 24:09 now the `children` of every single `value`, but we don't know what operation created this `value`. 知道了每个值的 `children`，但不知道是什么操作创建了这个值。

`children`

子节点 产生当前值的输入值集合，用来建立计算图中的连接关系。

反向传播与梯度

`value`

Value 对象 `micrograd` 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 24:14 So we need one more element here. Let's call it `_op`. And by default, this is the `empty set` 所以这里还需要一个元素。我们叫它 `_op`。默认情况下，它是空集

`_op`

操作符 记录产生当前值所用的运算字符串（如 `+` 或 `*`），用于选择对应的反向传播规则。

代码与工程

`empty set`

空集 不包含任何元素的集合，叶子节点的 `_prev` 默认就是空集。

数学与微积分

▶ 24:21 for leaves. And then we'll just maintain it here. And now the operation will be just a simple string.
对于叶子节点。然后我们就在这里维护它。现在操作只是一个简单的字符串。

▶ 24:28 And in the case of addition, it's plus in the case of multiplication is times.
如果是加法，它就是 plus；如果是乘法，它就是 times。

▶ 24:33 So now we not just have d._prev, we also have a d._op. And we know that d was produced
所以现在我们有 d._prev，还有 d._op。我们知道 d 是由

_prev

前驱集合 当前值所依赖的子节点集合，反向传播时沿该集合向上追溯。

反向传播与梯度

_op

操作符 记录产生当前值所用的运算字符串（如 + 或 *），用于选择对应的反向传播规则。

代码与工程

▶ 24:40 by an addition of those two values. And so now we have the full mathematical expression.
那两个值相加产生的。所以现在我们有完整的数学表达式。

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

▶ 24:45 And we're building out this data structure. And we know exactly how each value came to be
我们正在构建这个数据结构。我们确切地知道每个值是如何

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

▶ 24:50 by what expression and from what other values. Now, because these expressions are about to get
由什么表达式、从哪些其他值产生的。现在，因为这些表达式将要变得

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

▶ 24:56 quite a bit larger, we'd like a way to nicely visualize these expressions that we're building
大不少，我们想要一种能很好地可视化我们正在构建的这些表达式的方法。

▶ 25:01 out. So for that, I'm going to copy paste a bunch of slightly scary code that's going to
为此，我要复制粘贴一堆稍微有点吓人的代码，它会

可视化表达式图 00:25

- ▶ 25:06 visualize this, these `expression` graphs for us. So here's the code and I'll explain it in a bit.
为我们可视化这些表达式图。所以这就是代码，我稍后会解释。

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 25:12 But first, let me just show you what this code does. Basically, what it does is it creates
但首先，让我先展示一下这段代码做了什么。基本上，它做的就是创建

- ▶ 25:16 a new function `draw_dot` that we can call on some root `node`. And then it's going to visualize it.
一个新函数 `draw_dot`，我们可以对某个根节点调用它。然后它会把图可视化出来。

draw_dot

`draw_dot` micrograd 中把 Value 计算图导出为 Graphviz 图形的辅助函数。

代码与工程

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

- ▶ 25:22 So if we call `draw_dot` on `d`, which is this final `value` here, that is $a \times b + c$,
所以如果我们对 `d` 调用 `draw_dot`，而 `d` 是这里的最终值，也就是 a 乘以 b 加 c ，

draw_dot

`draw_dot` micrograd 中把 Value 计算图导出为 Graphviz 图形的辅助函数。

代码与工程

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 25:29 it creates something like this. So this is `d`. And you see that this is $a \times b$,
它就会生成类似这样的图。所以这是 `d`。你看，这是 a 乘以 b ，

- ▶ 25:34 creating an intermediate `value` plus `c` gives us this `output node` `d`.
它创建一个中间值，再加上 `c` 就得到这个输出节点 `d`。

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

output node

输出节点 计算图中最终产生输出的节点。

神经网络基础

- ▶ 25:39 So that's `draw_dot` on `d`. And I'm not going to go through this in complete detail. You can take a
所以这就是对 `d` 调用 `draw_dot`。我不会完整详细地讲解这个。你可以

draw_dot

`draw_dot` micrograd 中把 Value 计算图导出为 Graphviz 图形的辅助函数。

代码与工程

- ▶ 25:44 look at `Graphviz` and its API. `Graphviz` is an open source `graph` visualization software.
去看 Graphviz 及其 API。Graphviz 是一个开源的图可视化软件。

graphviz

Graphviz 一个开源的图可视化软件，常用来把计算图渲染成图片。

代码与工程

graph

计算图 由变量和运算节点组成的有向无环图，记录计算过程并支持反向传播。

神经网络基础

- ▶ 25:49 And what we're doing here is we're building out this `graph` in `Graphviz` API. And you can
我们在这里做的是用 Graphviz API 构建出这个图。你可以

graph

计算图 由变量和运算节点组成的有向无环图，记录计算过程并支持反向传播。

神经网络基础

graphviz

Graphviz 一个开源的图可视化软件，常用来把计算图渲染成图片。

代码与工程

- ▶ 25:56 basically see that `trace` is this helper function that enumerates all of the nodes and edges
基本上看到 `trace` 是一个辅助函数，它枚举图中的所有节点和边

- ▶ 26:00 in the `graph`. So that just builds a set of all the nodes and edges. And then we iterate in the graph. So it just builds a set of all the nodes and edges. And then we iterate

graph

计算图 由变量和运算节点组成的有向无环图，记录计算过程并支持反向传播。

神经网络基础

- ▶ 26:05 for all the nodes and we create special `node` objects for them using `dot.node`.
对于所有节点，我们用 `dot.node` 为它们创建特殊的节点对象。

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

- ▶ 26:13 And then we also create edges using `dot.edge`. And the only thing that's like slightly tricky here
然后我们也用 `dot.edge` 创建边。这里唯一稍微有点棘手的地方

- ▶ 26:18 is you'll notice that I basically add these fake nodes, which are these operation nodes.
是你会注意到，我基本上添加了这些假节点，也就是这些操作节点。

- ▶ 26:23 So for example, this `node` here is just like a `plus node`. And I create these
比如，这里的这个节点就是一个加号节点。然后我创建这些

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

plus node

加法节点 计算图中执行加法运算的节点，其局部导数对该节点的每个输入均为 1。

神经网络基础

- ▶ 26:30 special op nodes here. And I connect them accordingly. So these nodes, of course,
特殊的 op 节点。然后我把它相应地连接起来。所以这些节点，当然，

- ▶ 26:38 are not actual nodes in the original `graph`. They're not actually a `Value object`. The only
并不是原始图中的实际节点。它们实际上不是 Value 对象。这里唯一

graph

计算图 由变量和运算节点组成的有向无环图，记录计算过程并支持反向传播。

神经网络基础

value object

Value 对象 micrograd 中包装标量数值并记录计算图关系的对象。

代码与工程

- ▶ 26:44 **Value** objects here are the things in squares. Those are actual **Value** objects or representations of the Value 对象是方框中的东西。那些才是实际的 Value 对象，或者它们的表示

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 26:49 thereof. And these op nodes are just created in this **draw_dot** routine, so that it looks nice. 而这些 op 节点只是在这个 draw_dot 例程中创建的，让图看起来更漂亮。

draw_dot

draw_dot micrograd 中把 Value 计算图导出为 Graphviz 图形的辅助函数。

代码与工程

- ▶ 26:55 Let's also add labels to these graphs just so we know what variables are where. 我们还要给这些图加上标签，这样我们就知道哪个变量在哪个位置。

- ▶ 26:59 So let's create a special underscore label. Or let's just do label equals empty by default 所以我们来创建一个特殊的下划线 label。或者我们干脆就让 label 默认等于空字符串

- ▶ 27:07 and save it in each **node**. And then here we're going to do label is a, label is b, label is c. 然后把它保存在每个节点里。接着这里我们会写 label 是 a, label 是 b, label 是 c。

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

- ▶ 27:19 And then let's create E. E equals a times B. 然后让我们创建 E。E 等于 a 乘以 b。

- ▶ 27:30 And label will be E. It's kind of naughty. And D will be E plus C. And the label will be D. 标签会是 E。这有点调皮。然后 D 将等于 E 加 C。标签会是 D。

前向传播与手动反向传播 00:27

- ▶ 27:42 Okay, so nothing really changes. I just added this new E function, 好的，所以其实没什么变化。我只是添加了这个新的 E 函数，
- ▶ 27:46 a new E variable. And then here when we are printing this, I'm going to print the label here. 一个新的 E 变量。然后在这里当我们打印这个时，我会在这里打印标签。

▶ 27:54 So this will be a percent S bar. And this will be the label. And so now
所以这将是百分号 S 竖线。然后这将是标签。所以现在

▶ 28:03 we have the label on the left here. So it says a times b creates e and then e plus c creates d,
我们在左边这里有标签。所以它显示 a 乘以 b 创建 e, 然后 e 加 c 创建 d,

▶ 28:08 just like we have it here. And finally, let's make this expression just one layer deeper.
就像我们这里有的那样。最后, 让我们把这个表达式再深入一层。

expression

表达式 由多个运算组合而成的计算式, 在计算图中对应多个节点。

代码与工程

layer

层 一组独立计算的神经元, 通常将同一维度的输入映射到多个输出。

神经网络基础

▶ 28:14 So D will not be the final output node. Instead, after D, we are going to create a new value object
所以 D 不会是最终的输出节点。相反, 在 D 之后, 我们将创建一个新的 Value 对象

output node

输出节点 计算图中最终产生输出的节点。

神经网络基础

value object

Value 对象 micrograd 中包装标量数值并记录计算图关系的对象。

代码与工程

▶ 28:21 called F. We're going to start running out of variables soon. F will be negative 2.0.
叫做 F。我们很快就要用完变量了。F 将是 -2.0。

▶ 28:27 And its label will of course just be F. And then L capital L will be the output of our graph.
当然它的标签就是 F。然后 L (大写的 L) 将是我们图的输出。

graph

计算图 由变量和运算节点组成的有向无环图, 记录计算过程并支持反向传播。

神经网络基础

▶ 28:35 And L will be D times F. So L will be negative eight is the output.
然后 L 将等于 D 乘以 F。所以 L 将是 -8, 这就是输出。

▶ 28:41 So now we don't just draw a D, we draw L. Okay. And somehow the label of L is undefined.
所以现在不只是画 D, 我们画 L。好的。不知怎么的, L 的标签是未定义的。

▶ 28:54 Oops. All that label has to be explicitly given to it. There we go. So L is the output.
哎呀。所有标签都必须显式地赋予它。好了。所以 L 是输出。

▶ 29:01 So let's quickly recap what we've done so far. We are able to build out mathematical expressions
所以让我们快速回顾一下到目前为止我们做了什么。我们能够构建数学表达式

▶ 29:06 using only plus and times so far. They are `scalar` valued along the way. And we can do this forward
目前只使用加法和乘法。它们沿途都是标量值。我们可以进行这个前向

scalar

标量 单个数值，区别于向量、矩阵或更高维张量。

数学与微积分

▶ 29:13 pass and build out a mathematical `expression`. So we have multiple inputs here, A, B, C and F,
传递并构建出一个数学表达式。所以我们这里有多输入，A、B、C 和 F，

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

▶ 29:19 going into a mathematical `expression` that produces a single output L. And this here
进入一个数学表达式，产生单个输出 L。而这这里

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

▶ 29:25 is visualizing the `forward pass`. So the output of the `forward pass` is negative eight. That's the
正在可视化前向传递。所以前向传递的输出是 -8。那就是

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

▶ 29:30 `value`. Now what we'd like to do next is we'd like to run `back propagation`. And in `back propagation`,
值。现在接下来我们想做的是运行反向传播。在反向传播中，

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

back propagation

反向传播 利用链式法则从输出向输入高效计算损失函数梯度的算法。

反向传播与梯度

- ▶ 29:36 we are going to start here at the end, and we're going to reverse and calculate the **gradient**
我们将从末尾这里开始，然后反向并计算梯度

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 29:42 along all these intermediate values. And really what we're computing for every single **value** here,
沿着所有这些中间值。实际上，我们在这里对每一个值所计算的，

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 29:49 we're going to compute the **derivative** of that **node** with respect to L. So the **derivative** of L
我们将计算该节点关于 L 的导数。所以 L

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

- ▶ 29:57 with respect to L is just one. And then we're going to derive what is the **derivative** of L with
关于 L 的导数就是一。然后我们将推导 L 关于

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 30:02 respect to F with respect to D with respect to C with respect to E with respect to B and with respect
F、D、C、E、B 以及

- ▶ 30:09 to A. And in **neural network** setting, you'd be very interested in the **derivative** of basically this
A 的导数。在神经网络场景中，你会非常关心基本上这个

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 30:14 **loss function** L with respect to the **weights** of a **neural network**. And here, of course, we have 损失函数 L 关于神经网络权重的导数。当然，这里我们有

loss function

损失函数 衡量模型预测与目标之间差异、训练中需要最小化的函数。

训练与优化

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

▶ 30:20 just these variables A , B , C and F . But some of these will eventually represent the **weights** 只是这些变量 A 、 B 、 C 和 F 。但其中一些最终会表示权重

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

▶ 30:24 of a **neural net**. And so we'll need to know how those **weights** are impacting the **loss function**. So 神经网络的。所以我们需要知道这些权重如何影响损失函数。所以

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

▶ 30:30 we'll be interested basically in the **derivative** of the output with respect to some of its **leaf nodes**. 我们基本上会关心输出关于它的一些叶子节点的导数。

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

leaf node

叶子节点 计算图中没有输入边的节点，通常代表输入变量或可训练参数。

神经网络基础

- ▶ 30:35 And those **leaf nodes** will be the **weights** of the **neural net**. And the other **leaf nodes**, of course, 那些叶子节点将是神经网络的权重。而另一些叶子节点，当然，

leaf node

叶子节点 计算图中没有输入边的节点，通常代表输入变量或可训练参数。

神经网络基础

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

- ▶ 30:39 will be the data itself. But usually we will not want or use the **derivative** of the **loss function** 将是数据本身。但通常我们不会想要或使用损失函数

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

loss function

损失函数 衡量模型预测与目标之间差异、训练中需要最小化的函数。

训练与优化

- ▶ 30:44 with respect to data, because the data is fixed, but the **weights** will be iterated on 关于数据的导数，因为数据是固定的，但权重会

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

- ▶ 30:50 using the **gradient** information. So next, we are going to create a variable inside the **Value class** 使用梯度信息进行迭代。所以接下来，我们将在 Value 类中创建一个变量

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

value class

Value 类 micrograd 中表示标量节点及其梯度与反向传播函数的类。

代码与工程

- ▶ 30:54 that maintains the **derivative** of L with respect to that **value**. And we will call this variable 来维护 L 关于该值的导数。我们将把这个变量称为

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 31:02 **grad**. So there's a dot data, and there's a self dot **grad**. And initially, it will be zero. And remember grad. 所以有一个 dot data，还有一个 self dot grad。初始时，它将为零。记住

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 31:09 that zero is basically means no effect. So at initialization, we're assuming that every **value** 零基本上意味着没有影响。所以在初始化时，我们假设每个值

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 31:15 does not impact does not affect the output, right? Because if the **gradient** zero, that means that 不影响也不改变输出，对吧？因为如果梯度为零，那就意味着

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 31:21 changing this variable is not changing the **loss function**. So by default, we assume that the **gradient** 改变这个变量不会改变损失函数。所以默认情况下，我们假设梯度

loss function

损失函数 衡量模型预测与目标之间差异、训练中需要最小化的函数。

训练与优化

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 31:27 is zero. And then now that we have `grad`, and it's 0.0, we are going to be able to visualize it here 为零。然后现在有了 `grad`，它是 0.0，我们将能够在这里可视化它

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 31:38 after data. So here `grad` is 0.0. And this will be in the `grad`. And now we are going to be showing 在 data 之后。所以这里 `grad` 是 0.0。这将在 `grad` 中。现在我们将展示

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 31:47 both the data and the `grad` initialized at zero. And we are just about getting ready to 数据和梯度都初始化为零。我们正准备开始

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 31:55 calculate the `backpropagation`. And of course, this `grad` again, as I mentioned, is representing 计算反向传播。当然，正如我提到的，这个梯度再次表示

backpropagation

反向传播 从输出节点开始，沿计算图反向递归应用链式法则，计算每个中间节点和输入对输出的导数。

反向传播与梯度

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 32:00 the `derivative` of the output, in this case, L, with respect to this `value`. So with respect to, 输出（在这个例子中是 L）关于这个值的导数。所以关于

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 32:06 so this is the **derivative** of L with respect to F , respect to D , and so on. So let's now fill in
所以这是 L 关于 F 的导数，关于 D 的导数，等等。现在让我们填充

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 32:11 those **gradients** and actually do **backpropagation** manually. So let's start filling in these
这些梯度，并真正手动进行反向传播。让我们开始填充这些

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

backpropagation

反向传播 从输出节点开始，沿计算图反向递归应用链式法则，计算每个中间节点和输入对输出的导数。

反向传播与梯度

- ▶ 32:15 **gradients** and start all the way at the end, as I mentioned here. First, we are interested to fill in
梯度，并像这里提到的那样从最后开始。首先，我们想要填充

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 32:19 this **gradient** here. So what is the **derivative** of L with respect to L ? In other words, if I change L
这里的梯度。那么 L 关于 L 的导数是什么？换句话说，如果我改变 L

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 32:26 by a tiny amount of h , how much does L change? It changes by h . So it's proportional and therefore
一个很小的量 h ， L 会改变多少？它改变 h 。所以是成比例的，因此

- ▶ 32:35 the **derivative** will be one. We can of course measure these or estimate these numerical **gradients**
导数将是 1。我们当然可以测量或估计这些数值梯度

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 32:41 numerically, just like we've seen before. So if I take this **expression** and I create a def
数值上，就像我们之前看到的。所以如果我取这个表达式，然后我创建一个 def

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 32:46 lol function here and put this here. Now the reason I'm creating a def lol function here
lol 函数放在这里，并把这个放进去。现在我创建一个 def lol 函数的原因是

- ▶ 32:53 is because I don't want to pollute or mess up the **global scope** here. This is just kind of like
因为我不想污染或搞乱这里的全局作用域。这只是一个

global scope

全局作用域 在 Python 中，函数外部可访问的命名空间，函数内部变量默认不会影响它。

代码与工程

- ▶ 32:57 a little staging area. And as you know, in Python, all of these will be local variables to
小暂存区。如你所知，在 Python 中，所有这些都会成为这个函数的局部变量

- ▶ 33:01 this function. So I'm not changing any of the **global scope** here. So here, L1 will be L.
这个函数的局部变量。所以我没有改变这里的任何全局作用域。那么这里，L1 将是 L。

global scope

全局作用域 在 Python 中，函数外部可访问的命名空间，函数内部变量默认不会影响它。

代码与工程

- ▶ 33:10 And then copy pasting this **expression**, we're going to add a small amount h in, for example, A.
然后复制粘贴这个表达式，我们将添加一个很小的量 h，比如加到 A 上。

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 33:20 Right. And this would be measuring the derivative of L with respect to A.
对。这将测量 L 关于 A 的导数。

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 33:25 So here, this will be L2. And then we want to print test derivative. So print L2 minus L1,
所以这里，这将是 L2。然后我们想打印测试导数。所以打印 L2 减去 L1，

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 33:32 which is how much L changed and then normalize it by h. So this is the rise over run.
即 L 改变了多少，然后除以 h 进行归一化。这就是斜率（rise over run）。

rise over run

纵向变化除以横向变化 用两点间 $\Delta y / \Delta x$ 近似斜率，是导数的数值计算起点。

数学与微积分

- ▶ 33:39 And we have to be careful because L is a value node. So we actually want its data.
我们必须小心，因为 L 是一个值节点。所以我们实际上想要它的 data。

value node

值节点 micrograd 中存储数据 data 和梯度 grad 的计算图节点。

代码与工程

- ▶ 33:46 So that these are floats dividing by h. And this should print the derivative of L with respect to A
这样这些就是浮点数，再除以 h。这应该打印出 L 关于 A 的导数

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 33:52 because a is the one that we bumped a little bit by h. So what is the derivative of L with respect
因为我们把 a 稍微增加了 h。那么 L 关于 a 的导数是多少

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 33:58 to a? It's 6. Okay. And obviously, if we change L by h, then that would be here effectively.
关于 a? 是 6。好。显然，如果我们把 L 改变 h，那么实际上这里就是那样。

- ▶ 34:12 This looks really awkward, but changing L by h, you see the **derivative** here is 1.
这看起来很奇怪，但把 L 改变 h，你看这里的导数是 1。

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 34:20 That's kind of like the base case of what we are doing here. So basically, we come up here
这有点像我们这里所做的基准情况。所以基本上，我们走到这里

- ▶ 34:26 and we can manually set L.grad to 1. This is our manual **back propagation**.
然后我们可以手动把 L.grad 设为 1。这就是我们手动反向传播。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

back propagation

反向传播 利用链式法则从输出向输入高效计算损失函数梯度的算法。

反向传播与梯度

- ▶ 34:31 L.grad is 1. And let's redraw. And we'll see that we filled in **grad** is 1 for L.
L.grad 是 1。让我们重画一下。我们会看到我们把 L 的 grad 填成了 1。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 34:39 We're now going to continue the **back propagation**. So let's here look at the derivatives of L
我们现在继续反向传播。那么让我们在这里看 L 的导数

back propagation

反向传播 利用链式法则从输出向输入高效计算损失函数梯度的算法。

反向传播与梯度

- ▶ 34:43 with respect to d and f. Let's do d first. So what we are interested in, if I create a markdown
关于 d 和 f 的导数。先做 d。我们感兴趣的是，如果我在这里创建一个 markdown

- ▶ 34:50 on here, is we'd like to know, basically, we have that L is d times f. And we'd like to know
我们想知道的是，基本上，L 等于 d 乘以 f。我们想知道

- ▶ 34:55 what is dL by dd ? What is that? And if you know your **calculus**, L is d times f . So what is dL by dd ? dL/dd 是多少? 那是什么? 如果你懂微积分, L 是 d 乘以 f 。那么 dL/dd 是多少?

calculus

微积分 研究变化率与累积量的数学分支。

数学与微积分

- ▶ 35:06 It would be f . And if you don't believe me, we can also just derive it because the proof would 它就是 f 。如果你不相信我, 我们也可以直接推导它, 因为证明会

- ▶ 35:11 be fairly straightforward. We go to the **definition of the derivative**, which is f of x plus h 相当直接。我们回到导数的定义, 也就是 $f(x+h)$

definition of the derivative

导数的定义 用极限 $f(x+h)-f(x)$ 除以 h 来定义导数的方式。

数学与微积分

- ▶ 35:19 minus f of x divided by h as a **limit** as h goes to zero of this kind of **expression**. So when we have L as 减去 $f(x)$ 除以 h , 当 h 趋于 0 时这种表达式的极限。所以当我们有 L 等于

limit

极限 当自变量趋于某个值时函数值趋近的目标, 导数的定义就是差商的极限。

数学与微积分

expression

表达式 由多个运算组合而成的计算式, 在计算图中对应多个节点。

代码与工程

- ▶ 35:26 d times f , then increasing d by h would give us the output of d plus h times f . That's basically d 乘以 f , 那么把 d 增加 h 会得到 $(d+h)$ 乘以 f 的输出。那基本上就是

- ▶ 35:36 f of x plus h , right? Minus d times f . And then divide h . And symbolically, expanding out here, $f(x+h)$, 对吧? 减去 d 乘以 f 。然后除以 h 。符号化地, 在这里展开,

- ▶ 35:46 we would have basically d times f plus h times f minus d times f divided h . And then you see how the 我们基本上会得到 d 乘以 f 加上 h 乘以 f 减去 d 乘以 f 除以 h 。然后你会看到

- ▶ 35:53 $d f$ minus $d f$ cancels. So you're left with h times f divided h , which is f . So in the **limit** as h goes $d f$ 减去 $d f$ 抵消了。所以你剩下 h 乘以 f 除以 h , 也就是 f 。所以在极限中, 当 h 趋于

limit

极限 当自变量趋于某个值时函数值趋近的目标, 导数的定义就是差商的极限。

数学与微积分

- ▶ 36:01 to zero of, you know, **derivative** definition, we just get f in a case of d times f .
0 时, 你知道, 根据导数定义, 在 d 乘以 f 的情况下, 我们得到 f 。

derivative

导数 描述函数在某一点的变化率, 即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 36:11 So symmetrically, dL by df will just be d . So what we have is that f . **grad**, we see now,
对称地, dL/df 就是 d 。所以我们现在看到 $f.grad$

grad

梯度 计算图中某个变量累积的导数, 表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 36:21 is just the **value** of d , which is four. And we see that d . **grad** is just the **value** of f .
就是 d 的值, 也就是 4。我们看到 $d.grad$ 就是 f 的值。

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

grad

梯度 计算图中某个变量累积的导数, 表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 36:33 And so the **value** of F is negative two. So we'll set those manually. Let me erase this markdown
所以 F 的值是负二。这些我们就手动设置。让我把这个 markdown

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 36:45 **node** and then let's redraw what we have. Okay. And let's just make sure that these were correct.
节点擦掉, 然后我们把目前的内容重新画一下。好。我们再确认一下这些是不是对的。

node

节点 计算图中的基本单元, 表示一个值或一次运算。

神经网络基础

- ▶ 36:52 So we seem to think that dL by dD is negative two. So let's double check. Let me erase this
我们似乎认为 dL/dD 是负二。那我们再检查一下。让我把这个

- ▶ 36:59 plus h from before. And now we want the **derivative** with respect to F. So let's just come here when 之前那个加 h。现在我们要的是对 F 的导数。我们来到这边，当

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 37:04 I create F and let's do a plus h here. And this should print the **derivative** of L with respect 我创建 F 时，这里做一个加 h。这应该会打印出 L 对

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 37:10 to F. So we expect to see four. Yeah, and this is four up to floating point funkiness. F 的导数。所以我们期望看到 4。没错，在浮点数那点怪毛病范围内，这就是 4。

- ▶ 37:18 And then dL by dD should be F, which is negative two. **Grad** is negative two. So if we again come here 然后 dL/dD 应该等于 F，也就是负二。梯度是负二。所以如果我们再次来到这里

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 37:28 and we change D, D.data += h right here. So we expect, so we've added a little h 然后我们改一下 D，也就是在这里写 D.data += h。所以我们期望，我们加了一个小小的 h

- ▶ 37:37 and then we see how L changed and we expect to print negative two. There we go. So we've 然后看看 L 怎么变化，我们期望打印出负二。成了。所以我们已经

- ▶ 37:47 numerically verified, what we're doing here is what's kind of like an inline **gradient** check **gradient** check is 做了数值验证。我们这里做的，有点像是在做内联的梯度检查。梯度检查就是

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

▶ 37:53 when we are deriving this like **back propagation** and getting the **derivative** with respect to all
当我们推导这个，像反向传播那样，求出对所有这些

back propagation

反向传播 利用链式法则从输出向输入高效计算损失函数梯度的算法。

反向传播与梯度

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 37:58 the intermediate results. And then **numerical gradient** is just, you know, estimating it
中间结果的导数。然后数值梯度只是，你知道，估计它

numerical gradient

数值梯度 通过微小扰动输入并计算输出变化来近似估计的梯度。

数学与微积分

▶ 38:03 using small **step size**. Now we're getting to the crux of **back propagation**. So this will be the most
用小步长来估计。现在我们进入反向传播的关键部分。所以这将是

step size

步长 数值梯度估计中用于扰动输入的小量，通常记为 h 。

数学与微积分

back propagation

反向传播 利用链式法则从输出向输入高效计算损失函数梯度的算法。

反向传播与梯度

▶ 38:10 important **node** to understand because if you understand the **gradient** for this **node**, you understand
all

最重要的节点，因为如果你理解了这个节点的梯度，你就理解了所有

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 38:15 of **back propagation** and all training on **neural nets**, basically. So we need to derive dL by dC .
反向传播, 以及基本上所有神经网络的训练。所以我们需要推导 dL/dC 。

back propagation

反向传播 利用链式法则从输出向输入高效计算损失函数梯度的算法。

反向传播与梯度

neural net

神经网络 由多层可学习参数组成的函数逼近器, 用于从输入预测输出。

神经网络基础

- ▶ 38:23 In other words, the **derivative** of L with respect to C , because we've computed all these
换句话说, 就是 L 对 C 的导数, 因为我们已经计算了所有

derivative

导数 描述函数在某一点的变化率, 即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 38:27 other **gradients** already. Now we're coming here and we're continuing the **back propagation**
其他梯度。现在我们来到这里, 继续反向传播

gradient

梯度 损失函数对参数的变化率, 指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

back propagation

反向传播 利用链式法则从输出向输入高效计算损失函数梯度的算法。

反向传播与梯度

- ▶ 38:32 manually. So we want dL by dC and then we'll also derive dL by dE . Now here's the problem.
手动继续。所以我们要求 dL/dC , 然后我们还会推导 dL/dE 。现在问题来了。

链式法则与反向传播 00:38

- ▶ 38:40 How do we derive dL by dC ? We actually know the **derivative** of L with respect to D . So we know how
我们怎么推导 dL/dC ? 我们其实已经知道 L 对 D 的导数。所以我们知道

derivative

导数 描述函数在某一点的变化率, 即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 38:48 L is sensitive to D . But how is L sensitive to C ? So if we wiggle C , how does that impact L
 L 对 D 有多敏感。但 L 对 C 有多敏感呢? 所以如果我们稍微拨动 C , 它会如何影响 L

- ▶ 38:55 through D? So we know dL by dD and we also here know how C impacts D. And so just very intuitively, 经过 D? 所以我们知道 dL/dD , 而且这里也知道 C 如何影响 D。所以非常直观地,
- ▶ 39:05 if you know the impact that C is having on D and the impact that D is having on L, 如果你知道 C 对 D 的影响, 以及 D 对 L 的影响,
- ▶ 39:11 then you should be able to somehow put that information together to figure out how C impacts L. 那么你应该能够以某种方式把这些信息组合起来, 弄清楚 C 如何影响 L。
- ▶ 39:16 And indeed, this is what we can actually do. So in particular, we know just concentrating 确实, 这就是我们实际上能做的事。特别地, 我们知道, 只要先集中
- ▶ 39:21 on D first. Let's look at how what is the **derivative** basically of D with respect to C. 在 D 上。我们来看看 D 关于 C 的导数基本上是什么。

derivative

导数 描述函数在某一点的变化率, 即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 39:26 So in other words, what is dD by dC ? So here we know that D is C plus E. That's what we know. 换句话说, dD/dC 是什么? 这里我们知道 D 是 C 加 E。这就是我们知道的。
- ▶ 39:36 And now we're interested in dD by dC . If you just know your **calculus** again and you remember 现在我们对 dD/dC 感兴趣。如果你还记得微积分, 你就会记得

calculus

微积分 研究变化率与累积量的数学分支。

数学与微积分

- ▶ 39:41 that differentiating C plus E with respect to C, you know that that gives you 1.0. And we can 对 C 加 E 关于 C 求导, 会得到 1.0。我们还可以
- ▶ 39:48 also go back to the basics and derive this because again, we can go to our f of x plus h minus f of x 回到基础来推导它, 因为我们可以回到 $f(x+h)$ 减 $f(x)$
- ▶ 39:54 divided by h as the **definition of a derivative** as h goes to zero. And so here focusing on C 除以 h , 作为 h 趋于零时导数的定义。所以这里关注 C

definition of a derivative

导数的定义 用极限 $f(x+h)-f(x)$ 除以 h 在 $h \rightarrow 0$ 时的值来严格定义导数。

数学与微积分

- ▶ 40:02 and its effect on D, we can basically do the f of x plus h will be C is incremented by h plus E. 以及它对 D 的影响, 我们基本上可以写出 $f(x+h)$ 会是 C 增加 h 再加 E。

- ▶ 40:10 That's the first evaluation of our function minus C plus E. And then divide by h.
这是我们函数的第一项求值，再减去 C 加 E。然后除以 h。
- ▶ 40:18 And so what is this? Just expanding this out. This will be C plus h plus E minus C minus E
那这是什么？把它展开。这会等于 C 加 h 加 E 减 C 减 E
- ▶ 40:25 divided by h. And then you see here how C minus C cancels, E minus E cancels we're left with h over h,
除以 h。然后你会看到，C 减 C 抵消，E 减 E 抵消，剩下 h 除以 h，
- ▶ 40:31 which is 1.0. And so by symmetry also, dD by dE will be 1.0 as well. So basically the **derivative**
也就是 1.0。由对称性，dD/dE 也会是 1.0。所以基本上，导数

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 40:44 of a **sum expression** is very simple. And this is the **local derivative**. So I call this the local
对于一个求和表达式来说非常简单。这就是局部导数。我称之为局部

sum expression

求和表达式 由加法构成的表达式，例如 $D = C + E$ ；其对各输入的局部导数为 1。

数学与微积分

local derivative

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

- ▶ 40:49 **derivative** because we have the final output **value** all the way at the end of this **graph**. And we're
导数，因为最终的输出值在计算图的末端。而我们

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 40:53 now like a small **node** here. And this is a little **plus node**. And it the little **plus node** doesn't 现在就像一个小节点。这是一个小小的加法节点。这个小小的加法节点并不知道

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

plus node

加法节点 计算图中执行加法运算的节点，其局部导数对该节点的每个输入均为 1。

神经网络基础

- ▶ 40:59 know anything about the rest of the **graph** that it's embedded in. All it knows is that it did 关于它所在的计算图的其余部分，它什么都不知道。它只知道它做了

graph

计算图 由变量和运算节点组成的有向无环图，记录计算过程并支持反向传播。

神经网络基础

- ▶ 41:04 a plus, it took a C and an E added them and created D. And this **plus node** also knows the 一个加法，它取了一个 C 和一个 E，把它们相加并创建了 D。而且这个加法节点还知道

plus node

加法节点 计算图中执行加法运算的节点，其局部导数对该节点的每个输入均为 1。

神经网络基础

- ▶ 41:10 local influence of C on D, or rather the **derivative** of D with respect to C. And it also knows the C 对 D 的局部影响，更准确地说，是 D 关于 C 的导数。而且它也知道

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 41:17 **derivative** of D with respect to E. But that's not what we want. That's just the **local derivative**. D 关于 E 的导数。但那不是我们想要的。那只是局部导数。

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

local derivative

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

- ▶ 41:22 What we actually want is dL by dC . And L could, L is here just one step away. But in a general 我们真正想要的是 dL/dC 。而 L 嘛，L 在这里只差一步。但在一般

- ▶ 41:30 case, this little **plus node** is could be embedded in like a massive **graph**. So again, we know how 情况下, 这个小小的加号节点可能嵌在一个巨大的图里。所以, 我们又一次知道了

plus node

加法节点 计算图中执行加法运算的节点, 其局部导数对该节点的每个输入均为 1。

神经网络基础

graph

计算图 由变量和运算节点组成的有向无环图, 记录计算过程并支持反向传播。

神经网络基础

- ▶ 41:36 L impacts D. And now we know how C and E impact D. How do we put that information together to arrive

L 如何影响 D。现在我们也知道了 C 和 E 如何影响 D。那我们怎么把这些信息组合起来, 得到

- ▶ 41:42 dL by dC ? And the answer, of course, is the **chain rule** in **calculus**. And so dL/dC 呢? 答案当然就是微积分中的链式法则。所以

chain rule

链式法则 微积分中计算复合函数导数的法则, 即外层函数的导数乘以内层函数的导数。

数学与微积分

calculus

微积分 研究变化率与累积量的数学分支。

数学与微积分

- ▶ 41:50 I pulled up a **chain rule** here from Wikipedia. And I'm going to go through this very briefly. 我从维基百科上找来了一段链式法则的内容。我打算非常简略地过一遍。

chain rule

链式法则 微积分中计算复合函数导数的法则, 即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 41:55 So **chain rule**, Wikipedia sometimes can be very confusing and **calculus** can be very confusing, 链式法则嘛, 维基百科有时候写得很让人犯晕, 微积分本身也很容易让人犯晕,

chain rule

链式法则 微积分中计算复合函数导数的法则, 即外层函数的导数乘以内层函数的导数。

数学与微积分

calculus

微积分 研究变化率与累积量的数学分支。

数学与微积分

- ▶ 42:01 like this is the way I learned **chain rule** and it was very confusing, like what is happening?
比如我当年就是这么学链式法则的，真的非常让人困惑，就是——这到底是什么鬼？

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 42:08 It's just complicated. So I like this **expression** much better.
它就是很复杂。所以我更喜欢这个表达式。

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 42:13 If a variable Z depends on a variable Y , which itself depends on the variable X ,
如果变量 Z 依赖于变量 Y ，而 Y 本身又依赖于变量 X ，

- ▶ 42:18 then Z depends on X as well, obviously through the **intermediate variable** Y .
那么 Z 也就依赖于 X ，当然是通过中间变量 Y 传递的。

intermediate variable

中间变量 在依赖链中起传递作用的变量，例如 $Z \rightarrow Y \rightarrow X$ 中的 Y 。

数学与微积分

- ▶ 42:22 And in this case, the **chain rule** is expressed as if you want dZ by dX ,
在这种情况下，链式法则就是说：如果你想要 dZ/dX ，

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 42:28 then you take the dZ by dY and you multiply it by dY by dX . So the **chain rule** fundamentally is telling
你就取 dZ/dY ，再乘以 dY/dX 。所以链式法则本质上就是在告诉

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 42:35 you how we chain these derivatives together correctly. So to **differentiate** through a function
你如何把这些导数正确地串起来。所以，要对一个函数

differentiate

求导 计算函数导数的过程，在反向传播中用于得到梯度。

反向传播与梯度

▶ 42:44 composition, we have to apply a multiplication of those derivatives. So that's really what 复合进行求导, 我们就必须把这些导数相乘。这其实就是

▶ 42:53 **chain rule** is telling us. And there's a nice little intuitive explanation here, which I also think is 链式法则要告诉我们的。这里还有一个很不错的直观解释, 我个人觉得也

chain rule

链式法则 微积分中计算复合函数导数的法则, 即外层函数的导数乘以内层函数的导数。

数学与微积分

▶ 42:58 kind of cute. The **chain rule** states that knowing the **instantaneous rate of change** of Z with respect 挺妙的。链式法则说的是, 只要知道 Z 相对于

chain rule

链式法则 微积分中计算复合函数导数的法则, 即外层函数的导数乘以内层函数的导数。

数学与微积分

instantaneous rate of change

瞬时变化率 函数在某一点处的变化快慢, 也就是该点处的导数值。

数学与微积分

▶ 43:02 to Y and Y relative to X allows one to calculate the **instantaneous rate of change** of Z relative to Y 的瞬时变化率, 以及 Y 相对于 X 的瞬时变化率, 就可以算出 Z 相对于

instantaneous rate of change

瞬时变化率 函数在某一点处的变化快慢, 也就是该点处的导数值。

数学与微积分

▶ 43:06 X as a product of those two rates of change, simply the product of those two. So here's a X 的瞬时变化率, 也就是这两个变化率的乘积, 就是简单的乘积。那么这里有一个

▶ 43:13 good one. If a car travels twice as fast as a bicycle and the bicycle is four times as fast 很好。如果汽车的速度是自行车的两倍, 而自行车的速度是步行的人的四倍

▶ 43:18 as walking men, then the car travels two times four, eight times as fast as the men. And so this 也就是步行的人, 那么汽车的速度就是二乘四, 也就是人的八倍。所以这

▶ 43:25 makes it very clear that the correct thing to do sort of is to multiply. So cars twice as fast as 就非常清楚地表明, 正确的做法差不多就是相乘。所以汽车速度是

▶ 43:33 bicycle and bicycle is four times as fast as men. So the car will be eight times as fast as the 自行车的两倍, 而自行车是人的四倍。所以汽车将会是

▶ 43:39 men. And so we can take these intermediate rates of change, if you will, and multiply them 人的八倍。所以我们可以把这些中间的变化率, 如果你愿意这么说, 乘起来

- ▶ 43:45 together. And that justifies the **chain rule** intuitively. So have a look at **chain rule**. But here 乘在一起。这就直观地解释了链式法则。所以看一下链式法则。但这里

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 43:52 really what it means for us is there's a very simple recipe for deriving what we want, which is dL by dC . 它对我们真正意味着，有一个非常简单的推导方法可以得到我们想要的东西，也就是 dL/dC 。

- ▶ 43:59 And what we have so far is we know 而到目前为止我们所知道的是，我们知道

- ▶ 44:03 what we want and we know what is the impact of D on L . So we know dL by dD , the **derivative** L with respect

我们想要什么，而且我们知道 D 对 L 的影响。所以我们知道 dL/dD ，也就是 L 的导数，关于

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 44:14 to D . We know that that's negative two. And now because of this local reasoning that we've D . 我们知道那是负二。现在，因为这种局部推理，我们已经

- ▶ 44:20 done here, we know dD by dC . So how does C impact D ? And in particular, this is a **plus node**. So the 在这里完成了，我们知道 dD/dC 。那么 C 如何影响 D ? 特别是，这是一个加法节点。所以

plus node

加法节点 计算图中执行加法运算的节点，其局部导数对该节点的每个输入均为 1。

神经网络基础

- ▶ 44:29 **local derivative** is simply 1.0. It's very simple. And so the **chain rule** tells us that dL by dC , 局部导数就是 1.0。非常简单。所以链式法则告诉我们， dL/dC ,

local derivative

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 44:37 going through this intermediate variable, will just be simply dL by dD times dD by dC .
经过这个中间变量，就等于 dL/dD 乘以 dD/dC 。

intermediate variable

中间变量 在依赖链中起传递作用的变量，例如 $Z \rightarrow Y \rightarrow X$ 中的 Y 。

数学与微积分

- ▶ 44:51 That's chain rule. So this is identical to what's happening here, except Z is L , Y is D , and X is
这就是链式法则。所以这和这里发生的情况完全一样，只不过 Z 是 L ， Y 是 D ，而 X 是

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 45:02 C . So we literally just have to multiply these. And because these local derivatives like dD by dC are
 C 。所以我们实际上只需要把它们乘起来。而且因为这些局部导数，比如 dD/dC ，
- ▶ 45:12 just 1, we basically just copy over dL by dD because this is just times 1. So because dL by dD is negative
就是 1，我们基本上只是把 dL/dD 复制过来，因为这相当于乘以 1。所以因为 dL/dD 是负
- ▶ 45:22 2, what is dL by dC ? Well, it's the local gradient 1.0 times dL by dD , which is negative 2.
二，那么 dL/dC 是多少？它就是局部梯度 1.0 乘以 dL/dD ，也就是负二。

local gradient

局部梯度 某个运算节点对其输入变量的导数，只由该节点自身的运算决定。

反向传播与梯度

- ▶ 45:31 So literally what a plus node does, you can look at it that way, is it literally just routes the gradient
所以，加法节点实际上做的，你可以这样看，就是直接把梯度路由过去

plus node

加法节点 计算图中执行加法运算的节点，其局部导数对该节点的每个输入均为 1。

神经网络基础

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 45:37 because the plus nodes local derivatives are just 1. And so in the chain rule, 1 times dL by dD is
因为加法节点的局部导数就是 1。所以在链式法则中，1 乘以 dL/dD 就是

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 45:48 just dL by dD. And so that **derivative** just gets routed to both C and to E in this case.
就是 dL/dD 。所以这个导数在这种情况下就直接被路由到 C 和 E 两者。

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 45:55 So basically we have that E. **grad**, or let's start with C, since that's the one we've looked at, 所以基本上我们有 E.grad，或者让我们从 C 开始，因为那是我们之前看过的那个，

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 46:02 is negative 2 times 1, negative 2. And in the same way, by symmetry, E. **grad** will be negative 2.
是负 2 乘以 1，等于负 2。同样地，根据对称性，E.grad 也会是负 2。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 46:12 That's the claim. So we can set those, we can redraw, and you see how we just assign negative 2
这就是结论。所以我们可以设置这些，我们可以重画，然后你会看到我们如何直接把负 2

- ▶ 46:21 to negative 2. So this **backpropagating signal**, which is carrying the information of like, what
赋给负 2。所以这个反向传播信号，所携带的信息就像是，

backpropagating signal

反向传播信号 在计算图中从输出向输入反向传递的梯度信息，用来按链式法则计算各节点的导数。

反向传播与梯度

- ▶ 46:25 is the **derivative** of L with respect to all the intermediate nodes? We can imagine it almost
L 对所有中间节点的导数是什么？我们可以想象它几乎

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 46:30 like flowing backwards through the **graph**, and a **plus node** will simply distribute the **derivative** to 像在图中反向流动，而一个加法节点会简单地把导数分发给

graph

计算图 由变量和运算节点组成的有向无环图，记录计算过程并支持反向传播。

神经网络基础

plus node

加法节点 计算图中执行加法运算的节点，其局部导数对该节点的每个输入均为 1。

神经网络基础

- ▶ 46:34 all the **leaf nodes**, sorry, to all the **children** nodes of it. So this is the claim, and now 它所有的叶节点，抱歉，是它所有的子节点。所以这就是结论，现在

leaf node

叶子节点 计算图中没有输入边的节点，通常代表输入变量或可训练参数。

神经网络基础

children

子节点 产生当前值的输入值集合，用来建立计算图中的连接关系。

反向传播与梯度

- ▶ 46:40 let's verify it. So let me remove the plus h here from before. And now instead what we're 让我们验证它。所以让我把之前的 plus h 去掉。而现在相反，我们

- ▶ 46:45 going to do is we want to increment C. So C.data will be incremented by h. And when I run this, 要做的是增加 C。所以 C.data 会被 h 增加。当我运行这个时，

- ▶ 46:51 we expect to see negative 2. Negative 2. And then of course for E, so E.data **plus equals** h, 我们期望看到负 2。负 2。然后当然对于 E，所以 E.data plus equals h，

plus equals

加等于 (+=) 代码中在原值上累加的写法，用于实现梯度累加，避免覆盖已有梯度。

代码与工程

- ▶ 47:00 and we expect to see negative 2. Simple. So those are the derivatives of these internal nodes. 我们期望看到负 2。很简单。所以这些就是这些内部节点的导数。

- ▶ 47:11 And now we're going to recurse our way backwards again. And we're again going to apply the 现在我们要再次反向递归。我们再次应用

- ▶ 47:16 **chain rule**. So here we go, our second application of **chain rule**, and we will apply it all the way 链式法则。所以开始吧，我们第二次应用链式法则，我们会把它一路

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 47:21 through the **graph**, which just happened to only have one more **node** remaining. We have that DL 应用到整个图，而这里恰好只剩下一个节点。我们有 DL

graph

计算图 由变量和运算节点组成的有向无环图，记录计算过程并支持反向传播。

神经网络基础

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

- ▶ 47:27 by DE, as we have just calculated, is negative 2. So we know that. So we know the **derivative** of L 除以 DE，正如我们刚计算的，是负 2。所以我们知道这个。所以我们知道 L

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 47:33 with respect to E. And now we want DL by DA, right? And the **chain rule** is telling us that 对 E 的导数。现在我们想要 DL 除以 DA，对吧？链式法则告诉我们

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 47:44 that's just DL by DE, negative 2, times the **local gradient**. So what is the **local gradient**? 那只是 DL 除以 DE，负 2，乘以局部梯度。那么局部梯度是什么？

local gradient

局部梯度 某个运算节点对其输入变量的导数，只由该节点自身的运算决定。

反向传播与梯度

- ▶ 47:53 Basically DE by DA. We have to look at that. So I'm a little **times node** inside a massive **graph**.
基本上就是 DE 除以 DA。我们得看一下。所以我是这个巨大图里的小乘法节点。

times node

乘法节点 计算图中执行乘法的节点，某个输入的梯度等于另一输入的数据乘以输出梯度。

反向传播与梯度

graph

计算图 由变量和运算节点组成的有向无环图，记录计算过程并支持反向传播。

神经网络基础

- ▶ 48:04 And I only know that I did A times B, and I produced an E. So now what is DE by DA and DE
我只知道我做了 A 乘以 B，然后产生了 E。那么现在 DE 除以 DA 和 DE

- ▶ 48:11 by DB? That's the only thing that I sort of know about. That's my **local gradient**.
除以 DB 是什么？这就是我唯一大概知道的东西。这就是我的局部梯度。

local gradient

局部梯度 某个运算节点对其输入变量的导数，只由该节点自身的运算决定。

反向传播与梯度

- ▶ 48:17 So because we have that E is A times B, we're asking what is dE/dA ? And of course, we just
所以因为 E 等于 A 乘以 B，我们问的是 dE/dA 是多少？当然，我们刚刚

- ▶ 48:25 did that here. We had a times. So I'm not going to redraw it. But if you want to **differentiate**
在这里做过了。我们有 A 乘以 B。所以我不打算重画。但如果你想对

differentiate

求导 计算函数导数的过程，在反向传播中用于得到梯度。

反向传播与梯度

- ▶ 48:31 this with respect to A, you'll just get B, right? The **value** of B, which in this case is negative
这个式子关于 A 求导，你只会得到 B，对吧？B 的值，在这个例子中是负

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 48:38 3.0. So basically we have that dL/dA . Well, let me just do it right here. We have that A
3.0。所以基本上我们有 dL/dA 。好吧，让我就在这里做。我们有 A

- ▶ 48:47 **grad** and we are applying **chain rule** here is dL/dE , which we see here is negative 2 times $grad$, 我们在这里应用链式法则，就是 dL/dE ，我们在这里看到它是负 2 乘以

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 48:57 what is dE/dA ? It's the **value** of B , which is negative 3. That's it. And then we have B
 dE/dA 是多少？它是 B 的值，也就是负 3。就是这样。然后我们有 B

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 49:08 dot **grad** is again dL/dE , which is negative 2, just the same way times what is dE/dB
dot $grad$ 同样是 dL/dE ，也就是负 2，以同样的方式乘以 dE/dB 是多少

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 49:19 is the **value** of A , which is 2.0. That's the **value** of A . So these are our claimed derivatives.
是 A 的值，也就是 2.0。那就是 A 的值。所以这些就是我们声称的导数。

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 49:28 Let's redraw. And we see here that a. **grad** turns out to be 6 because that is negative 2 times
让我们重画一下。我们在这里看到 a.grad 结果是 6，因为那是负 2 乘以

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 49:37 negative 3. And b. **grad** is negative 2 times 2, which is negative 4. So those are our claims.
负 3。而 b.grad 是负 2 乘以 2，也就是负 4。所以这些就是我们的断言。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 49:46 Let's delete this and let's verify them. We have a here, $a.data += h$.
让我们删掉这个，然后验证它们。我们这里有 a ， $a.data += h$ 。

▶ 49:57 So the claim is that a `grad` is 6. Let's verify 6. And we have $b.data += h$.
所以断言是 $a.grad$ 是 6。让我们验证 6。然后我们有 $b.data += h$ 。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 50:08 So nudging b by h and looking at what happens, we claim it's negative 4 and indeed it's negative 4 plus
所以把 b 微调 h ，看看会发生什么，我们声称它是负 4，确实它是负 4 加上

▶ 50:17 minus again `float`, oddness. And that's it. That was the manual `backprop` application
减去，又是浮点数的小问题。就是这样。那就是手动反向传播的应用

float

浮点数 带小数部分的数值类型，与整数一起可作为标量输入。

数学与微积分

backprop

反向传播 从损失函数出发，沿计算图反向逐节点应用链式法则来计算梯度的算法。

反向传播与梯度

▶ 50:28 all the way from here to all the `leaf nodes`. And we've done it piece by piece.
从这里一路到所有叶子节点。我们是一步一步完成的。

leaf node

叶子节点 计算图中没有输入边的节点，通常代表输入变量或可训练参数。

神经网络基础

▶ 50:33 And really all we've done is as you saw, we iterated through all the nodes one by one
而实际上我们所做的就是，如你所见，我们逐个遍历所有节点

▶ 50:37 and locally applied the `chain rule`. We always know what is the `derivative` of L with respect to
并局部应用链式法则。我们总是知道 L 关于

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 50:42 this little output. And then we look at how this output was produced. This output was produced
这个小输出的导数是什么。然后我们看这个输出是如何产生的。这个输出是通过

▶ 50:47 through some operation. And we have the pointers to the **children** nodes of this operation.
某个操作产生的。而且我们有指向这个操作的子节点的指针。

children

子节点 产生当前值的输入值集合，用来建立计算图中的连接关系。

反向传播与梯度

▶ 50:52 And so in this little operation, we know what the local derivatives are.
所以在这个小操作中，我们知道局部导数是什么。

▶ 50:56 And we just multiply them onto the **derivative** always. So we just go through and recursively
我们总是把它们乘到导数上。所以我们只是遍历并递归地

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 51:01 multiply on the local derivatives. And that's what **backpropagation** is.
乘上局部导数。这就是反向传播。

backpropagation

反向传播 从输出节点开始，沿计算图反向递归应用链式法则，计算每个中间节点和输入对输出的导数。

反向传播与梯度

▶ 51:05 It's just a recursive application of **chain rule** backwards through the **computation graph**.
它不过是在计算图上反向递归地应用链式法则。

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

computation graph

计算图 把变量和运算表示为节点与边的有向图，用于前向计算和反向梯度传播。

神经网络基础

▶ 51:10 Let's see this power in action just very briefly. What we're going to do is we're going to
我们非常简短地看看这种能力如何发挥作用。我们要做的是，我们将

- ▶ 51:15 **nudge** our inputs to try to make L go up. So in particular what we're doing is we want 微调我们的输入，试图让 L 上升。具体来说，我们要做的是，我们希望

nudge

微调 对输入做极小扰动，用来看函数输出如何变化。

数学与微积分

- ▶ 51:21 a.data, we're going to change it. And if we want L to go up, that means we just a.data, 我们要改变它。如果我们想让 L 上升，那就意味着我们只需要

- ▶ 51:26 have to go in the direction of the **gradient**. So a should increase in the direction of **gradient** 沿着梯度的方向走。所以 a 应该沿着梯度方向增加

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 51:32 by like some small step amount. This is the **step size**. And we don't just want this for a , but also 增加大约某个小步长。这就是步长。我们不只是想对 a 这样做，还要

step size

步长 数值梯度估计中用于扰动输入的小量，通常记为 h 。

数学与微积分

- ▶ 51:38 for b , also for c , also for f . Those are **leaf nodes**, which we usually have control over. 对 b , 也对 c , 也对 f 。这些是叶子节点，我们通常可以控制它们。

leaf node

叶子节点 计算图中没有输入边的节点，通常代表输入变量或可训练参数。

神经网络基础

- ▶ 51:50 And if we **nudge** in direction of the **gradient**, we expect a positive influence on L . So we expect 如果我们沿着梯度方向微调，我们期望对 L 有正向影响。所以我们期望

nudge

微调 对输入做极小扰动，用来看函数输出如何变化。

数学与微积分

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 51:56 L to go up positively. So it should become less negative. It should go up to say negative, L 正向上升。所以它应该变得不那么负。它应该上升到比如说负的

▶ 52:03 you know, six or something like that. It's hard to tell exactly. And we'd have to rerun the forward 六左右。很难确切判断。而且我们必须重新运行前向

▶ 52:09 path. So let me just do that here. This would be the forward pass. f would be unchanged. 路径。让我在这里这样做。这将是前向传播。 f 不会改变。

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

▶ 52:19 This is effectively the forward pass. And now if we print $L.data$, we expect, because we 这实际上就是前向传播。现在如果我们打印 $L.data$ ，我们期望，因为我们已经

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

▶ 52:26 nudged all the values or the inputs in the direction of gradient, we expect it less negative L . 把所有值或输入都沿着梯度方向微调了，我们期望 L 不那么负。

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

▶ 52:31 We expect it to go up. So maybe it's negative six or so. Let's see what happens. Okay, negative seven. 我们期望它上升。所以也许是负六左右。看看会发生什么。好，负七。

▶ 52:38 And this is basically one step of an optimization that will end up running. And really this gradient 这基本上就是后续优化过程的一步。而实际上，这个梯度

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

▶ 52:45 just give us some power because we know how to influence the final outcome. And this 给了我们某种能力，因为我们知道如何影响最终结果。而这

▶ 52:49 will be extremely useful for training neural nets as well as CNNs. So now I would like to do one 对于训练神经网络以及 CNN 来说将极其有用。所以现在我想做一

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 52:54 more example of manual **backpropagation** using a bit more complex and useful example. We are going to
个更复杂、更有用的例子来手动做反向传播。我们将

backpropagation

反向传播 从输出节点开始，沿计算图反向递归应用链式法则，计算每个中间节点和输入对输出的导数。

反向传播与梯度

- ▶ 53:03 **backpropagate** through a **neuron**. So we want to eventually build out **neural networks**. And in the
对神经元做反向传播。我们最终想要搭建出神经网络。而在

backpropagate

反向传播 利用链式法则从输出端向输入端逐层计算梯度，从而高效地训练神经网络。

反向传播与梯度

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 53:10 simplest case, these are multi **layer** perceptrons as they're called. So this is a two **layer** neural
最简单的情况下，这些就是所谓多层感知机。所以这是一个两层的神经

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

- ▶ 53:14 net. And it's got these hidden layers made up of neurons. And these neurons are fully connected
网络。它有这些由神经元组成的隐藏层。而这些神经元之间是全连接的
- ▶ 53:18 to each other. Now biologically, neurons are very complicated devices. But we have very simple
。在生物学上，神经元是非常复杂的器件。但我们有非常简单的

神经元模型与激活函数 00:53

- ▶ 53:23 mathematical models of them. And so this is a very simple mathematical model of a **neuron**.
数学模型来描述它们。所以这是一个非常简单的神经元数学模型。

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 53:29 You have some inputs, X's, and then you have these synapses that have **weights** on them. So 你有一些输入，也就是 X，然后有这些带着权重的突触。所以

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

- ▶ 53:36 the W's are **weights**. And then the synapse interacts with the input to this **neuron** W 就是权重。然后突触与这个神经元的输入

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 53:43 multiplicatively. So what flows to the cell body of this **neuron** is W times X. But there's 做乘性的交互。所以流入这个神经元细胞体的是 W 乘以 X。但这里面有

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 53:50 multiple inputs. There's many W times X's flowing to the cell body. The cell body then has also 多个输入。有很多个 W 乘以 X 流入细胞体。细胞体还

- ▶ 53:56 like some **bias**. So this is kind of like the inherent innate sort of trigger happiness of this **neuron**. 有一个偏置。所以这有点像这个神经元与生俱来的那种触发敏感度。

bias

偏置 神经元中与输入无关的可学习常数项，用来平移激活函数的输出。

神经网络基础

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 54:03 So this **bias** can make it a bit more trigger happy or a bit less trigger happy, regardless of the 所以这个偏置可以让它更容易触发一点，或者更不容易触发一点，而不管

bias

偏置 神经元中与输入无关的可学习常数项，用来平移激活函数的输出。

神经网络基础

- ▶ 54:07 input. But basically, we're taking all the W times X of all the inputs, adding the **bias**, 输入是什么。但基本上，我们把所有输入的 W 乘以 X 加起来，再加上偏置，

bias

偏置 神经元中与输入无关的可学习常数项，用来平移激活函数的输出。

神经网络基础

- ▶ 54:13 and then we take it through an **activation function**. And this **activation function** is 然后把它送进一个激活函数。而这个激活函数

activation function

激活函数 作用在神经元加权和上的非线性函数，使网络具备拟合非线性关系的能力。

神经网络基础

- ▶ 54:18 usually some kind of a squashing function, like a sigmoid or **tanh** or something like that. 通常是某种挤压函数，比如 sigmoid 或者 tanh 之类的。

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 54:23 So as an example, we're going to use the **tanh** in this example. NumPy has a **np.tanh**. So 举例来说，这个例子里我们要用 tanh。NumPy 里有 np.tanh。所以

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 54:32 we can call it on a range and we can plot it. This is the **tanh** function. And you see that 我们可以在一个区间上调用它，然后把它画出来。这就是 tanh 函数。你可以看到

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 54:38 the inputs as they come in, get squashed on the y coordinate here. So right at zero, 输入进来之后，会被挤压到这里的 y 坐标上。所以在正好零点处，

- ▶ 54:46 we're going to get exactly zero. And then as you go more positive in the input, 我们会得到正好是零。然后随着输入往正的方向走，

- ▶ 54:50 then you'll see that the function will only go up to one and then plateau out. And so if you pass 你会看到这个函数最多只升到 1，然后就变得平缓。所以如果你传入

- ▶ 54:57 in very positive inputs, we're going to cap it smoothly at one. And on the negative side, we're 非常正的输入，它会平滑地把结果限制在 1。而在负的那一侧，我们

- ▶ 55:01 going to cap it smoothly to negative one. So that's **tanh**. And that's the squashing function 会平滑地截断到负一。所以那就是 tanh。这就是挤压函数。

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 55:08 or an **activation function**. And what comes out of this **neuron** is just the **activation function** 或者叫激活函数。这个神经元输出的就是激活函数

activation function

激活函数 作用在神经元加权和上的非线性函数，使网络具备拟合非线性关系的能力。

神经网络基础

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 55:13 applied to the **dot product** of the **weights** and the inputs. So let's write one out. 作用在权重和输入的点积上。那我们来写一个出来。

dot product

点积 两个向量对应元素相乘后求和，这里指权重与输入的内积。

数学与微积分

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

- ▶ 55:22 I'm going to copy paste because I don't want to type too much. But okay, so here we have the inputs 我要复制粘贴，因为我不想打太多字。不过好吧，这里我们有输入

- ▶ 55:31 x_1, x_2 . So this is a two dimensional **neuron**. So two inputs are going to come in. x_1, x_2 . 所以这是一个二维神经元。也就是说会有两个输入进来。

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 55:35 These are thought of as the **weights** of this **neuron**, **weights** w_1, w_2 . And these **weights** again 这些被视为这个神经元的权重，权重 w_1, w_2 。而这些权重呢，

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 55:41 are the synaptic strengths for each input. And this is the **bias** of the **neuron**, b . 是每个输入的突触强度。而这是神经元的偏置， b 。

bias

偏置 神经元中与输入无关的可学习常数项，用来平移激活函数的输出。

神经网络基础

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 55:49 And now what we want to do is according to this model, we need to multiply x_1 times w_1 现在我们要做的是，根据这个模型，我们需要把 x_1 乘以 w_1

- ▶ 55:55 and x_2 times w_2 . And then we need to add **bias** on top of it. And it gets a little messy here, 以及 x_2 乘以 w_2 。然后我们需要在上面加上偏置。这里会有点乱，

bias

偏置 神经元中与输入无关的可学习常数项，用来平移激活函数的输出。

神经网络基础

- ▶ 56:02 but all we are trying to do is x_1w_1 plus x_2w_2 plus b . And these are multiplied here. 但我们想做的其实就是 x_1w_1 加上 x_2w_2 再加上 b 。这些在这里是相乘的。

- ▶ 56:09 Except I'm doing it in small steps so that we actually have pointers to all these intermediate 只不过我是分小步来做，这样我们实际上就有指向所有这些中间

- ▶ 56:14 **nodes**. So we have x_1w_1 variable, x_2w_2 variable, and I'm also labeling them. 节点的指针。所以我们有 x_1w_1 变量、 x_2w_2 变量，而且我也给它们加了标签。

- ▶ 56:21 So n is now the cell body raw **activation** without the **activation function** for now.
所以 n 现在是细胞体的原始激活值，暂时还没有经过激活函数。

activation

激活值 神经元对输入加权求和并加上偏置后得到的数值。

神经网络基础

activation function

激活函数 作用在神经元加权求和上的非线性函数，使网络具备拟合非线性关系的能力。

神经网络基础

- ▶ 56:30 And this should be enough to basically plot it. So **draw_dot** of n gives us x_1 times w_1 ,
这应该就足够把它画出来了。所以对 n 调用 **draw_dot** 会得到 x_1 乘以 w_1 ,

draw_dot

draw_dot micrograd 中把 Value 计算图导出为 Graphviz 图形的辅助函数。

代码与工程

- ▶ 56:39 x_2 times w_2 being added. Then the **bias** gets added on top of this. And this n is this sum.
 x_2 乘以 w_2 被加进来。然后偏置加在这上面。这个 n 就是这个和。

bias

偏置 神经元中与输入无关的可学习常数项，用来平移激活函数的输出。

神经网络基础

- ▶ 56:49 So we're now going to take it through an **activation function**. And let's say we use the **tanh**.
现在我们把它送进一个激活函数。假设我们用 **tanh**

activation function

激活函数 作用在神经元加权求和上的非线性函数，使网络具备拟合非线性关系的能力。

神经网络基础

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 56:54 so that we produce the output. So what we'd like to do here is we'd like to do the output.
以便产生输出。所以我们这里想做的就是得到输出。

- ▶ 56:59 And I'll call it o is n dot **tanh**. Okay, but we haven't yet written the **tanh**.
我会把它称为 o ，也就是 n dot **tanh**。好吧，但我们还没写 **tanh**。

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 57:08 Now the reason that we need to implement another `tanh` function here is that `tanh` is a hyperbolic function.

现在我们需要在这里再实现一个 `tanh` 函数的原因是，`tanh` 是一个双曲函数。

`tanh`

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 57:15 And we've only so far implemented a plus and a times. And you can't make a `tanh` out of just 而我们目前只实现了加法和乘法。你没法只用这些就做出 `tanh`

`tanh`

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 57:20 pluses and times. You also need `exponentiation`. So `tanh` is this kind of a formula here. 加法和乘法。你还需要指数运算。所以 `tanh` 就是这样的一个公式。

`exponentiation`

指数运算 计算 e 的幂的运算，是 `tanh` 等函数的基础运算。

数学与微积分

`tanh`

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 57:27 You can use either one of these. And you see that there was `exponentiation` involved, 你可以使用其中任意一个。你可以看到这里涉及了指数运算，

`exponentiation`

指数运算 计算 e 的幂的运算，是 `tanh` 等函数的基础运算。

数学与微积分

- ▶ 57:30 which we have not implemented yet for our `Value node` here. So we're not going to be able 我们的 `Value` 节点还没有实现这个。所以我们还不能

`value node`

值节点 micrograd 中存储数据 `data` 和梯度 `grad` 的计算图节点。

代码与工程

▶ 57:35 to produce `tanh` yet. And we have to go back up and implement something like it. Now one option here

还不能产生 `tanh`。我们必须回到上面实现类似的东西。现在一个选择是

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

▶ 57:42 is we could actually implement `exponentiation`, right? And we could return the `exp` of the `value` 是我们可以实现指数运算，对吧？我们可以返回值的 `exp`,

exponentiation

指数运算 计算 e 的幂的运算，是 `tanh` 等函数的基础运算。

数学与微积分

exp

指数函数 以自然常数 e 为底的指数函数，`tanh` 可以由它组合表达。

数学与微积分

▶ 57:50 instead of a `tanh` of the `value`. Because if we had `exp`, then we have everything else that we need. 而不是值的 `tanh`。因为如果我们有 `exp`，那么我们就有了所需的一切。

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

▶ 57:55 So because we know how to add and we know how to, we know how to add and we know how 因为我们知道如何加法，我们知道如何，我们知道如何加，也知道如何

▶ 58:02 to multiply. So we'd be able to create `tanh` if we knew how to `exp`. But for the purposes of this example,

乘法。所以如果我们知道如何 `exp`，就能创建 `tanh`。但在这个例子中，

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

exp

指数函数 以自然常数 e 为底的指数函数，`tanh` 可以由它组合表达。

数学与微积分

▶ 58:08 I specifically wanted to show you that we don't necessarily need to have the most atomic pieces
我特别想向你们展示，我们不一定需要拥有最原子的片段

▶ 58:15 in this **Value object**. We can actually like create functions at arbitrary
在这个 Value 对象中。我们实际上可以在任意

value object

Value 对象 micrograd 中包装标量数值并记录计算图关系的对象。

代码与工程

▶ 58:23 points of abstraction. They can be complicated functions, but they can be also very, very
抽象层次上创建函数。它们可以是复杂的函数，也可以是非常非常

▶ 58:27 simple functions like a plus. And it's totally up to us. The only thing that matters is that
简单的函数，比如加法。完全取决于我们。唯一重要的是

▶ 58:31 we know how to **differentiate** through any one function. So we take some inputs and we make an
我们知道如何对任意一个函数求导。我们取一些输入，产生一个

differentiate

求导 计算函数导数的过程，在反向传播中用于得到梯度。

反向传播与梯度

▶ 58:36 output. The only thing that matters can be arbitrarily complex function, as long as you know how to
输出。唯一重要的是，它可以是一个任意复杂的函数，只要你知道如何

▶ 58:41 create the **local derivative**. If you know the **local derivative** of how the inputs impact the
计算局部导数。如果你知道输入如何影响输出的局部导数，

local derivative

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

▶ 58:45 output, then that's all you need. So we're going to cluster up all of this **expression** and we're
那么这就是你所需要的。所以我们要把整个表达式聚在一起，

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

▶ 58:51 not going to break it down to its atomic pieces. We're just going to directly implement **tanh**.
不把它拆解成原子片段。我们直接实现 tanh。

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- 58:55 So let's do that. Def `tanh` and then out will be a `Value` of and we need this `expression` here. So 所以我们来做。定义 `tanh`，然后 out 将是一个 `Value`，我们需要这个表达式。所以

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

value

`Value` 对象 `micrograd` 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- 59:08 let me actually copy paste. Let's grab `x`, which is `self.data`. And then this, I believe, is the 让我实际复制粘贴。我们取 `x`，也就是 `self.data`。然后这个，我相信是

- 59:19 `tanh`. `math.exp` of `2x` minus 1 over `math.exp` of `2x` plus 1. Maybe I can call this `t`. Just so that it matches exactly.

`tanh`. `math.exp(2x)` 减 1 除以 `math.exp(2x)` 加 1。也许我可以把它叫做 `t`。只是为了完全匹配。

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

exp

指数函数 以自然常数 e 为底的指数函数，`tanh` 可以由它组合表达。

数学与微积分

- 59:35 Okay. And now this will be `t` and `children` of this `node`. There's just one child. And I'm wrapping it in a tuple. So this is a tuple of one object, just `self`.

好。现在这个将会是 `t`，以及这个节点的 `children`。这里只有一个子节点。我把它包装成一个元组。所以这是一个只包含一个对象的元组，就是 `self`。

children

子节点 产生当前值的输入值集合，用来建立计算图中的连接关系。

反向传播与梯度

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

- 59:45 And here the name of this operation will be 而这里这个操作的名字将是

59:50 `tanh`. And we're going to return that. Okay. So now `Value` should be implementing `tanh`. And now we can `tanh`. 然后我们会返回它。好。所以现在 `Value` 应该实现 `tanh` 了。现在我们就可以

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

value

`Value` 对象 `micrograd` 中表示标量及其梯度、参与计算图的基本类。

代码与工程

1:00:02 scroll all the way down here. And we can actually do `n.dot(tanh)`. And that's going to return the 一路滚动到下面。我们实际上可以调用 `n.tanh`。它会返回 `n` 的

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

1:00:08 `tanh` output of `n`. And now we should be able to draw a dot of `o`, not of `n`. So let's see how that `tanh` 输出。现在我们应该能画出 `o` 的图，而不是 `n` 的。我们看看这

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

1:00:15 worked. There we go. And went through `tanh` to produce this `o`. So now `tanh` is a sort of our little 效果如何。出来了。它经过了 `tanh` 产生这个 `o`。所以现在 `tanh` 算是我们小小的

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

1:00:28 `micrograd` supported `node` here as an operation. And as long as we know the derivative of `tanh`, `micrograd` 所支持的一个节点，作为一个操作。只要我们知道了 `tanh` 的导数，

micrograd

`micrograd` 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

▶ 1:00:36 then we'll be able to back propagate through it. Now let's see this tanh in action.
就能通过它做反向传播。现在来看看这个 tanh 的实际效果。

propagate

传播 梯度沿计算图从输出向输入传递的过程。

反向传播与梯度

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

▶ 1:00:40 Currently, it's not squashing too much because the input to it is pretty low. So the bias was
目前它没有压缩太多，因为输入到它的值相当低。所以如果把偏置

bias

偏置 神经元中与输入无关的可学习常数项，用来平移激活函数的输出。

神经网络基础

▶ 1:00:45 increased to say eight. Then we'll see that what's flowing into the tanh now is two. And tanh is
增大到比如 8。然后我们会看到流进 tanh 的现在是 2。而 tanh 把它

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

▶ 1:00:55 squashing it to 0.96. So we're already hitting the tail of this tanh. And it will sort of smoothly go up
压缩到 0.96。所以我们已经碰到了这个 tanh 的尾巴。它会比较平滑地上升到

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

▶ 1:01:01 to one and then plateau out over there. Okay. So now I'm going to do something slightly strange.
1，然后在那里进入平台期。好。现在我要做一件稍微有点奇怪的事。

▶ 1:01:05 I'm going to change this bias from eight to this number, 6.88, et cetera. And I'm going to
我要把这个偏置从 8 改成这个数字，6.88 之类。我这样做

bias

偏置 神经元中与输入无关的可学习常数项，用来平移激活函数的输出。

神经网络基础

- ▶ 1:01:12 do this for specific reasons because we're about to start **back propagation**. And I want to make sure 是有具体原因的，因为我们即将开始反向传播。我想确保

back propagation

反向传播 利用链式法则从输出向输入高效计算损失函数梯度的算法。

反向传播与梯度

- ▶ 1:01:18 that our numbers come out nice. They're not like very crazy numbers. They're nice numbers that we can

我们的数字结果很漂亮。它们不是那种很疯狂的数字。是那种我们可以在

- ▶ 1:01:22 sort of understand in our head. Let me also add those label. O is short for output here. 脑子里理解的好数字。我也来加上那个标签。O 在这里是 output（输出）的缩写。

- ▶ 1:01:30 So that's the O. Okay. So 0.88 flows into **tanh** comes out 0.7. So now we're going to do back 所以那就是 O。好。0.88 流入 tanh，出来 0.7。现在我们要做反向

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 1:01:37 propagation. And we're going to fill in all the **gradients**. So what is the **derivative** O with respect 传播。我们要把所有梯度填上。那么 O 对所有

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 1:01:43 to all the inputs here? And of course, in a typical **neural network** setting, what we really care about 输入的导数是什么？当然，在典型的神经网络设定中，我们真正关心的

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- ▶ 1:01:48 the most is the **derivative** of these neurons on the **weights** specifically, the W2 and W1, 最这些神经元对权重的导数，具体来说，就是 W2 和 W1，

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

- ▶ 1:01:55 because those are the **weights** that we're going to be changing part of the optimization. 因为这些权重就是我们在优化过程中要改变的那部分。

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

- ▶ 1:01:59 And the other thing that we have to remember is here we have only a single **neuron**, 另外要记住的是，这里我们只有一个神经元，

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:02:02 but in the **neural net**, you typically have many neurons and they're connected. 但在神经网络里，通常有很多神经元，而且它们是相互连接的。

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 1:02:07 So this is only like a one small **neuron**, a piece of a much bigger puzzle. And eventually 所以这只是一个很小的神经元，是更大谜题中的一小块。最终

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:02:10 there's a **loss function** that sort of measures the accuracy of the **neural net**. And we're back
会有一个损失函数，用来衡量神经网络的准确率。然后我们

loss function

损失函数 衡量模型预测与目标之间差异、训练中需要最小化的函数。

训练与优化

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 1:02:14 propagating with respect to that accuracy and trying to increase it. So let's start off
针对这个准确率进行反向传播，并试图提高它。那我们开始

- ▶ 1:02:20 **backpropagation** here and then. What is the **derivative** of O with respect to O? The base
从这里做反向传播，然后。O 对 O 的导数是什么？基准

backpropagation

反向传播 从输出节点开始，沿计算图反向递归应用链式法则，计算每个中间节点和输入对输出的导数。

反向传播与梯度

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 1:02:25 case sort of we know always is that the **gradient** is just 1.0. So let me fill it in. And then let me
情况我们总是知道的，就是梯度就是 1.0。让我把它填进去。然后让我

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:02:35 split out the drawing function here. And then here let me clear this output here. Okay. So now
把绘图函数拆出来。然后这里，让我清空这个输出。好。那么现在

- ▶ 1:02:50 when we draw O, we'll see that **grad** is one. So now we're going to **backpropagate** through the **tanh**.

当我们画 O 时，会看到 grad 是 1。现在我们要通过 tanh 反向传播。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

backpropagate

反向传播 利用链式法则从输出端向输入端逐层计算梯度，从而高效地训练神经网络。

反向传播与梯度

1:02:56 So to **backpropagate** through **tanh**, we need to know the **local derivative** of **tanh**. So if we have that O is

所以要通过 **tanh** 反向传播, 我们需要知道 **tanh** 的局部导数。所以如果我们有 O 是

backpropagate

反向传播 利用链式法则从输出端向输入端逐层计算梯度, 从而高效地训练神经网络。

反向传播与梯度

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数, 常用作神经网络的激活函数。

数学与微积分

1:03:05 **tanh** of n , then what is dO by dn ? Now what you could do is you could come here and you could take $\tanh(n)$, 那么 dO/dn 是什么? 你可以做的是, 来到这里, 然后取

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数, 常用作神经网络的激活函数。

数学与微积分

1:03:14 this **expression** and you could do your **calculus derivative** taking. And that would work. But 这个表达式, 然后做微积分求导。那会行得通。但是

expression

表达式 由多个运算组合而成的计算式, 在计算图中对应多个节点。

代码与工程

calculus

微积分 研究变化率与累积量的数学分支。

数学与微积分

1:03:20 we can also just scroll down Wikipedia here into a section that hopefully tells us that **derivative** 我们也可以直接往下滚动维基百科, 到一个希望告诉我们那个导数的章节

derivative

导数 描述函数在某一点的变化率, 即输入发生微小变化时输出相应变化的比率。

数学与微积分

1:03:29 d by dx of **tanh** of x is any of these. I like this one, $1 - \tanh^2(x)$. So this is $1 - \tanh^2(x)$ $d/dx \tanh(x)$ 是这些中的任意一个。我喜欢这个: $1 - \tanh^2(x)$ 。所以这是 $1 - \tanh^2(x)$

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数, 常用作神经网络的激活函数。

数学与微积分

- 1:03:36 **tanh** of x squared. So basically what this is saying is that dO by dn is 1 minus **tanh** of n $\tanh(x)$ 的平方。所以这基本上是在说 dO/dn 等于 1 减去 $\tanh(n)$

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- 1:03:48 squared. And we already have **tanh** of n . It's just O . So it's 1 minus O squared. So O is the output 的平方。而我们已经有了 $\tanh(n)$ 。它就是 O 。所以是 1 减去 O 的平方。所以 O 是输出

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- 1:03:57 here. So the output is this number. $O \cdot data$ is this number. And then what this is saying is that 这里。所以输出是这个数。 $O \cdot data$ 是这个数。然后这说的是

- 1:04:09 dO by dn is 1 minus this squared. So 1 minus $O \cdot data$ squared is 0.5 conveniently. So the local dO/dn 是 1 减去这个的平方。所以 1 减去 $O \cdot data$ 的平方正好是 0.5。所以局部

- 1:04:19 **derivative** of this **tanh** operation here is 0.5. And so that would be dO by dn . So we can fill in that 这个 **tanh** 操作的导数是 0.5。所以那就是 dO/dn 。那我们可以把它填进去

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- 1:04:30 $n \cdot \text{grad}$ is 0.5. We'll just fill it in. So this is exactly 0.5, 1 half. So now we're going to $n \cdot \text{grad}$ 是 0.5。我们直接把它填上。所以这里正好是 0.5，也就是二分之一。现在我们要

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 1:04:45 continue the **back propagation**. This is 0.5 and this is a **plus node**. So how is back prop going to 继续反向传播。这里是 0.5，而这是一个加法节点。那么反向传播会

back propagation

反向传播 利用链式法则从输出向输入高效计算损失函数梯度的算法。

反向传播与梯度

plus node

加法节点 计算图中执行加法运算的节点，其局部导数对该节点的每个输入均为 1。

神经网络基础

▶ 1:04:54 do here? And if you remember our previous example, a plus is just a distributor of **gradient**. So this 在这里怎么做呢？如果你还记得之前的例子，加法节点就是一个梯度的分发器。所以这个

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

▶ 1:05:02 **gradient** will simply flow to both of these equally. And that's because the **local derivative** of this 梯度会简单地等量流向这两个分支。这是因为这个

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

local derivative

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

▶ 1:05:07 operation is one for every one of its inputs. So 1 times 0.5 is 0.5. So therefore we know that 运算对它的每一个输入的局部导数都是 1。所以 1 乘以 0.5 是 0.5。因此我们知道

▶ 1:05:14 this **node** here, which we called n, its **grad** is just 0.5. And we know that $b \cdot \text{grad}$ is also 这里的这个节点，我们称之为 n，它的 grad 就是 0.5。而且我们知道 $b \cdot \text{grad}$ 也是

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 1:05:23 0.5. So let's set those and let's draw. So those are 0.5. Continuing, we have another plus. 0.5, 0.5。我们把它设好，然后画出来。这些就是 0.5。继续，我们还有另一个加法节点。0.5，

▶ 1:05:33 again we'll just distribute. So 0.5 will flow to both of these. So we can set, there's x_2w_2 as well, 同样我们直接分发。所以 0.5 会流向这两个。我们可以设, 还有 x_2w_2 也是,

▶ 1:05:45 that **grad** is 0.5 as well. And let's redraw. Pluses are my favorite operations to back **propagate** 它的 grad 也是 0.5。让我们重新画一下。加法是我最喜欢的反向传播运算,

grad

梯度 计算图中某个变量累积的导数, 表示最终输出对该变量的敏感度。

反向传播与梯度

propagate

传播 梯度沿计算图从输出向输入传递的过程。

反向传播与梯度

▶ 1:05:51 through because it's very simple. So now what's flowing into these expressions is 0.5. 因为它非常简单。所以现在流入这些表达式的是 0.5。

▶ 1:05:57 And so really again keep in mind what the **derivative** is telling us at every point in time 所以再说一次, 要记住导数在这里的每一个时刻告诉了我们什么。

derivative

导数 描述函数在某一点的变化率, 即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 1:06:01 along here. This is saying that if we want the output of this **neuron** to increase, 沿着这里。它说的是, 如果我们想让这个神经元的输出增大,

neuron

神经元 神经网络的基本单元, 对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

▶ 1:06:08 then the influence on these expressions is positive on the output. Both of them are positive 那么这些表达式对输出的影响是正的。它们两个都是对输出的

▶ 1:06:17 contribution to the output. So now back propagating to x_2 and w_2 . First, this is a 正贡献。所以现在反向传播到 x_2 和 w_2 。首先, 这是一个

▶ 1:06:24 **times node**. So we know that the **local derivative** is just the other term. So if we want to calculate 乘法节点。我们知道局部导数就是另一项。所以如果我们想计算

times node

乘法节点 计算图中执行乘法的节点，某个输入的梯度等于另一输入的数据乘以输出梯度。

反向传播与梯度

local derivative

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

▶ 1:06:30 $x_2 \text{ dot } \text{grad}$, then can you think through what it's going to be? So $x_2 \text{ dot } \text{grad}$ will be $w_2 \text{ dot data}$ $x_2.\text{grad}$, 你能想一下它会有什么吗? $x_2.\text{grad}$ 将会是 $w_2.\text{data}$

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 1:06:44 times this $x_2 w_2 \text{ dot } \text{grad}$, right? And $w_2 \text{ dot } \text{grad}$ will be 乘以这个 $x_2 w_2.\text{grad}$, 对吧? 而 $w_2.\text{grad}$ 将会是

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 1:06:54 $x_2 \text{ dot data times } x_2 w_2 \text{ dot } \text{grad}$, right? So that's the little local piece of **chain rule**. $x_2.\text{data}$ 乘以 $x_2 w_2.\text{grad}$, 对吧? 这就是链式法则中局部的一小块。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

▶ 1:07:07 Let's set them and let's redraw. So here we see that the **gradient** on our **weight** 2 is 0 because x_2 's 我们把它设好，然后重新画一下。这里我们看到权重 2 的梯度是 0，因为 x_2 的

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

- ▶ 1:07:14 data was 0, right? But x_2 will have the **gradient** 0.5 because data here was 1.
data 是 0，对吧？但 x_2 的梯度会是 0.5，因为这里的 data 是 1。

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:07:20 And so what's interesting here, right, is because the input x_2 was 0, then because of the way the
所以这里有意思的是，对吧，因为输入 x_2 是 0，那么由于乘法这个操作的方式，当然这个梯度就会是 0。

- ▶ 1:07:26 times works, of course this **gradient** will be 0. And to think about intuitively why that is,
乘法运作的方式，当然这个梯度会是 0。而直观地想一想为什么会这样，

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:07:33 **derivative** always tells us the influence of this on the final output. If I wiggle w_2 ,
导数总是告诉我们这个量对最终输出的影响。如果我稍微扰动 w_2 ，

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 1:07:39 how is the output changing? It's not changing because we're multiplying by 0. So because it's
输出会怎么变化？它没有变化，因为我们在乘以 0。所以因为它

- ▶ 1:07:45 not changing, there is no **derivative** and 0 is the correct answer because we're
没有变化，就没有导数，0 就是正确答案，因为我们

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 1:07:49 squashing with that 0. And let's do it here. 0.5 should come here and flow through this times.
被那个 0 压掉了。那我们在这里做一下。0.5 应该到这里，并流过这个乘法。

- ▶ 1:07:57 And so we'll have that x_1 dot **grad** is, can you think through a little bit what this should be?
于是我们会得到 x_1 .grad 是，你能稍微想一想这应该是什么吗？

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 1:08:07 The **local derivative** of times with respect to x_1 is going to be w_1 . So w_1 's data times 乘法对 x_1 的局部导数会是 w_1 。所以 w_1 的 data 乘以

local derivative

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

▶ 1:08:15 $x_1 w_1 \text{ dot } \text{grad}$ and $w_1 \text{ dot } \text{grad}$ will be $x_1 \text{ dot data times } x_1 w_1 \text{ dot } \text{grad}$. $x_1 w_1 \text{ grad}$, 而 $w_1 \text{ grad}$ 会是 x_1 的 data 乘以 $x_1 w_1 \text{ grad}$ 。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 1:08:27 Let's see what those came out to be. So this is 0.5, so this would be negative 1.5 and this would be 我们看看它们算出来是多少。这个是 0.5，所以这个会是 -1.5，而这个会是

▶ 1:08:33 1. And we've backpropagated through this **expression**. These are the actual final derivatives. 1. 我们已经通过这个表达式做了反向传播。这些就是实际的最终导数。

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

▶ 1:08:39 So if we want this **neuron**'s output to increase, we know that what's necessary is that 所以如果我们想让这个神经元的输出增大，我们知道必要的是

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

▶ 1:08:47 w_2 , we have no **gradient**. w_2 doesn't actually matter to this **neuron** right now, w_2 ，我们没有梯度。 w_2 目前对这个神经元其实没有影响，

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:08:51 but this **neuron**, this **weight** should go up. So if this **weight** goes up, then this **neuron**'s
但这个神经元，这个权重应该增大。所以如果这个权重增大，那么这个神经元的

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

- ▶ 1:08:57 output would have gone up and proportionally because the **gradient** is 1. Okay, so doing the
输出就会增大，而且是成比例地增大，因为梯度是 1。好，那么做

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:09:02 **backpropagation** manually is obviously ridiculous. So we are now going to put an end to this
手动反向传播显然很荒谬。所以我们现在要结束这

backpropagation

反向传播 从输出节点开始，沿计算图反向递归应用链式法则，计算每个中间节点和输入对输出的导数。

反向传播与梯度

自动实现反向传播 01:09

- ▶ 1:09:06 suffering and we're going to see how we can implement the **backward** pass a bit more automatically.
种痛苦，然后看看如何能更自动地实现反向传播过程。

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:09:11 We're not going to be doing all of it manually out here. It's now pretty obvious to us, by example,
我们不会在这里全部手动做了。现在通过例子我们已经很清楚，

- ▶ 1:09:17 how these pluses and times are `backprop` ingredients. So let's go up to the `value`
这些加法和乘法是如何构成反向传播的要素的。所以我们往上到 Value

backprop

反向传播 从损失函数出发，沿计算图反向逐节点应用链式法则来计算梯度的算法。

反向传播与梯度

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:09:22 object and we're going to start codifying what we've seen in the examples below.
对象，然后开始把我们在下面的例子中看到的東西代码化。

- ▶ 1:09:29 So we're going to do this by storing a special self. `_backward`,
所以我们将通过存储一个特殊的 `self._backward` 来实现这一点，

_backward

反向传播函数 Value 上保存的函数，负责把当前节点的梯度按链式法则传给输入。

反向传播与梯度

- ▶ 1:09:36 and this will be a function which is going to do that little piece of `chain rule`. At each little
而这个函数将负责完成那一小部分链式法则。在每个小小的

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 1:09:41 `node` that took inputs and produced output, we're going to store how we are going to chain the
接收输入并产生输出的节点上，我们都会存储要如何把

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

- ▶ 1:09:49 output's `gradient` into the input's `gradients`. So by default, this will be a function that
输出的梯度链式地传到输入的梯度上。所以默认情况下，这会是一个

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

▶ 1:09:56 doesn't do anything. And you can also see that here in the `Value` in `micrograd`. So 什么都不做的函数。你也可以在 `micrograd` 的 `Value` 里看到这一点。所以

value

Value 对象 `micrograd` 中表示标量及其梯度、参与计算图的基本类。

代码与工程

micrograd

`micrograd` 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

▶ 1:10:04 with this `backward` function, by default, doesn't do anything. This is a complete no-op. 这个 `backward` 函数默认什么都不做。它就是一个完整的空操作。

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 1:10:10 And that would be sort of the case, for example, for a `leaf node`. For a `leaf node`, there's nothing 例如，对于叶子节点来说就是这种情况。对叶子节点来说，没有什么

leaf node

叶子节点 计算图中没有输入边的节点，通常代表输入变量或可训练参数。

神经网络基础

▶ 1:10:13 to do. But now, if when we're creating these out values, these out values are an addition 可做的。但现在，如果当我们创建这些 `out` 值时，这些 `out` 值是一个加法

▶ 1:10:21 of self and other. And so we want to set out's `_backward` to be the function that propagates 由 `self` 和 `other` 相加而来。所以我们想把 `out` 的 `_backward` 设为传播

_backward

反向传播函数 `Value` 上保存的函数，负责把当前节点的梯度按链式法则传给输入。

反向传播与梯度

▶ 1:10:31 the `gradient`. So let's define what should happen. And we're going to store it in a closure. 梯度的函数。那我们先定义应该发生什么。我们会把它存在一个闭包里。

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:10:42 Let's define what should happen when we call out's `_backward` . For addition, our job is to take 我们定义一下调用 out 的 `_backward` 时应该发生什么。对于加法，我们的任务是取

`_backward`

反向传播函数 Value 上保存的函数，负责把当前节点的梯度按链式法则传给输入。

反向传播与梯度

- ▶ 1:10:52 out's `grad` and `propagate` it into self's `grad` and other's `grad` . So basically, we want to out 的 `grad`，并把它传播到 self 的 `grad` 和 other 的 `grad`。所以基本上，我们想要

`grad`

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

`propagate`

传播 梯度沿计算图从输出向输入传递的过程。

反向传播与梯度

- ▶ 1:10:57 set self's `grad` to something. And we want to set other's `grad` to something. 把 self 的 `grad` 设为某个值。我们想要把 other 的 `grad` 设为某个值。

`grad`

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 1:11:05 And the way we saw below how `chain rule` works, we want to take the `local derivative` times the 就像我们下面看到的链式法则那样，我们想取局部导数乘以

`chain rule`

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

`local derivative`

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

- ▶ 1:11:12 sort of `global derivative` , I should call it, which is the `derivative` of the final output 所谓的全局导数——我应该这么叫它——也就是最终输出

`global derivative`

全局导数 最终输出对某个中间值的导数，反向传播中体现为该节点累积的梯度。

反向传播与梯度

`derivative`

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 1:11:16 of the **expression** with respect to out's data. With respect to out. So the **local derivative**
对该表达式关于 out 的 data 的导数。关于 out。所以局部导数

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

local derivative

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

▶ 1:11:25 of self in an addition is 1.0. So it's just 1.0 times out's **grad**. That's the **chain rule**.
在加法中对 self 来说是 1.0。所以它就是 1.0 乘以 out 的 grad。这就是链式法则。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

▶ 1:11:35 And other's **grad** will be 1.0 times out's **grad**. And what you basically what you're seeing here is
而 other 的 grad 将是 1.0 乘以 out 的 grad。你基本上在这里看到的是

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 1:11:41 that out's **grad** will simply be copied onto self's **grad** and other's **grad**, as we saw happens for an
out 的 grad 会直接复制到 self 的 grad 和 other 的 grad 上，就像我们看到的

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 1:11:48 addition operation. So we're going to later call this function to **propagate** the **gradient**,
加法操作那样。所以之后我们会调用这个函数来传播梯度，

propagate

传播 梯度沿计算图从输出向输入传递的过程。

反向传播与梯度

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:11:53 having done an addition. Let's now do multiplication. We're going to also define the `backward`, 做完加法之后。现在我们来做乘法。我们还要定义它的 `backward`,

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:12:02 and we're going to set its `backward` to be `backward`. And we want to chain out `grad` into 然后我们要把它的 `backward` 设为 `backward`。我们想把 `out.grad` 链式地加到

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 1:12:11 self `grad` and other `grad`. And this will be a little piece of `chain rule` for self `grad` 和 other `grad` 上。这就是链式法则的一小部分，用于

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 1:12:18 multiplication. So we'll have, so what should this be? Can you think through? So what is the 乘法。那我们来想想，这里应该是什么？你们能想清楚吗？那么局部

- ▶ 1:12:29 `local derivative`? Here, the `local derivative` was other.data, and then self.data, 导数是什么？这里，局部导数是 other.data，然后是 self.data，

local derivative

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

▶ 1:12:38 and then times out. `grad`. That's `chain rule`. And here we have `self.data` times out that 然后再乘以 `out.grad`。这就是链式法则。而这里我们用 `self.data` 乘以 `out`

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

▶ 1:12:44 `grad`. That's what we've been doing. And finally here for `tanh`, the `backward`, and then we want `.grad`。这就是我们一直在做的。最后对于 `tanh`，它的 `backward`，然后我们要

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

▶ 1:12:55 to set `out's backward` to be just `backward`. And here we need to `backpropagate`. We have `out` 把 `out` 的 `backward` 设为 `just backward`。这里我们需要反向传播。我们有 `out`

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

backpropagate

反向传播 利用链式法则从输出端向输入端逐层计算梯度，从而高效地训练神经网络。

反向传播与梯度

▶ 1:13:04 `.grad`, and we want to chain it into `self.grad`. And `self.grad` will be the `local derivative` `.grad`，我们想把它链式地加到 `self.grad`。而 `self.grad` 将是这个操作的局部

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

local derivative

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

1:13:13 of this operation that we've done here, which is `tanh`. And so we saw that the `local gradient` is 1 minus

导数，也就是我们这里做的 `tanh` 操作。我们看到局部梯度是 1 减去

`tanh`

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

`local gradient`

局部梯度 某个运算节点对其输入变量的导数，只由该节点自身的运算决定。

反向传播与梯度

1:13:19 the `tanh` of `x` squared, which here is `t`. That's the `local derivative` because that's `t` is the `tanh x` 的平方，这里就是 `t`。这就是局部导数，因为 `t` 是

`tanh`

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

`local derivative`

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

1:13:26 output of this `tanh`. So 1 minus `t` squared is the `local derivative`. And then `gradient` 这个 `tanh` 的输出。所以 1 减 `t` 的平方就是局部导数。然后梯度

`tanh`

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

`local derivative`

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

1:13:31 has to be multiplied because of the `chain rule`. So out's `grad` is chained through the `local gradient` 必须相乘，因为链式法则。所以 out 的 `grad` 通过局部梯度链式地

`chain rule`

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

`grad`

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 1:13:37 into self.grad. And that should be basically it. So we're going to redefine our value node.
传到 self.grad。基本上就是这样。现在我们要重新定义我们的 Value 节点。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

value node

值节点 micrograd 中存储数据 data 和梯度 grad 的计算图节点。

代码与工程

▶ 1:13:44 We're going to scroll all the way down here. And we're going to redefine our expression.
我们往下滚到最下面。然后重新定义我们的表达式。

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

▶ 1:13:52 Make sure that all the grads are zero. Okay. But now we don't have to do this manually
确保所有梯度都是零。好的。但现在我们不用手动做这些了

▶ 1:13:58 anymore. We are going to basically be calling the dot backward in the right order. So first we want to
。我们将按正确的顺序调用 .backward。首先我们要

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 1:14:06 call out's dot backward. So out was the outcome of tanh. Right. So calling out's backward, those
调用 out 的 .backward。因为 out 是 tanh 的输出。对。所以调用 out 的 .backward，那些

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

▶ 1:14:20 whose backward will be this function. This is what it will do. Now we have to be careful
它的 backward 就是那个函数。这就是它要做的事。现在我们必须小心

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:14:27 because there's a times out. `grad` and out. `grad` remember is initialized to zero. So here 因为这里有一个乘以 `out.grad`, 而 `out.grad` 记得初始化为零。所以这里

grad

梯度 计算图中某个变量累积的导数, 表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 1:14:39 we see `grad` zero. So as a base case, we need to set `o dot grad` to 1.0 to initialize this with one. 我们看到梯度为零。所以作为基本情况, 我们需要将 `o` 的梯度设置为 1.0, 用 1 来初始化它。

grad

梯度 计算图中某个变量累积的导数, 表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 1:14:53 And then once this is one, we can call `o's backward`. And what that should do is it should 然后一旦这个值为 1, 我们就可以调用 `o` 的反向传播。它应该做的是

backward

反向传播 从输出向输入传播梯度, 利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:14:58 `propagate` this `grad` through `o`. So the `local derivative` times the `global derivative`, which is 将这个梯度通过 `o` 传播。所以局部导数乘以全局导数, 这个全局导数

propagate

传播 梯度沿计算图从输出向输入传递的过程。

反向传播与梯度

grad

梯度 计算图中某个变量累积的导数, 表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 1:15:05 initialized at one. So this should, um, a dope. So I thought about redoing it, but I figured I should 初始化为 1。所以这应该, 嗯, 有点傻。我本来想重做, 但我觉得我应该

- ▶ 1:15:19 just leave the error in here because it's pretty funny. Why is `NoneType` object not callable? It's 就把这个错误留在里面, 因为它挺有趣的。为什么 `NoneType` 对象不可调用? 这是因为

- ▶ 1:15:25 because I screwed up. We're trying to save these functions. So this is correct. This here, 因为我搞砸了。我们试图保存这些函数。所以这是正确的。这里,

- ▶ 1:15:33 you don't want to call the function because that returns none. These functions return none. 你不想调用这个函数, 因为它返回 `None`。这些函数返回 `None`。

▶ 1:15:37 We just want to store the function. So let me redefine the `value object`. And then we're going 我们只是想存储这个函数。所以让我重新定义 Value 对象。然后我们将

value object

Value 对象 micrograd 中包装标量数值并记录计算图关系的对象。

代码与工程

▶ 1:15:42 to come back in redefine the `expression`, draw dot. Everything is great. O dot `grad` is one. 回来重新定义表达式，绘制点。一切都很棒。O 的梯度是 1。

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 1:15:50 O dot `grad` is one. And now math, this should work, of course. Okay. So o's `backward` O 的梯度是 1。现在，呃，这应该能工作，当然。好的。所以 o 的反向传播

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 1:15:57 should have, this `grad` should now be 0.5 if we redraw and if everything went correctly, 0.5. Yay. 应该已经，这个梯度现在应该是 0.5，如果我们重新绘制并且一切顺利的话，0.5。耶。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 1:16:05 Okay. So now we need to call `n.grad`. And that's `backward`. Sorry. 好的。现在我们需要调用 `n.grad`。那就是反向传播。抱歉。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 1:16:13 And it's `backward`. So that seems to have worked. So `n.backward`
它是反向传播。所以这似乎起作用了。所以 `n.backward`

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 1:16:19 routed the `gradient` to both of these. So this is looking great.
将梯度路由到了这两个。所以这看起来很棒。

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

▶ 1:16:24 Now we could, of course, called, uh, called `b.grad`, `b.backward`. Sorry.
现在我们可以，当然，调用，呃，调用 `b.grad`, `b.backward`。抱歉。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 1:16:29 What's going to happen? Well, `b` doesn't have a `backward`. `b's backward` because `b` is a `leaf node`.
会发生什么？嗯，`b` 没有 `backward`。`b` 的 `backward` 是因为 `b` 是一个叶子节点。

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

leaf node

叶子节点 计算图中没有输入边的节点，通常代表输入变量或可训练参数。

神经网络基础

▶ 1:16:37 `b's backward` is by initialization a no-op function. So nothing would happen, but we can call it on it.
`b` 的 `backward` 在初始化时是一个空操作函数。所以什么也不会发生，但我们可以调用它。

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:16:45 But when we call this one, it's `backward`, then we expect this 0.5 to get further routed.
但当我们调用这个时，它是反向传播，然后我们期望这个 0.5 被进一步路由。

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:16:57 Right. So there we go. 0.5.5. And then finally, we want to call it here on `x2w2`
对。所以就是这样。0.5.5。然后最后，我们想在这里调用 `x2w2` 上的

- ▶ 1:17:10 and on `x1w1`. Let's do both of those. And there we go. So we get 0.5, negative 1.5,
以及 `x1w1` 上的。让我们两个都做。这就对了。所以我们得到 0.5，负 1.5，

- ▶ 1:17:23 and 1 exactly as we did before. But now we've done it through calling that `backward`,
和 1，完全和我们之前做的一样。但这次我们是通过调用那个 `backward` 来实现的，

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:17:30 um, sort of manually. So we have one last piece to get rid of, which is us calling `underscore`
呃，基本上算是手动地。所以我们还剩最后一块要去掉，就是我们手动调用 `_`

- ▶ 1:17:36 `backward` manually. So let's think through what we are actually doing. We've laid out a mathematical
`backward`。那我们来想一下我们实际到底在做什么。我们铺开了一个数学

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:17:42 `expression`, and now we're trying to go backwards through that `expression`. So going backwards
表达式，现在我们要沿着这个表达式反向走一遍。所谓反向

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 1:17:48 through the **expression** just means that we never want to call a dot **backward** for any **node** 遍历这个表达式，意思就是：对任何一个节点，我们都不希望在

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:17:54 before we've done sort of everything after it. So we have to do everything after it before 它后面的所有东西都处理完之前调用它的 `.backward()`。所以我们必须先做完它后面的所有事，然后

- ▶ 1:18:01 we're ever going to call dot **backward** on any one **node**. We have to get all of its full dependencies, 才可能在任何一个节点上调用 `.backward()`。我们必须拿到它全部的依赖，

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

- ▶ 1:18:05 everything that it depends on has to **propagate** to it before we can continue **back propagation**. 它所依赖的一切都必须先传播到它这里，我们才能继续反向传播。

propagate

传播 梯度沿计算图从输出向输入传递的过程。

反向传播与梯度

back propagation

反向传播 利用链式法则从输出向输入高效计算损失函数梯度的算法。

反向传播与梯度

拓扑排序与自动反向传播 01:18

- ▶ 1:18:12 So this ordering of graphs can be achieved using something called topological sort. 而对图做这种排序，可以用一种叫做拓扑排序的方法来实现。

- ▶ 1:18:17 So topological sort is basically a laying out of a **graph** such that all the edges go only from left to right. 拓扑排序基本上就是把一张图铺展开来，使得所有的边都只从

graph

计算图 由变量和运算节点组成的有向无环图，记录计算过程并支持反向传播。

神经网络基础

- ▶ 1:18:24 left to right basically. So here we have a **graph**, it's a directed acyclic **graph**, a DAG, 左到右（大致如此）。比如这里有一张图，它是一个有向无环图，也就是 DAG，

graph

计算图 由变量和运算节点组成的有向无环图，记录计算过程并支持反向传播。

神经网络基础

- ▶ 1:18:31 and this is two different topological orders of it, I believe, where basically you'll see 我相信这是它的两种不同的拓扑序，基本上你会看到

- ▶ 1:18:36 that it's a laying out of the nodes such that all the edges go only one way from left to right. 它把节点铺展开来，使得所有的边都只朝一个方向，从左到右。

- ▶ 1:18:41 And implementing topological sort, you can look in Wikipedia and so on, I'm not going to go into the details of it. 至于拓扑排序的实现，你可以去维基百科之类的地方看，我不打算

- ▶ 1:18:46 through it in detail. But basically this is what builds a **topological graph**. We maintain a set of nodes and their dependencies. 详细讲它。但基本上这就是构建拓扑图的代码。我们维护一个

topological graph

拓扑图 由计算节点和依赖边构成、按拓扑顺序排列的图结构。

代码与工程

- ▶ 1:18:55 visited nodes and then we are going through starting at some root **node**, which for us is O, the set of visited nodes, then from some root node start traversing, for us this root node is O,

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

- ▶ 1:19:03 that's where we want to start the topological sort. And starting at O, we go through all of its children. 那就是我们想开始拓扑排序的地方。从 O 出发，我们遍历它所有的

- ▶ 1:19:08 **children** and we need to lay them out from left to right. And basically this starts at O, the root node, and we need to lay them out from left to right. Basically this process starts at O,

children

子节点 产生当前值的输入值集合，用来建立计算图中的连接关系。

反向传播与梯度

- ▶ 1:19:14 if it's not visited, then it marks it as visited. And then it iterates through all of its `children`
如果它还没被访问过，就把它标记为已访问。然后遍历它所有的子节点，

children

子节点 产生当前值的输入值集合，用来建立计算图中的连接关系。

反向传播与梯度

- ▶ 1:19:20 and calls `build_topo` on them. And then after it's gone through all the `children`,
对它们调用 `build_topo`。然后当它遍历完所有的子节点之后，

children

子节点 产生当前值的输入值集合，用来建立计算图中的连接关系。

反向传播与梯度

- ▶ 1:19:26 it adds itself. So basically this `node` that we're going to call it on, like say `O`,
它会把它自己加进去。所以基本上，我们要调用它的这个节点，比如 `O`，

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

- ▶ 1:19:33 is only going to add itself to the `topo` list after all of the `children` have been processed.
只有在所有子节点都被处理完之后，它才会把自己加到 `topo` 列表里。

children

子节点 产生当前值的输入值集合，用来建立计算图中的连接关系。

反向传播与梯度

- ▶ 1:19:39 And that's how this function is guaranteeing that you're only going to be in the list
这就是这个函数如何保证：你只会出现在列表里

- ▶ 1:19:43 once all of your `children` are in the list. And that's the invariant that is being maintained.
当你的所有子节点都已经在列表里之后。而这个不变式就是这样被维持的。

children

子节点 产生当前值的输入值集合，用来建立计算图中的连接关系。

反向传播与梯度

- ▶ 1:19:47 So if we build `topo` on `O` and then inspect this list, we're going to see that it ordered our
所以如果我们对 `O` 构建 `topo`，然后检查这个列表，我们会看到它把我们的

- 1:19:54 **value** objects. And the last one is the **value** of 0.707, which is the output. So this is O, value 对象排好了序。最后一个对象的值是 0.707，也就是输出。所以这个是 O，

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- 1:20:02 and then this is N. And then all the other nodes get laid out before it. 然后这个是 N。接着所有其他节点都会排在它前面。

- 1:20:08 So that builds the **topological graph**. And really what we're doing now is we're just calling dot 所以这就构建出了拓扑图。而实际上我们现在要做的，只是调用点号

topological graph

拓扑图 由计算节点和依赖边构成、按拓扑顺序排列的图结构。

代码与工程

- 1:20:15 underscore **backward** on all of the nodes in a **topological order**. So if we just reset the **gradients**, 下划线 backward，在拓扑顺序中对所有节点。所以如果我们只是重置梯度，

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

topological order

拓扑顺序 计算图中节点的一种线性排序，保证所有子节点都排在父节点之前，用于反向传播的依赖顺序。

代码与工程

- 1:20:22 they're all zero, what did we do? We started by setting O dot **grad** to be one. That's the base 它们就都为零。我们做了什么？我们先设置 O 点 grad 为 1。这就是基础

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- 1:20:32 case. Then we built the **topological order**. And then we went for **node** in reversed of topo. 情况。然后我们构建了拓扑顺序。接着我们遍历 reversed(topo) 中的节点。

topological order

拓扑顺序 计算图中节点的一种线性排序，保证所有子节点都排在父节点之前，用于反向传播的依赖顺序。

代码与工程

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

▶ 1:20:46 Now, in the reverse order, because this list goes from, you know, we need to go through it in 现在按逆序进行，因为这个列表从.....你知道，我们需要按逆序来遍历它。

▶ 1:20:52 reversed order. So starting at 0, `node` dot `backward`. And this should be it. There we go. 逆序。所以从 0 开始，`node` 点 `backward`。应该就是这样。好了。

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 1:21:05 Those are the correct derivatives. Finally, we are going to hide this functionality. So I'm going to 这些就是正确的导数。最后，我们要把这个功能隐藏起来。所以我要

▶ 1:21:11 copy this. And we're going to hide it inside the `value class`, because we don't want to have 复制这段。然后把它藏进 `Value` 类里，因为我们不想让

value class

`Value` 类 `micrograd` 中表示标量节点及其梯度与反向传播函数的类。

代码与工程

▶ 1:21:16 all that code lying around. So instead of an underscore `backward`, we're now going to 所有这些代码散落在外面。所以现在不再用下划线 `backward`，我们接下来要

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 1:21:20 define an actual `backward`. So that `backward` without the underscore. 定义一个真正的 `backward`。也就是不带下划线的那个 `backward`。

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 1:21:26 And that's going to do all the stuff that we just derived. So let me just clean this up a little bit. 它会完成我们刚刚推导出的所有内容。所以让我稍微清理一下这里。

- ▶ 1:21:30 So we're first going to build the `topological graph` starting at self. So build topo of self
我们首先会从 self 开始构建拓扑图。所以 build_topo(self)

topological graph

拓扑图 由计算节点和依赖边构成、按拓扑顺序排列的图结构。

代码与工程

- ▶ 1:21:44 will populate the `topological order` into the topo list, which is a local variable.
会把拓扑顺序填充进 topo 列表，而这个列表是个局部变量。

topological order

拓扑顺序 计算图中节点的一种线性排序，保证所有子节点都排在父节点之前，用于反向传播的依赖顺序。

代码与工程

- ▶ 1:21:49 Then we set self dot `grad` to be one. And then for each `node` in the reversed list,
然后我们把 self.grad 设为 1。接着对反转列表中的每个节点，

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

- ▶ 1:21:55 so starting at us, I'm going to call the `_backward`.
从我们自己开始，我会调用 _backward。

_backward

反向传播函数 Value 上保存的函数，负责把当前节点的梯度按链式法则传给输入。

反向传播与梯度

- ▶ 1:22:02 And that should be it. So save. Come down here. We define. Okay, all the grads are zero.
这样应该就可以了。保存一下。往下到这里，我们定义。好，所有梯度都是零。

- ▶ 1:22:13 And now what we can do is, call `.backward()` without the underscore. And there we go.
现在我们可以做的是，调用不带下划线的 .backward()。看，成功了。

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

梯度累积与零初始化 01:22

- ▶ 1:22:22 And that's **back propagation**. Plus, for one **neuron**. Now we shouldn't be too happy with ourselves, 这就是反向传播。另外，就一个神经元来说。现在我不该太得意，

back propagation

反向传播 利用链式法则从输出向输入高效计算损失函数梯度的算法。

反向传播与梯度

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:22:30 actually, because we have a bad bug. And we have not surfaced the bug because of some 其实，因为我们有一个严重的 bug。而且我们还没有让这个 bug 暴露出来，因为一些

- ▶ 1:22:35 specific conditions that we have. We have to think about right now. 我们所处的特定条件。我们现在必须仔细想想。

- ▶ 1:22:39 So here's the simplest case that shows the bug. Say I create a single **node** a, 所以这里有一个能展示这个 bug 的最简单例子。假设我创建一个单独的节点 a,

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

- ▶ 1:22:48 and then I create a B that is a plus a. And then I call it **backward**. So what's going to happen is 然后我创建一个 B，它等于 a 加 a。然后我调用它的 backward。那么会发生的是

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:22:56 A is three, and then b is a plus a. So there's two arrows on top of each other here. A 是 3，然后 b 是 a 加 a。所以这里有两个箭头叠在一起。

- ▶ 1:23:03 Then we can see that B is of course the **forward pass** works. B is just a plus a, which is six. 然后我们可以看到，B 当然，前向传播是正常的。B 就是 a 加 a，等于 6。

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

- ▶ 1:23:10 But the **gradient** here is not actually correct. That we calculated automatically.
但这里我们自动计算出来的梯度其实并不正确。

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:23:16 And that's because of course, just doing **calculus** in your head, the **derivative** of B with respect to A
这是因为它当然，用心算微积分就知道，B 对 A 的导数

calculus

微积分 研究变化率与累积量的数学分支。

数学与微积分

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 1:23:24 should be two. One plus one. It's not one. Intuitively, what's happening here, right? So B is
应该是 2。1 加 1。不是 1。直观地说，这里发生了什么，对吧？所以 B 是

- ▶ 1:23:32 the result of A plus a, and then we called **backward** on it. So let's go up and see what that does.
A 加 a 的结果，然后我们对它调用了 backward。所以我们往上看它做了什么。

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:23:43 B is the result of addition. So out is B. And then when we called **backward**, what happened is
B 是加法的结果。所以 out 是 B。然后当我们调用 backward 时，发生的是

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:23:50 self.**grad** was set to one, and then other.**grad** was set to one. But because we're
self.grad 被设为 1，然后 other.grad 也被设为 1。但因为我们

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 1:23:58 doing A plus a, self and other are actually the exact same object. So we are overriding the gradient.
做的是 A 加 a, self 和 other 实际上是同一个对象。所以我们在覆盖梯度。

gradient

梯度 损失函数对参数的变化率, 指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:24:05 We are setting it to one, and then we are setting it again to one. And that's why it stays at one.
我们把它设为 1, 然后又把它设为 1。这就是为什么它一直停留在 1。

- ▶ 1:24:13 So that's the problem. There's another way to see this in a little bit more complicated expression.
所以这就是问题所在。在稍微复杂一点的表达式里, 还有另一种方式看到这一点。

expression

表达式 由多个运算组合而成的计算式, 在计算图中对应多个节点。

代码与工程

- ▶ 1:24:21 So here we have a and b. And then d will be the multiplication of the two, and e will be the
所以这里我们有 a 和 b。然后 d 会是这两者的乘积, 而 e 会是这两者的

- ▶ 1:24:30 addition of the two. And then we multiply e times d to get f. And then we called f dot backward.
和。然后我们把 e 乘以 d 得到 f。然后我们调用 f 的 backward。

backward

反向传播 从输出向输入传播梯度, 利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:24:37 And these gradients, if you check, will be incorrect. So fundamentally what's happening here
而这些梯度, 如果你检查一下, 会是不正确的。所以从根本上, 这里发生的事情是

gradient

梯度 损失函数对参数的变化率, 指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:24:42 again is basically we're going to see an issue any time we use a variable more than once.
还是基本上, 只要我们多次使用一个变量, 就会看到一个问题。

- ▶ 1:24:49 Until now, in these expressions above, every variable is used exactly once. So we didn't
到目前为止, 在上面这些表达式里, 每个变量都只被使用了一次。所以我们没有

- ▶ 1:24:53 see the issue. But here, if a variable is used more than once, what's going to happen
看到这个问题。但在这里, 如果一个变量被使用了不止一次, 会发生什么

- ▶ 1:24:57 during **backward** pass? We're backpropagating from f to e to d, so far so good. But now e 在反向传播过程中? 我们从 f 反向传播到 e 再到 d, 到目前为止没问题。但现在 e

backward

反向传播 从输出向输入传播梯度, 利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:25:03 calls its **backward**, and it deposits its **gradients** to a and b. But then we come back to d and call 调用它的 backward, 并把它的梯度存入 a 和 b。但接着我们回到 d 并调用

backward

反向传播 从输出向输入传播梯度, 利用链式法则计算每个变量的梯度。

反向传播与梯度

gradient

梯度 损失函数对参数的变化率, 指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:25:09 **backward**, and it overwrites those **gradients** at a and b. So that's obviously a problem. backward, 它就会覆盖 a 和 b 上的那些梯度。所以这显然是个问题。

backward

反向传播 从输出向输入传播梯度, 利用链式法则计算每个变量的梯度。

反向传播与梯度

gradient

梯度 损失函数对参数的变化率, 指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:25:17 And the solution here, if you look at the multivariate case of the **chain rule** and its 而这里的解决方案, 如果你看链式法则的多元情形及其

chain rule

链式法则 微积分中计算复合函数导数的法则, 即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 1:25:22 **generalization** there, the solution there is basically that we have to accumulate these 推广, 那里的解决方案基本上就是我们必须累加这些

- ▶ 1:25:27 **gradients**, these **gradients** add. And so instead of setting those **gradients**, 梯度, 这些梯度会相加。所以与其直接设置那些梯度,

gradient

梯度 损失函数对参数的变化率, 指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

▶ 1:25:34 we can simply do `plus equals`. We need to accumulate those `gradients`. `Plus equals`, plus
我们可以简单地用 `+=`。我们需要累加这些梯度。`+=`, `+`

plus equals

加等于 (`+=`) 代码中在原值上累加的写法, 用于实现梯度累加, 避免覆盖已有梯度。

代码与工程

gradient

梯度 损失函数对参数的变化率, 指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

▶ 1:25:40 `equals`, `plus equals`, `plus equals`. And this will be okay, remember, because we are initializing
`=`, `+=`, `+=`。这样就可以了, 记住, 因为我们初始化

plus equals

加等于 (`+=`) 代码中在原值上累加的写法, 用于实现梯度累加, 避免覆盖已有梯度。

代码与工程

▶ 1:25:49 `them at zero`. So we start at zero, and then any contribution that flows backwards will simply add.
它们为零。所以我们从零开始, 然后任何反向流过来的贡献都会直接相加。

▶ 1:25:58 So now if we redefine this one, because the `plus equals`, this now works, because a dot `grad`
所以现在如果我们重新定义这个, 因为用了 `+=`, 这就能正常工作了, 因为 `a.grad`

plus equals

加等于 (`+=`) 代码中在原值上累加的写法, 用于实现梯度累加, 避免覆盖已有梯度。

代码与工程

grad

梯度 计算图中某个变量累积的导数, 表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 1:26:06 `started at zero`, and we called `b dot backward`, we deposit one, and then we deposit one again,
从零开始, 然后我们调用 `b` 的 `backward`, 我们存入 1, 然后我们又存入 1,

backward

反向传播 从输出向输入传播梯度, 利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 1:26:12 `and now this is two`, which is correct. And here, this will also work, and we'll get
现在它变成 2, 这是正确的。而这里, 这也会正常工作, 我们会得到

- ▶ 1:26:17 correct **gradients**, because when we call `e dot backward`, we will deposit the **gradients** from this
正确的梯度，因为当我们调用 `e` 的 `backward` 时，我们会把来自这个

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:26:21 `branch`, and then we get to `d dot backward`, it will deposit its own **gradients**, and then those
分支的梯度存入，然后我们到达 `d` 的 `backward`，它会存入它自己的梯度，然后那些

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:26:27 **gradients** simply add on top of each other. And so we just accumulate those **gradients**, and that
梯度就是直接叠加在一起。所以我们只是累加这些梯度，然后这

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:26:32 fixes the issue. Okay, now before we move on, let me actually do a bit of cleanup here and
就解决了这个问题。好，现在在继续之前，让我在这里稍微清理一下，

- ▶ 1:26:36 delete some of this intermediate work. So I'm not going to need any of this now that we've
删掉一些中间代码。既然我们已经

- ▶ 1:26:43 derived all of it. We are going to keep this, because I want to come back to it. Delete the **tanh**,
推导出全部内容了，我现在不再需要这些。这个我们要保留，因为我想回头再看。删掉 `tanh`，

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 1:26:51 delete our model and any example, delete this step, delete this, keep the code that draws,
删掉我们的模型和任何示例，删掉这一步，删掉这个，保留负责画图的代码，

- ▶ 1:26:59 and then delete this example, and leave behind only the definition of `value`.
然后删掉这个示例，只留下 Value 的定义。

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:27:05 And now let's come back to this `non-linearity` here that we implemented the `tanh`. Now I told
现在我们回到这里这个非线性部分，也就是我们实现的 tanh。我告诉过

non-linearity

非线性 神经网络中使模型能拟合复杂函数的弯曲变换。

神经网络基础

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 1:27:09 you that we could have broken down `tanh` into its explicit atoms in terms of other expressions
你，我们本可以把 tanh 拆解成由其他表达式构成的显式原子操作，

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 1:27:15 if we had the `exp` function. So if you remember, `tanh` is defined like this, and we chose to
前提是我们有 exp 函数。如果你还记得，tanh 是这样定义的，而我们选择

exp

指数函数 以自然常数 e 为底的指数函数，tanh 可以由它组合表达。

数学与微积分

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 1:27:21 develop `tanh` as a single function, and we can do that because we know its `derivative`,
把 tanh 开发成一个单一函数，我们能这样做是因为我们知道它的导数，

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

▶ 1:27:25 and we can **backpropagate** through it. But we can also break down **tanh** and express 并且可以通过它进行反向传播。但我们也可以拆解 tanh，把它表示

backpropagate

反向传播 利用链式法则从输出端向输入端逐层计算梯度，从而高效地训练神经网络。

反向传播与梯度

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

▶ 1:27:29 it as a function of **exp**. And I would like to do that now because I want to prove to you that 为 exp 的函数。我现在想这么做，因为我想向你证明

exp

指数函数 以自然常数 e 为底的指数函数，tanh 可以由它组合表达。

数学与微积分

▶ 1:27:33 you get all the same results and all the same **gradients**, but also because it forces us 你会得到完全相同的结果和完全相同的梯度，同时也因为它迫使我们

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

▶ 1:27:38 to implement a few more expressions. It forces us to do **exponentiation**, addition, subtraction, 再实现一些表达式。它迫使我们实现幂运算、加法、减法、

exponentiation

指数运算 计算 e 的幂的运算，是 tanh 等函数的基础运算。

数学与微积分

▶ 1:27:43 **division**, and things like that. And I think it's a good exercise to go through a few more of these. 除法之类的操作。我觉得多过一遍这些内容是很好的练习。

division

除法 一种二元运算，可以改写为乘以倒数的形式。

数学与微积分

▶ 1:27:48 Okay, so let's scroll up to the definition of **value**. And here one thing that we currently 好，那让我们向上滚动到 Value 的定义。这里目前有一件事我们

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:27:53 can't do is we can do like a `value` of say 2.0, but we can't do, you know, here for 做不到：我们可以创建一个比如 2.0 的 Value，但我们不能，你知道，比如这里

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:28:00 example, we want to add `constant` 1, and we can't do something like this. And we can't do 我们想加上常数 1，但我们不能这样做。我们不能这么做

constant

常数 在表达式中固定的数值，如整数或浮点数。

数学与微积分

支持数字类型与更多运算 01:28

- ▶ 1:28:05 it because it's just int object has no attribute data. That's because `a + 1` comes right here to `__add__`, 是因为 int 对象没有 data 属性。这是因为 `a + 1` 会直接走到 `__add__` 这里，
- ▶ 1:28:12 and then other is the integer 1. And then here Python is trying to access `one.data`, 然后 other 是整数 1。接着 Python 在这里试图访问 `one.data`，
- ▶ 1:28:17 and that's not a thing. That's because basically one is not a `Value object`, and we only have addition 这行不通。因为基本上 1 并不是一个 Value 对象，而我们只给 Value 对象实现了加法。

value object

Value 对象 micrograd 中包装标量数值并记录计算图关系的对象。

代码与工程

- ▶ 1:28:22 for `Value` objects. So as a matter of convenience, so that we can create expressions like this and 只给 Value 对象实现了加法。所以为了方便，为了让我们能写出这样的表达式，并

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:28:27 make them make sense, we can simply do something like this. Basically, we let other alone, 让它们说得通，我们可以简单地这样做。基本上，我们让 other 保持原样，

- ▶ 1:28:35 if other is an `instance` of `Value`, but if it's not an `instance` of `Value`, we're going to assume
如果 other 是 Value 的实例，但如果它不是 Value 的实例，我们就假设

instance

实例 由某个类创建出来的对象；这里用于判断 other 是否已经是 Value。

代码与工程

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:28:39 that it's a number like an integer or `float`, and we're going to simply wrap it in `Value`.
它是一个整数或浮点数这样的数字，然后简单地把它包装成 Value。

float

浮点数 带小数部分的数值类型，与整数一起可作为标量输入。

数学与微积分

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:28:44 And then other will just become `Value` of other, and then other will have a `data attribute`,
然后 other 就会变成 Value(other)，接着 other 就会有 data 属性，

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

data attribute

数据属性 对象中保存实际数值的成员，例如 Value 的 .data。

代码与工程

- ▶ 1:28:48 and this should work. So if I just say a plus 1, then this should work.
这样应该就能工作了。所以如果我只写 $a + 1$ ，这应该就能工作。

- ▶ 1:28:53 There we go. Okay, now let's do the exact same thing for multiply, because we can't do
很好。好，现在我们对乘法做完全一样的事，因为我们不能做

- ▶ 1:28:57 something like this again, for the exact same reason. So we just have to go to mul, and
同样的事情，原因完全相同。所以我们只需去 mul 里，然后

- ▶ 1:29:03 if other is not a `Value`, then let's wrap it in `Value`. Let's redefine `Value`, and now this works.
如果 other 不是 `Value`, 就把它包装成 `Value`。重新定义 `Value`, 现在这就工作了。

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:29:10 Now here's a kind of unfortunate and not obvious part. `A times two` works, we saw that, but `two times` 现在有个有点不幸且不那么明显的地方。`A 乘以 2` 能工作, 我们看到了, 但 `2 乘以`
- ▶ 1:29:16 `A`, is that going to work? You'd expect it to, right? But actually it will not. And the reason `A`, 这能工作吗? 你会觉得应该能, 对吧? 但实际上它不行。原因是
- ▶ 1:29:23 it won't is because Python doesn't know, like when you do `A times two`, basically, so `A times` 它不行是因为 Python 不知道, 比如当你做 `A 乘以 2` 时, 基本上, 所以 `A 乘以`
- ▶ 1:29:30 `two`, Python will go and it will basically do something like `A dot mul of two`. That's basically `2` 时, Python 会去调用类似 `A 点 mul 二` 这样的东西。那基本上就是
- ▶ 1:29:36 what it will call. But to it, `two times A` is the same as `two dot mul of A`. And it doesn't, 它会调用的东西。但对它来说, `2 乘以 A` 等同于 `2 点 mul A`。而它不会,
- ▶ 1:29:42 `two` can't multiply `Value`. And so it's really confused about that. So instead, what happens `2` 不能乘 `Value`。所以它对此很困惑。所以相反, 实际发生的是

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:29:48 is in Python, the way this works is you are free to define something called the `rmul`.
在 Python 中, 机制是你完全可以定义所谓的 `rmul`。
- ▶ 1:29:54 And `rmul` is kind of like a `fallback`. So if Python can't do `two times A`, it will check if `rmul` 有点像回退机制。所以如果 Python 不能做 `2 乘以 A`, 它会检查

fallback

回退机制 当主方法无法处理运算时, Python 转向调用的备用实现。

代码与工程

- ▶ 1:30:02 by any chance `A` knows how to multiply `two`. And that will be called into `rmul`.
`A` 是否恰好知道怎么乘以 `2`。那就会调用到 `rmul`。

- ▶ 1:30:08 So because Python can't do two times A, it will check is there an `rmul` in `Value`? And because 所以因为 Python 不能做 2 乘以 A, 它会检查 Value 里有没有 `rmul`? 而因为

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:30:14 there is, it will now call that, and what we'll do here is we will swap the order of the operands. 有的话, 它现在就会调用那个, 而我们这里要做的就是交换操作数的顺序。

- ▶ 1:30:20 So basically, two times a will redirect to `__rmul__` and `__rmul__` basically call a times two. 所以基本上, $2 * a$ 会重定向到 `__rmul__`, 而 `__rmul__` 基本上会调用 $a * 2$ 。

`__rmul__`

反向乘法方法 Python 中当右操作数为自定义对象时, 用于处理乘法运算符重载的特殊方法。

代码与工程

- ▶ 1:30:26 And that's how that will work. So redefining that with `__rmul__`, two times a becomes four. Okay, 这就是它的工作方式。那么用 `__rmul__` 重新定义之后, $2 * a$ 就变成 4。好,

`__rmul__`

反向乘法方法 Python 中当右操作数为自定义对象时, 用于处理乘法运算符重载的特殊方法。

代码与工程

- ▶ 1:30:32 now looking at the other elements that we still need, we need to know how to exponentiate and 现在看看我们仍然需要的其他要素, 我们需要知道如何做指数运算, 以及

- ▶ 1:30:36 how to divide. So let's first do the explanation part. We're going to introduce single function `exp` here.

如何做除法。所以先讲解释部分。我们要引入单个函数 `exp`。

exp

指数函数 以自然常数 e 为底的指数函数, `tanh` 可以由它组合表达。

数学与微积分

- ▶ 1:30:43 And `exp` is going to mirror `tanh` in the sense that it's a simple function that 而 `exp` 会模仿 `tanh`, 从某种意义上说, 它是一个简单函数,

exp

指数函数 以自然常数 e 为底的指数函数, `tanh` 可以由它组合表达。

数学与微积分

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数, 常用作神经网络的激活函数。

数学与微积分

- ▶ 1:30:49 transforms a single `scalar value` and outputs a single `scalar value`. So we pop out the Python 把单个标量值变换后输出单个标量值。所以我们取出 Python

scalar

标量 单个数值，区别于向量、矩阵或更高维张量。

数学与微积分

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:30:54 number, we use `math.exp` to exponentiate it, create a new `value object`, everything that we've 数字，用 `math.exp` 对它做指数运算，再创建一个新的 Value 对象，这些都是我们

exp

指数函数 以自然常数 e 为底的指数函数，`tanh` 可以由它组合表达。

数学与微积分

value object

Value 对象 micrograd 中包装标量数值并记录计算图关系的对象。

代码与工程

- ▶ 1:30:59 seen before. The tricky part, of course, is how do you `backpropagate` through e to the x ? 之前见过的。当然，棘手的地方在于，如何通过 e^x 做反向传播？

backpropagate

反向传播 利用链式法则从输出端向输入端逐层计算梯度，从而高效地训练神经网络。

反向传播与梯度

- ▶ 1:31:04 And so here, you can potentially pause the video and think about what should go here. 所以在这里，你可以暂停视频，想一想这里应该填什么。

- ▶ 1:31:13 Okay, so basically, we need to know what is the `local derivative` of e to the x . So d by dx of e to 好，所以基本上，我们需要知道 e^x 的局部导数是什么。 e^x

local derivative

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

- ▶ 1:31:19 the x is famously just e to the x . And we've already just calculated e to the x . And it's x 对 x 的导数，众所周知就是 e^x 。而我们刚刚已经计算出了 e^x 。它

▶ 1:31:25 inside out.data. So we can do out.data times out.grad, that's the chain.
就在 out.data 里面。所以我们可以做 out.data 乘以 out.grad，这就是链式法则。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 1:31:32 So we're just chaining on to the current running grad. And this is what the expression looks like.
所以我们只是把梯度链式累加到当前运行中的 grad 上。这个表达式看起来

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

▶ 1:31:36 like. It looks a little confusing, but this is what it is. And that's the explanation. So
就是这样。它看起来有点令人困惑，但事实就是这样。解释完了。那么

▶ 1:31:42 redefining, we should now be able to call a.exp. And hopefully the backward pass works as well.
重新定义后，我们现在应该能够调用 a.exp 了。希望反向传播也能正常工作。

exp

指数函数 以自然常数 e 为底的指数函数，tanh 可以由它组合表达。

数学与微积分

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 1:31:48 Okay, and the last thing we'd like to do, of course, is we'd like to be able to divide.
好，当然，我们想做的最后一件事就是能够做除法。

▶ 1:31:52 Now, I actually will implement something slightly more powerful than division,
现在，我实际上会实现一个比除法稍微更强大的东西，

division

除法 一种二元运算，可以改写为乘以倒数的形式。

数学与微积分

- ▶ 1:31:55 because **division** is just a special case of something a bit more powerful.
因为除法只是某个更强大东西的特例。

division

除法 一种二元运算，可以改写为乘以倒数的形式。

数学与微积分

- ▶ 1:31:59 So in particular, just by rearranging, if we would have some kind of a / b equals
所以特别地，只需重新排列，如果有某种 a / b 等于

- ▶ 1:32:04 **value** of 4.0 here, we'd like to basically be able to do a divide b. And we'd like this to be able to
这里的值是 4.0，我们基本希望能够做 a 除以 b 。并且我们希望它能够

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:32:09 give us 0.5. Now, **division** actually can be reshuffled as follows. If we have a divide b, that's
给我们 0.5。现在，除法实际上可以像下面这样重新排列。如果有 a 除以 b ，那

division

除法 一种二元运算，可以改写为乘以倒数的形式。

数学与微积分

- ▶ 1:32:17 actually the same as a multiplying one over b. And that's the same as a multiplying b to the
实际上等同于 a 乘以 $1/b$ 。而这又等同于 a 乘以 b 的

- ▶ 1:32:22 power of negative one. And so what I'd like to do instead is I basically like to implement
-1 次方。所以我想做的替代方案是，我基本上想实现

- ▶ 1:32:27 the operation of x to the k for some **constant** k . So it's an integer or a **float**.
 x 的 k 次方这个运算，其中 k 是某个常数。所以它是整数或浮点数。

constant

常数 在表达式中固定的数值，如整数或浮点数。

数学与微积分

float

浮点数 带小数部分的数值类型，与整数一起可作为标量输入。

数学与微积分

- ▶ 1:32:34 And we would like to be able to differentiate this. And then as a special case, 并且我们希望能够对它求导。然后作为一个特例,

differentiate

求导 计算函数导数的过程, 在反向传播中用于得到梯度。

反向传播与梯度

- ▶ 1:32:38 negative one will be division. And so I'm doing that just because it's more general and -1 就会变成除法。我这样做只是因为这样更通用, 而且

division

除法 一种二元运算, 可以改写为乘以倒数的形式。

数学与微积分

- ▶ 1:32:44 yeah, you might as well do it that way. So basically what I'm saying is we can redefine 是的, 你不妨就那样做。所以基本上我想说的是, 我们可以重新定义

- ▶ 1:32:49 division, which we will put here somewhere. Yeah, we can put it here somewhere. What I'm 除法, 我们会把它放在这里某个地方。是的, 我们可以把它放在这里某个地方。我

division

除法 一种二元运算, 可以改写为乘以倒数的形式。

数学与微积分

- ▶ 1:32:56 saying is that we can redefine division. So self divide other can actually be rewritten as self times 想说的是, 我们可以重新定义除法。所以 self 除以 other 实际上可以重写成 self 乘以

division

除法 一种二元运算, 可以改写为乘以倒数的形式。

数学与微积分

- ▶ 1:33:03 other to the power of negative one. And now value raised to the power of negative one, other 的 -1 次方。而现在 value 的 -1 次方,

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:33:09 we have to now define that. So here's so we need to implement the power function. 我们现在必须定义它。所以这里, 我们需要实现 power 函数。

power function

幂函数 计算某个数或 Value 的整数/浮点数次方的函数。

数学与微积分

▶ 1:33:15 Where am I going to put the `power function` ? Maybe here somewhere.
我要把 `power` 函数放在哪里呢？也许这里某个地方。

power function

幂函数 计算某个数或 `Value` 的整数/浮点数次方的函数。

数学与微积分

▶ 1:33:20 This is this column for it. So this function will be called when we try to raise a `value` to
这一列就是留给它的。所以当我们试图把一个值提升到

value

`Value` 对象 `micrograd` 中表示标量及其梯度、参与计算图的基本类。

代码与工程

▶ 1:33:25 some power and other will be that power. Now I'd like to make sure that other is only an int or a
某个幂时，这个函数就会被调用，而 `other` 将会是那个幂。现在我想确保 `other` 只能是 int 或

▶ 1:33:31 `float` . Usually other is some kind of a different `Value object` . But here other will be forced to be
`float`。通常 `other` 是某种不同的 `Value` 对象。但在这里 `other` 会被强制为

float

浮点数 带小数部分的数值类型，与整数一起可作为标量输入。

数学与微积分

value object

`Value` 对象 `micrograd` 中包装标量数值并记录计算图关系的对象。

代码与工程

▶ 1:33:37 an int or a `float` . Otherwise the math won't work for what we're trying to achieve in the specific
int 或 `float`。否则，数学运算就无法实现我们在特定情况下想要达到的目的。

float

浮点数 带小数部分的数值类型，与整数一起可作为标量输入。

数学与微积分

▶ 1:33:43 case. That would be a different `derivative expression` if we wanted other to be a `Value` .
情况。如果我们想让 `other` 是一个 `Value`，那就会是不同的导数表达式。

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

1:33:49 So here we create a `Value`, which is just, you know, this data raised to the
所以这里我们创建一个 `Value`，它就只是，你知道，这个 `data` 的

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

1:33:53 power of other and other here could be for example negative one. That's what we are hoping to achieve.

other 次方，而这里的 other 可以是，例如，-1。这就是我们希望实现的结果。

1:33:59 And then this is the `backward` stub. And this is the fun part, which is what is the `chain rule`

然后这就是 `backward` 的桩函数。接下来是有趣的部分，也就是链式法则

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

1:34:05 expression here for backpropagating through the power function where the power is

这里的表达式，用于通过幂函数进行反向传播，其中幂次

expression

表达式 由多个运算组合而成的计算式, 在计算图中对应多个节点。

代码与工程

power function

幂函数 计算某个数或 Value 的整数/浮点数次方的函数。

数学与微积分

1:34:13 to the power of some kind of a constant. So this is the exercise and maybe pause the video
是某个常数的幂。所以这是一个练习，也许你可以暂停视频

constant

常数 在表达式中固定的数值，如整数或浮点数。

数学与微积分

1:34:17 here and see if you can figure it out yourself as to what we should put here. Okay, so
在这里看看你能否自己弄清楚这里应该填什么。好，那么

- ▶ 1:34:27 you can actually go here and look at the **derivative** rules as an example. And we see lots of the
你实际上可以到这里看看导数法则作为例子。我们会看到很多

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

- ▶ 1:34:33 **derivative** rules that you can hopefully know from **calculus**. In particular, what we're looking for is the
power
你希望从微积分里知道的导数法则。特别地，我们要找的是幂

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

calculus

微积分 研究变化率与累积量的数学分支。

数学与微积分

- ▶ 1:34:38 rule. Because that's telling us that if we're trying to take d/dx of x to the n , which is
法则。因为它告诉我们，如果我们试图对 x 的 n 次方求 d/dx ，这也就是
- ▶ 1:34:43 what we're doing here, then that is just n times x to the n minus one, right? Okay. So that's
我们在这里做的事情，那么它就等于 n 乘以 x 的 n 减 1 次方，对吧？好，所以这
- ▶ 1:34:51 telling us about the **local derivative** of this power operation. So all we want here, basically n is now
告诉我们这个幂运算的局部导数。所以我们这里要的，基本上 n 现在就是

local derivative

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

- ▶ 1:34:59 other and `self.data` is x . And so this now becomes other, which is n times `self.data`, which is now
other，而 `self.data` 就是 x 。于是现在这就变成 other，也就是 n 乘以 `self.data`，而它现在是
- ▶ 1:35:10 a Python int or a **float**. It's not a **Value object**. We're accessing the **data attribute** raised to
一个 Python int 或 float。它不是 Value 对象。我们正在访问 data 属性，把它提升到

float

浮点数 带小数部分的数值类型，与整数一起可作为标量输入。

数学与微积分

value object

Value 对象 micrograd 中包装标量数值并记录计算图关系的对象。

代码与工程

▶ 1:35:17 the power of other minus one or n minus one. I can put brackets around this, but this doesn't matter
other 减 1 次方，也就是 n 减 1 次方。我可以在这里加括号，但这没关系

▶ 1:35:23 because power takes precedence over multiply and by the way. So that would have been okay. And that's
因为幂运算的优先级高于乘法，顺便说一句。所以那样本来也可以。而这

▶ 1:35:29 the **local derivative** only. But now we have to chain it. And we chain it just simply by
只是局部导数。但现在我们必须把它链起来。而我们链起来的方式就是简单地通过

local derivative

局部导数 计算图中某个运算节点对其直接输出变量的导数，不涉及图的其他部分。

反向传播与梯度

▶ 1:35:33 multiplying by out.**grad**. That's **chain rule**. And this should technically work.
乘以 out.grad。这就是链式法则。技术上这应该能工作。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

▶ 1:35:40 And we're gonna find out soon. But now if we do this, this should now work.
我们很快就会知道。但现在如果我们这样做，这应该就能工作了。

▶ 1:35:46 And we get point five. So the **forward pass** works, but does the **backward** pass work? And I realized
我们得到 0.5。所以前向传播能工作，但反向传播能工作吗？我意识到

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 1:35:51 that we actually also have to know how to subtract. So right now, a minus b will not work to make it
我们其实还必须知道如何做减法。所以现在，a 减 b 还不能工作，要让它

▶ 1:35:57 work. We need one more piece of code here. And basically, this is the subtraction. And the way
工作，我们还需要再加一段代码。基本上，这就是减法。而

- ▶ 1:36:06 we're going to implement subtraction is we're going to implement it by addition of a negation.
我们实现减法的方式，就是通过加上一个取负值来实现。
- ▶ 1:36:10 And then to implement negation, we're going to multiply by negative one. So just again,
然后为了实现取负，我们要乘以负一。所以还是同样地，
- ▶ 1:36:14 using the stuff we've already built and just expressing it in terms of what we have,
使用我们已经构建好的东西，并且只用我们已有的东西来表达它，
- ▶ 1:36:18 and a minus b does not work. Okay, so now let's scroll again to this **expression** here for this **neuron**.
而 a 减 b 是行不通的。好，那么现在让我们再次滚动到这个神经元的这个表达式。

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:36:25 And let's just compute the **backward** pass here once we've defined O. And let's draw it.
一旦我们定义了 O，我们就来计算这里的反向传播。然后把它画出来。

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:36:32 So here's the **gradients** for all of these **leaf nodes** for this two dimensional **neuron**
所以这是这个二维神经元的所有这些叶子节点的梯度

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

leaf node

叶子节点 计算图中没有输入边的节点，通常代表输入变量或可训练参数。

神经网络基础

- ▶ 1:36:35 that has a **tanh** that we've seen before. So now what I'd like to do is I'd like to
它有一个我们之前见过的 tanh。那么现在我想做的是，我想

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 1:36:40 break up this `tanh` into this `expression` here. So let me copy paste this here. And now instead of 把这个 `tanh` 拆解成这里的这个表达式。所以让我把这里复制粘贴过来。现在不是

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 1:36:48 we'll preserve the label, and we will change how we define O. So in particular, we're going to 我们会保留标签，然后改变我们定义 O 的方式。所以具体来说，我们要

用基础运算重写激活函数 01:36

- ▶ 1:36:55 implement this formula here. So we need e to the two x minus one over e to the two x plus one. 实现这里的这个公式。所以我们需要 e 的 $2x$ 次方减一，除以 e 的 $2x$ 次方加一。
- ▶ 1:37:00 So e to the two x, we need to take two times x and we need to exponentiate it. That's e to 所以 e 的 $2x$ 次方，我们需要取 2 乘以 x，然后对它做指数运算。这就是 e 的
- ▶ 1:37:07 the two x. And then because we're using it twice, let's create an `intermediate variable` e, $2x$ 次方。然后因为我们要用它两次，让我们创建一个中间变量 e，

intermediate variable

中间变量 在依赖链中起传递作用的变量，例如 $Z \rightarrow Y \rightarrow X$ 中的 Y。

数学与微积分

- ▶ 1:37:12 and then define O as e minus one over e plus one. 然后定义 O 为 e 减一除以 e 加一。
- ▶ 1:37:22 And that should be it. And then we should be able to `draw_dot` of O. So now before I run this, 应该就是这样。然后我们应该能够对 O 调用 `draw_dot`。那么在我运行这个之前，

draw_dot

`draw_dot` micrograd 中把 Value 计算图导出为 Graphviz 图形的辅助函数。

代码与工程

- ▶ 1:37:28 what do we expect to see? Number one, we're expecting to see a much longer 我们期望看到什么？第一，我们期望看到一个长得多的

- ▶ 1:37:33 **graph** here because we've broken up **tanh** into a bunch of other operations. But those operations 图, 因为我们已经把 tanh 拆解成了一堆其他操作。但这些操作

graph

计算图 由变量和运算节点组成的有向无环图, 记录计算过程并支持反向传播。

神经网络基础

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数, 常用作神经网络的激活函数。

数学与微积分

- ▶ 1:37:38 are mathematically equivalent. And so what we're expecting to see is number one, the same result 在数学上是等价的。所以我们期望看到的是, 第一, 相同的结果

- ▶ 1:37:44 here. So the **forward pass** works. And number two, because of that mathematical equivalence, 这里。所以前向传播能正常工作。第二, 由于这种数学等价性,

forward pass

前向传播 按计算图从输入计算到输出, 得到预测值或损失。

神经网络基础

- ▶ 1:37:48 we expect to see the same **backward** pass and the same **gradients** on these **leaf nodes**. 我们期望看到相同的反向传播, 以及这些叶子节点上相同的梯度。

backward

反向传播 从输出向输入传播梯度, 利用链式法则计算每个变量的梯度。

反向传播与梯度

gradient

梯度 损失函数对参数的变化率, 指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:37:52 So these **gradients** should be identical. So let's run this. So number one, let's verify that 所以这些梯度应该是完全相同的。那么让我们运行这个。第一, 让我们验证

gradient

梯度 损失函数对参数的变化率, 指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:38:00 instead of a single **tanh node**, we have now x and we have plus, we have times negative one.
现在我们不是单个 tanh 节点，而是有 x，有加号，有乘以负一。

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

node

节点 计算图中的基本单元，表示一个值或一次运算。

神经网络基础

- ▶ 1:38:07 This is the **division**. And we end up with the same **forward pass** here. And then the **gradients**,
这是除法。我们在这里得到了相同的前向传播结果。然后梯度，

division

除法 一种二元运算，可以改写为乘以倒数的形式。

数学与微积分

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

- ▶ 1:38:12 we have to be careful because they're in slightly different order potentially.
我们得小心，因为它们的顺序可能略有不同。

- ▶ 1:38:16 The **gradients** for W2 X2 should be zero and point five, W2 and X2 are zero and point five.
W2 和 X2 的梯度应该是 0.5，W2 和 X2 都是 0.5。

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:38:22 And W1 X1 are one and negative one point five, one and negative one point five. So that
而 W1 和 X1 是 1 和 -1.5，1 和 -1.5。所以

- ▶ 1:38:27 means that both our forward passes and **backward** passes were correct because this turned out to be
这意味着我们的前向传播和反向传播都是正确的，因为结果证明它

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:38:32 equivalent to `tanh` before. And so the reason I wanted to go through this exercise is number one, 等价于之前的 `tanh`。我之所以想讲这个练习，第一，

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 1:38:38 we got to practice a few more operations and writing more backwards passes. And number two, 我们练习了更多运算，也写了更多反向传播。第二，

- ▶ 1:38:44 I wanted to illustrate the point that the level at which you implement your operations is 我想说明一点：你在什么层级实现你的运算

- ▶ 1:38:50 totally up to you. You can implement `backward` passes for tiny expressions like a single individual 完全由你决定。你可以为非常小的表达式实现反向传播，比如单个的

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:38:54 plus or a single times, or you can implement that for say `tanh`, which is a kind of a potentially 加号或单个乘号，或者你也可以为比如 `tanh` 实现反向传播，它可以说是一种潜在的

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 1:39:01 you can see it as a composite operation because it's made up of all these more atomic operations. 你可以把它看作一个复合运算，因为它由所有这些更原子的运算组成。

- ▶ 1:39:06 But really, all of this is kind of like a fake concept. All that matters is we have some kind 但实际上，所有这些都是某种虚假的概念。重要的是我们有某种

对比 PyTorch 自动求导 01:39

- ▶ 1:39:09 of inputs and some kind of an output. And this output is a function of the inputs in some way. 输入和某种输出。而这个输出以某种方式作为输入的函数。

- ▶ 1:39:13 And as long as you can do **forward pass** and the **backward** pass of that little operation, 只要你能够对那个小运算做前向传播和反向传播,

forward pass

前向传播 按计算图从输入计算到输出, 得到预测值或损失。

神经网络基础

backward

反向传播 从输出向输入传播梯度, 利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:39:18 it doesn't matter what that operation is and how composite it is. If you can write the 那个运算是什么、有多复合都不重要。如果你能写出

- ▶ 1:39:23 **local gradients**, you can chain the **gradient** and you can continue **backpropagation**. So the design 局部梯度, 你就能把梯度链起来, 并继续反向传播。所以

local gradient

局部梯度 某个运算节点对其输入变量的导数, 只由该节点自身的运算决定。

反向传播与梯度

gradient

梯度 损失函数对参数的变化率, 指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:39:28 of what those functions are is completely up to you. So now I would like to show you how you can do 这些函数的设计完全由你决定。现在我想向你们展示如何

- ▶ 1:39:33 the exact same thing by using a modern deep **neural network** library, like for example, **PyTorch**, 使用现代深度神经网络库来做完全相同的事情, 比如 PyTorch,

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

pytorch

PyTorch 主流的深度学习框架, micrograd 的接口设计大致仿照它。

代码与工程

▶ 1:39:38 which I've roughly modeled `micrograd` by. And so `PyTorch` is something you would use in production.

micrograd 大致就是仿照它来建模的。PyTorch 是你在生产环境中会用到的东西。

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

▶ 1:39:45 And I'll show you how you can do the exact same thing, but in `PyTorch` API. So I'm just going 我会展示如何做完全相同的事情，但使用 PyTorch API。所以我现在就

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

▶ 1:39:50 to copy paste it in and walk you through it a little bit. This is what it looks like. 把它复制粘贴进来，然后稍微带你们过一遍。它看起来是这样的。

▶ 1:39:54 So we're going to import `PyTorch`. And then we need to define these `value` objects like we have here. 所以我们要导入 PyTorch。然后我们需要像这里一样定义这些 Value 对象。

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

▶ 1:40:01 Now `micrograd` is a `scalar` valued engine. So we only have `scalar` values like 2.0. 现在 micrograd 是一个标量值引擎。所以我们只有像 2.0 这样的标量值。

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

scalar

标量 单个数值，区别于向量、矩阵或更高维张量。

数学与微积分

- ▶ 1:40:08 But in `PyTorch`, everything is based around tensors. And like I mentioned, tensors are just
但在 `PyTorch` 中，一切都是围绕张量构建的。就像我提到的，张量只是

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

- ▶ 1:40:13 `n` dimensional arrays of scalars. So that's why things get a little bit more complicated here.
`n` 维的标量数组。所以这就是为什么这里会稍微复杂一些。

- ▶ 1:40:18 I just need a `scalar` value to `tensor` with just a single element. But by default,
我只需要一个标量值变成张量，一个只有单个元素的张量。但默认情况下，

scalar

标量 单个数值，区别于向量、矩阵或更高维张量。

数学与微积分

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:40:24 when you work with `PyTorch`, you would use more complicated tensors like this. So if I import
当你使用 `PyTorch` 时，你会使用像这样更复杂的张量。所以如果我导入

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

- ▶ 1:40:31 `PyTorch`, then I can create tensors like this. And this `tensor`, for example, is a two by three
`PyTorch`，那么我可以创建像这样的张量。例如，这个张量是一个二乘三

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

tensor

张量 多维数组，是深度学习框架中组织标量并进行高效并行运算的基本数据结构。

神经网络基础

- ▶ 1:40:38 array of scalars in a single compact representation. So we can check its shape. We see that it's
的标量数组，采用单一紧凑的表示。所以我们可以检查它的形状。我们看到它是

- ▶ 1:40:46 a two by three array and so on. So this is usually what you would work with in the actual libraries.
一个二乘三的数组，等等。所以这通常是你在实际库中会使用的东西。

- ▶ 1:40:53 So here I'm creating a `tensor` that has only a single element 2.0. And then I'm casting it to be double
所以这里我创建了一个只有一个元素 2.0 的张量。然后我把它转换为 double

tensor

张量 多维数组，是深度学习框架中组织标量并进行高效并行运算的基本数据结构。

神经网络基础

- ▶ 1:41:03 because Python is by default using double precision for its floating point numbers.
因为 Python 默认对其浮点数使用双精度。

- ▶ 1:41:07 So I like everything to be identical by default. The data type of these tensors will be `float`
所以我希望一切都默认相同。这些张量的数据类型将是 float

float

浮点数 带小数部分的数值类型，与整数一起可作为标量输入。

数学与微积分

- ▶ 1:41:13 32. So it's only using a single precision `float`. So I'm casting it to double so that we have `float`
32. 所以它只使用单精度浮点数。所以我将其转换为 double，这样我们就有 float

float

浮点数 带小数部分的数值类型，与整数一起可作为标量输入。

数学与微积分

- ▶ 1:41:20 64 just like in Python. So I'm casting to double and then we get something similar to `value` of 2.
64，就像在 Python 中一样。所以我转换为 double，然后我们得到类似于 value 为 2 的东西。

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 1:41:28 The next thing I have to do is because these are `leaf nodes`, by default `PyTorch` assumes that they
接下来我要做的是，因为这些是叶节点，默认情况下 PyTorch 假设它们

leaf node

叶子节点 计算图中没有输入边的节点，通常代表输入变量或可训练参数。

神经网络基础

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

- ▶ 1:41:32 do not require `gradients`. So I need to explicitly say that all of these nodes require `gradients`.
不需要梯度。所以我需要明确说明所有这些节点都需要梯度。

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:41:38 Okay, so this is going to construct `scalar`, valued, one element tensors. Make sure that `PyTorch`
好的，所以这将构造标量值的单元元素张量。确保 `PyTorch`

scalar

标量 单个数值，区别于向量、矩阵或更高维张量。

数学与微积分

pytorch

`PyTorch` 主流的深度学习框架，`micrograd` 的接口设计大致仿照它。

代码与工程

- ▶ 1:41:44 knows that they require `gradients`. Now by default these are set to false by the way because of
知道它们需要梯度。顺便说一下，默认情况下这些被设置为 `false`，因为

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:41:48 efficiency reasons because usually you would not want `gradients` for `leaf nodes` like the inputs
效率原因，因为通常你不会想要像输入这样的叶节点的梯度。

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

leaf node

叶子节点 计算图中没有输入边的节点，通常代表输入变量或可训练参数。

神经网络基础

- ▶ 1:41:54 to the network. And this is just trying to be efficient in the most common cases.
到网络。而这只是试图在最常见的情况下保持高效。

- ▶ 1:41:59 So once we've defined all of our values in `PyTorch` land, we can perform arithmetic just
所以一旦我们在 `PyTorch` 领域定义了所有的值，我们就可以进行算术运算，就像

pytorch

`PyTorch` 主流的深度学习框架，`micrograd` 的接口设计大致仿照它。

代码与工程

▶ 1:42:03 like we can here in `micrograd` land. So this would just work. And then there's a `torch.Tensor` also. 我们在 `micrograd` 领域这里所做的一样。所以这就能直接工作。然后还有一个 `torch.Tensor`。

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

tensor

张量 多维数组，是深度学习框架中组织标量并进行高效并行运算的基本数据结构。

神经网络基础

▶ 1:42:09 And when we get back as a `tensor` again, and we can just like in `micrograd`, it's got a `data` attribute and it's got `grad` attributes.

而当我们再次得到一个张量时，就像在 `micrograd` 中一样，它有一个 `data` 属性，还有一个 `grad` 属性。

tensor

张量 多维数组，是深度学习框架中组织标量并进行高效并行运算的基本数据结构。

神经网络基础

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

▶ 1:42:16 So these `tensor` objects just like in `micrograd` have a `data` attribute. 所以这些张量对象就像在 `micrograd` 中一样，有一个 `.data`

tensor

张量 多维数组，是深度学习框架中组织标量并进行高效并行运算的基本数据结构。

神经网络基础

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

▶ 1:42:21 And a dot `grad`. And the only difference here is that we need to call a dot item because of a `grad`. 这里唯一的区别是，我们需要调用 `.item`，因为

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 1:42:27 otherwise `PyTorch` dot item basically takes a single `tensor` of one element and it just returns
否则 `PyTorch` 的 `.item` 基本上是接收一个单元素的张量，然后它只是返回

pytorch

`PyTorch` 主流的深度学习框架，`micrograd` 的接口设计大致仿照它。

代码与工程

tensor

张量 多维数组，是深度学习框架中组织标量并进行高效并行运算的基本数据结构。

神经网络基础

- ▶ 1:42:34 that element stripping out the `tensor`. So let me just run this and hopefully we are going to get
那个元素，剥离掉张量。所以让我运行一下，希望我们将会得到

tensor

张量 多维数组，是深度学习框架中组织标量并进行高效并行运算的基本数据结构。

神经网络基础

- ▶ 1:42:40 this is going to print the `forward pass`, which is 0.707. And this will be the `gradients`,
这会打印前向传播的结果，也就是 0.707。然后这会是梯度，

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:42:46 which hopefully are 0.50, negative 0.5 and 1. So if we just run this,
希望是 0.50、负 0.5 和 1。所以如果我们运行这个，

- ▶ 1:42:53 there we go, 0.7, so the `forward pass` agrees and then 0.50, negative 0.5 and 1. So `PyTorch`
出来了，0.7，所以前向传播结果一致，然后 0.50、负 0.5 和 1。所以 `PyTorch`

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

pytorch

`PyTorch` 主流的深度学习框架，`micrograd` 的接口设计大致仿照它。

代码与工程

- ▶ 1:43:01 agrees with us. And just to show you here basically, oh, here's a `tensor` with a single element. 和我们一致。然后给你们展示一下，基本上，哦，这里有一个单元素的张量。

tensor

张量 多维数组，是深度学习框架中组织标量并进行高效并行运算的基本数据结构。

神经网络基础

- ▶ 1:43:08 And it's a double. And we can call that item on it to just get the single number out. 而且它是一个 double 类型。我们可以在它上面调用 item 来取出那个单独的数字。

- ▶ 1:43:14 So that's what item does. And oh, is a `tensor` object, like I mentioned, 所以这就是 item 所做的。哦，它是一个张量对象，就像我提到的，

tensor

张量 多维数组，是深度学习框架中组织标量并进行高效并行运算的基本数据结构。

神经网络基础

- ▶ 1:43:18 and it's got a `backward` function just like we've implemented. And then all of these also have 而且它有一个 backward 函数，就像我们实现的那样。然后所有这些也都有

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 1:43:23 a dot `grad`. So like x2, for example, and the `grad`, and it's a `tensor`. And we can pop out the individual

一个 `.grad`。比如 x2，然后 `grad`，它是一个张量。我们可以弹出单个的

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

tensor

张量 多维数组，是深度学习框架中组织标量并进行高效并行运算的基本数据结构。

神经网络基础

- ▶ 1:43:28 number with dot item. So basically, tensors can do what we did in `micrograd` as a special case 数字，用 `.item`。所以基本上，张量可以做我们在 `micrograd` 中所做的，作为一个特例

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 1:43:36 when your tensors are all single element tensors. But the big deal with `PyTorch` is that 当你的张量都是单元素张量时。但 `PyTorch` 的重大意义在于

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

- ▶ 1:43:42 everything is significantly more efficient because we are working with these `tensor` 一切都显著更高效，因为我们在处理这些张量

tensor

张量 多维数组，是深度学习框架中组织标量并进行高效并行运算的基本数据结构。

神经网络基础

- ▶ 1:43:46 objects. And we can do lots of operations in parallel on all of these tensors. 对象。我们可以在所有这些张量上并行地做很多操作。

- ▶ 1:43:51 But otherwise, what we've built very much agrees with the API of `PyTorch`. 但除此之外，我们构建出来的东西和 `PyTorch` 的 API 非常一致。

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

- ▶ 1:43:55 Okay, so now that we have some machinery to build out pretty complicated mathematical 好，现在我们已经有了有一些机制，可以构建相当复杂的数学

- ▶ 1:43:58 expressions, we can also start building out `neural nets`. And as I mentioned, `neural nets` are 表达式，我们也可以开始搭建神经网络了。而且正如我提到的，神经网络

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 1:44:03 just a specific class of mathematical expressions. So we're going to start building out a neural 只是数学表达式的一个特定类别。所以我们将开始搭建一个神经

▶ 1:44:08 net piece by piece, and eventually we'll build out a two **layer** **multi layer perceptron**, 网络, 一步一步地搭建, 最终我们会构建出一个两层的多层感知机,

layer

层 一组独立计算的神经元, 通常将同一维度的输入映射到多个输出。

神经网络基础

multi layer perceptron

多层感知机 由多个线性层与非线性激活函数堆叠而成的前馈神经网络, 也常称为 MLP。

神经网络基础

▶ 1:44:13 as it's called. And I'll show you exactly what that means. Let's start with a single individual **neuron**. 也就是所谓的 MLP。我会向你们展示这到底是什么意思。让我们从单个神经元开始。

neuron

神经元 神经网络的基本单元, 对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

实现神经元层与多层感知机 01:44

▶ 1:44:17 We've implemented one here. But here I'm going to implement one that also subscribes to the **PyTorch** 我们这里已经实现了一个。但这里我要实现一个同样遵循 PyTorch

pytorch

PyTorch 主流的深度学习框架, micrograd 的接口设计大致仿照它。

代码与工程

▶ 1:44:23 API and how it designs its **neural network** modules. So just like we saw that we can like match the API 以及它如何设计神经网络模块。就像我们看到的, 我们可以匹配

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

▶ 1:44:29 API of PyTorch on the autograd side, we're going to try to do that on the neural network
PyTorch 在自动求导侧的 API，我们也要尝试在神经网络

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

autograd

自动求导 自动计算梯度的机制，会记录前向计算图并支持反向传播。

反向传播与梯度

▶ 1:44:34 modules. So here's class Neuron . And just for the sake of efficiency, I'm going to copy paste
模块方面做到这一点。所以这里是 class Neuron。为了效率起见，我要复制粘贴

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

▶ 1:44:41 some sections that are relatively straightforward. So the constructor will take number of inputs to this
一些相对直接的部分。所以构造函数会接收这个

constructor

构造函数 创建对象时自动调用的初始化方法，用于设置实例属性。

代码与工程

▶ 1:44:48 neuron , which is how many inputs come to a neuron . So this one, for example, has three inputs.
神经元的输入数量，也就是有多少个输入进入一个神经元。比如这个例子，有三个输入。

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

▶ 1:44:55 And then it's going to create a weight . There is some random number between negative one and one
然后它会创建一个权重。有一个介于 -1 和 1 之间的随机数

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

▶ 1:44:59 for every one of those inputs, and a bias that controls the overall trigger happiness of this
对应每一个输入，以及一个偏置，它控制这个神经元整体的触发倾向

bias

偏置 神经元中与输入无关的可学习常数项，用来平移激活函数的输出。

神经网络基础

- ▶ 1:45:04 **neuron**. And then we're going to implement a `def __call__` of self and x, 神经元。然后我们要实现一个 `def __call__`, 接收 self 和 x,

neuron

神经元 神经网络的基本单元, 对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:45:12 some input x. And really what we do here is w times x plus b, or w times x here is a dot 某个输入 x。实际上我们在这里做的是 w 乘以 x 加 b, 或者说这里的 w 乘以 x 是一个点

- ▶ 1:45:19 product specifically. Now if you haven't seen `__call__`, let me just return 0.0 here for now. 积, 具体来说是点积。现在如果你没见过 `__call__`, 我先在这里直接返回 0.0。

- ▶ 1:45:26 The way this works now is we can have an x which is say like 2.0 3.0, 现在它的工作方式是, 我们可以有一个 x, 比如说是 2.0、3.0,

- ▶ 1:45:30 then we can initialize a **neuron** that is two dimensional, because these are two numbers. 然后我们可以初始化一个二维的神经元, 因为这是两个数字。

neuron

神经元 神经网络的基本单元, 对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:45:35 And then we can feed those two numbers into that **neuron** to get an output. And so when you use this

然后我们可以把这两个数字输入到那个神经元里, 得到一个输出。所以当你使用这个

neuron

神经元 神经网络的基本单元, 对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:45:40 note that ``__call__(self, x)``, Python will use call. So currently, call just returns 0.0. Now we'd like to actually

注意 ``__call__(self, x)`` 这里, Python 会调用 call。目前 call 只是返回 0.0。现在我们要真正地

- ▶ 1:45:51 do the **forward pass** of this **neuron** instead. So we're going to do here first is we need to 去做这个神经元的前向传播。所以我们在这里首先要做的是, 我们需要

forward pass

前向传播 按计算图从输入计算到输出, 得到预测值或损失。

神经网络基础

neuron

神经元 神经网络的基本单元, 对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:45:57 basically multiply all of the elements of w with all of the elements of x pairwise. We 把 w 的所有元素和 x 的所有元素逐对相乘。我们
- ▶ 1:46:02 need to multiply them. So the first thing we're going to do is we're going to zip up 需要把它们相乘。所以我们要做的第一件事，就是把
- ▶ 1:46:07 $self.w$ and x . And in Python `zip` takes two iterators, and it creates a new iterator that iterates over $self.w$ 和 x 打包起来。在 Python 里，`zip` 接收两个迭代器，并创建一个新的迭代器，它遍历
- ▶ 1:46:15 the tuples of their corresponding entries. So for example, just to show you we can print 它们对应项组成的元组。举个例子演示一下，我们可以打印
- ▶ 1:46:20 this list and still return 0.0 here. So we see that these w 's are paired up with the x 's, 这个列表，这里仍然返回 0.0。可以看到，这些 w 和 x 配成了对，
- ▶ 1:46:37 w with x . And now what we want to do is for w_i, x_i in, we want to multiply w_i times x_i , w 和 x 配对。现在我们想做的是，对于每一对 w_i, x_i ，我们要把 w_i 乘以 x_i ，
- ▶ 1:46:54 and then we want to sum all of that together to come up with an **activation** and add also $self.b$ on 然后把它们全部加起来，得到一个激活值，再加上 $self.b$ ，

activation

激活值 神经元对输入加权求和并加上偏置后得到的数值。

神经网络基础

- ▶ 1:47:01 top. So that's the raw **activation**. And then of course we need to pass that through a nonlinearity. 叠加在上面。这就是原始的激活值。当然，接下来我们要把它送进一个非线性函数。

activation

激活值 神经元对输入加权求和并加上偏置后得到的数值。

神经网络基础

- ▶ 1:47:06 So what we're going to be returning is `act.tanh()`. And here's out. So now we see that we are getting 所以我们要返回的是 `act.tanh()`。这就是 out。现在可以看到，我们得到了

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 1:47:14 some outputs. And we get a different output from **neuron** each time because we are initializing 一些输出。每次从神经元得到的输出都不一样，因为我们初始化了

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:47:18 different **weights** and biases. And then to be a bit more efficient here actually, sum by the way 不同的权重和偏置。然后为了更高效一点——其实，顺便说一下，sum

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

- ▶ 1:47:24 takes a second optional parameter, which is the start. And by default, the start is 0. So these 还有第二个可选参数，也就是 start。默认情况下 start 是 0。所以这些

- ▶ 1:47:32 elements of this sum will be added on top of 0 to begin with. But actually we can just start with 求和元素一开始会加在 0 上面。但其实我们可以直接从

- ▶ 1:47:36 self.b. And then we just have an **expression** like this. And then the generator **expression** here must self.b 开始。然后我们就得到了这样一个表达式。这里的生成器表达式必须

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 1:47:47 be parenthesized in Python. There we go. Yep. So now we can forward a single **neuron**. Next up, 在 Python 里加上括号。好了。对。现在我们可以对单个神经元做前向传播了。接下来，

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:47:56 we're going to define a **layer** of neurons. So here we have a schematic for a **MLP**. So we see 我们要定义一个神经元层。这里有一个 MLP 的示意图。可以看到

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

mlp

多层感知机 由多个全连接层堆叠而成的前馈神经网络。

神经网络基础

- ▶ 1:48:03 that these MLPs, each **layer**, this is one **layer**, has actually a number of neurons, and they're 这些 MLP，每一层——这是一层——实际上有若干个神经元，它们

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

- ▶ 1:48:08 not connected to each other, but all of them are fully connected to the input. So what is a **layer** of 彼此之间并不相连，但它们全都与输入全连接。那么，一层究竟是

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

- ▶ 1:48:12 neurons? It's just a set of neurons evaluated independently. So in the interest of time, 神经元？它只是一组独立求值的神经元。所以为了节省时间，

- ▶ 1:48:19 I'm going to do something fairly straightforward here. It's literally a **layer** is just a list 我在这里就做一个相当直接的东西。实际上，一层就只是一个列表，

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

- ▶ 1:48:27 of neurons. And then how many neurons do we have? We take that as an input argument here. 里面是神经元。那我们要多少个神经元呢？我们把它作为这里的一个输入参数。

- ▶ 1:48:32 How many neurons do you want in your **layer**? Number of outputs in this **layer**. 你希望这一层有多少个神经元？也就是这一层的输出数量。

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

- ▶ 1:48:36 And so we just initialize completely independent neurons with this given **dimensionality**. 于是我们就用给定的维度初始化完全独立的神经元。

dimensionality

维度 输入或输出的特征数量，决定权重向量的长度。

神经网络基础

- ▶ 1:48:41 And when we call on it, we just independently evaluate them. So now instead of a **neuron**, 当我们调用它时，我们只是独立地对它们求值。所以现在不再是一个神经元，

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:48:47 we can make a **layer** of neurons. They are two dimensional neurons and let's have three of them.
我们可以构造一层神经元。它们接收二维输入，我们放三个。

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

- ▶ 1:48:52 And now we see that we have three independent evaluations of three different neurons.
现在我们可以看到，我们对三个不同的神经元做了三次独立的求值。

- ▶ 1:48:58 Okay, finally, let's complete this picture and define an entire **multilayer perceptron**,
好，最后，让我们补全这个图景，定义完整的多层感知机，

multilayer perceptron

多层感知机 由若干全连接神经元层依次堆叠而成的前馈神经网络。

神经网络基础

- ▶ 1:49:03 or **MLP**. And as we can see here in an **MLP**, these layers just feed into each other sequentially.
也就是 MLP。如我们这里所见，在 MLP 中，这些层只是依次向前馈送。

mlp

多层感知机 由多个全连接层堆叠而成的前馈神经网络。

神经网络基础

- ▶ 1:49:09 So let's come here and I'm just going to copy the code here in the interest of time.
那我们到这里来，为了节省时间，我直接把代码复制过来。

- ▶ 1:49:14 So an **MLP** is very similar. We're taking the number of inputs as before. But now instead of
所以 MLP 非常类似。我们和之前一样接收输入数量。但现在不是

mlp

多层感知机 由多个全连接层堆叠而成的前馈神经网络。

神经网络基础

- ▶ 1:49:20 taking a single `n_out`, which is number of neurons in a single **layer**, we're going to take a list
接收单个 `n_out`，也就是单层中的神经元数量，而是接收一个列表

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

- ▶ 1:49:25 of `n_outs`. And this list defines the sizes of all the layers that we want in our `MLP`.
`n_outs`。这个列表定义了 MLP 中每一层的大小。

mlp

多层感知机 由多个全连接层堆叠而成的前馈神经网络。

神经网络基础

- ▶ 1:49:30 So here we just put them all together and then iterate over consecutive pairs of these sizes
这里我们把它们放在一起，然后遍历这些大小的相邻对，

- ▶ 1:49:35 and create a `Layer` object for them. And then in the `__call__` function, we are just calling them
为它们创建 Layer 对象。然后在 `__call__` 函数中，我们只是依次调用它们

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

- ▶ 1:49:39 sequentially. So that's an `MLP` really. And let's actually re-implement this picture. So we want
顺序调用。所以 MLP 其实就是这么回事。我们来实际重新实现这个图景。我们想要

mlp

多层感知机 由多个全连接层堆叠而成的前馈神经网络。

神经网络基础

- ▶ 1:49:45 three input neurons and then two layers of four and an output unit. So we want
三个输入神经元，然后两个大小为四的层，以及一个输出单元。所以我们需要

- ▶ 1:49:52 three dimensional input. Say this is an example input. We want three inputs into
三维输入。假设这是一个示例输入。我们需要三个输入进入

- ▶ 1:49:57 two layers of four and one output. And this of course is an `MLP`. And there we go. That's a forward
两个大小为四的层，并得到一个输出。这当然就是一个 MLP。这样就成了。这是一次前向

mlp

多层感知机 由多个全连接层堆叠而成的前馈神经网络。

神经网络基础

- ▶ 1:50:05 pass of an `MLP`. To make this a little bit nicer, you see how we have just a single element, but it's
MLP 的一次前向传播。为了让它更友好一点，你可以看到我们这里只有一个元素，但它

mlp

多层感知机 由多个全连接层堆叠而成的前馈神经网络。

神经网络基础

- ▶ 1:50:10 wrapped in a list because `layer` always returns lists. So for convenience, return `outs[0]` if 被包在一个列表里，因为 `layer` 总是返回列表。所以为了方便，如果

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

- ▶ 1:50:17 `len(outs)` is exactly a single element, else return full list. And this will allow us to just get `len(outs)` 恰好只有一个元素，就返回 `outs[0]`；否则返回完整列表。这样我们就能

- ▶ 1:50:23 a single `value` out at the last `layer` that only has a single `neuron`. And finally, we should be able to 在最后一层只有一个神经元时，直接得到一个单值输出。最后，我们应该能够

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

- ▶ 1:50:29 `draw_dot` of `n` of `x`. And as you might imagine, these expressions are now getting relatively involved. 对 `n(x)` 调用 `draw_dot`。可以想象，这些表达式现在已经相当复杂了。

draw_dot

`draw_dot` micrograd 中把 Value 计算图导出为 Graphviz 图形的辅助函数。

代码与工程

- ▶ 1:50:38 So this is an entire `MLP` that we're defining now all the way until a single output. Okay. 所以这就是我们现在定义的整个 MLP，一直到单个输出。好。

mlp

多层感知机 由多个全连接层堆叠而成的前馈神经网络。

神经网络基础

- ▶ 1:50:49 And so obviously, you would never `differentiate` on pen and paper these expressions, but with 显然，你绝不会在纸上手动对这些表达式求导，但有了

differentiate

求导 计算函数导数的过程，在反向传播中用于得到梯度。

反向传播与梯度

▶ 1:50:53 **micrograd**, we will be able to **backpropagate** all the way through this and **backpropagate** **micrograd**, 我们就能一路反向传播, 并反向传播

micrograd

micrograd 一个轻量级自动微分库, 用 Python 从零实现标量计算图与反向传播。

代码与工程

backpropagate

反向传播 利用链式法则从输出端向输入端逐层计算梯度, 从而高效地训练神经网络。

反向传播与梯度

▶ 1:50:59 into these **weights** of all these neurons. So let's see how that works. Okay, so let's create ourselves 到所有这些神经元的权重中。我们来看看这是怎么做到的。好, 那我们创建

weight

权重 神经网络中与输入相乘的可学习参数, 决定输入对神经元输出的影响。

神经网络基础

▶ 1:51:05 a very simple example data set here. So this data set has four examples. And so we have four possible 一个非常简单的示例数据集。这个数据集有四个样本。于是我们有四个可能的

▶ 1:51:13 inputs into the **neural net**. And we have four desired **targets**. So we'd like the **neural net** to 输入进入神经网络。并且我们有四个期望的目标值。我们希望神经网络

neural net

神经网络 由多层可学习参数组成的函数逼近器, 用于从输入预测输出。

神经网络基础

targets

目标值 训练样本中希望模型输出接近的真实标签或期望值。

训练与优化

▶ 1:51:18 assign or output 1.0 when it's fed this example, negative one when it's fed these examples, and 在喂入这个样本时输出 1.0, 在喂入这些样本时输出负一, 并且

构建训练数据与损失函数 01:51

- ▶ 1:51:26 one when it's fed this example. So it's a very simple **binary classifier** **neural net** basically 在喂入这个样本时输出一。所以基本上它就是一个非常简单的二分类神经网络

binary classifier

二分类器 将输入分为两个类别的模型。

神经网络基础

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 1:51:30 that we would like here. Now let's think what the **neural net** currently thinks about these four 这就是我们这里想要的。现在看看神经网络当前对这四个

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 1:51:34 examples, we can just get their predictions. Basically, we can just call n of x for x in 样本，我们可以直接得到它们的预测。基本上，我们可以直接对 x 在

- ▶ 1:51:40 x s. And then we can print. So these are the outputs of the **neural net** on those four examples. x s 中调用 $n(x)$ 。然后我们可以打印。这些就是神经网络在这四个样本上的输出。

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 1:51:48 So the first one is point nine one, but we'd like it to be one. So we should push this one higher, 第一个是 0.91，但我们希望它是一。所以我们应该把这个值推高，

- ▶ 1:51:55 this one we want to be higher. This one says point eight eight, and we want this to be negative 这个我们希望它更高。这个显示的是 0.88，而我们希望它是负

- ▶ 1:52:01 one. This is point eight, we want it to be negative one. And this one is point eight, we 一。这个是 0.8，我们希望它是负一。这个是 0.8，我们

▶ 1:52:06 want it to be one. So how do we make the **neural net** and how do we tune the **weights** to better predict

希望它是一。那么我们如何让神经网络，如何调整权重来更好地预测

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

▶ 1:52:14 the desired **targets**? And the trick used in deep learning to achieve this is to calculate a single
想要的目标？而深度学习中用来实现这一点的技巧，就是计算一个单一的

targets

目标值 训练样本中希望模型输出接近的真实标签或期望值。

训练与优化

▶ 1:52:21 number that somehow measures the total performance of your **neural net**. And we call the single
number

数字，它能以某种方式衡量你神经网络的总性能。我们把这个单一数字

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

▶ 1:52:26 the **loss**. So the **loss** first is a single number that we're going to define that basically measures
称为损失。所以损失首先是一个单一数字，我们将定义它，它基本上衡量

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

▶ 1:52:34 how well the **neural net** is performing. Right now we have the intuitive sense that it's not
神经网络表现得多好。现在我们有直观感觉，它并不

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 1:52:37 performing very well, because we're not very much close to this. So the **loss** will be high, 表现得很好, 因为我们离这个还不太近。所以损失会很高,

loss

损失 衡量模型预测与目标差距的单一数值, 训练时通常希望它越小越好。

训练与优化

- ▶ 1:52:42 and we'll want to **minimize the loss**. So in particular, in this case, what we're going to do 而我们会想要最小化损失。所以具体来说, 在这种情况下, 我们要做的是

minimize the loss

最小化损失 通过调整模型参数使损失数值尽可能小。

训练与优化

- ▶ 1:52:46 is we're going to implement the **mean squared error loss**. So what this is doing is we're going 实现均方误差损失。所以它做的事情是, 我们要

mean squared error

均方误差 把预测值与真实值之差平方后取平均得到的常用损失。

训练与优化

loss

损失 衡量模型预测与目标差距的单一数值, 训练时通常希望它越小越好。

训练与优化

- ▶ 1:52:51 to basically iterate for y **ground truth** and y output in zip of y's and y_pred. So we're 基本上遍历 zip(ys, y_pred) 中的 y ground truth 和 y output。所以我们

ground truth

真实标签 数据中作为正确答案的目标值, 用来监督模型训练。

训练与优化

- ▶ 1:53:01 going to pair up the ground truths with the predictions and the zip iterates over the tuples 要把真实标签和预测配对, 而 zip 会遍历这些

- ▶ 1:53:07 of them. And for each y **ground truth** and y output, we're going to subtract them and square 元组。对于每个 y ground truth 和 y output, 我们会把它们相减并平方

ground truth

真实标签 数据中作为正确答案的目标值, 用来监督模型训练。

训练与优化

- ▶ 1:53:18 So let's first see what these losses are. These are individual **loss** components.
所以先看看这些损失是什么。这些是个体损失分量。

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 1:53:22 And so basically for each one of the four, we are taking the prediction and the **ground truth**,
所以基本上对这四个中的每一个，我们取预测值和真实值，

ground truth

真实标签 数据中作为正确答案的目标值，用来监督模型训练。

训练与优化

- ▶ 1:53:29 we are subtracting them and squaring them. So because this one is so close to its target,
把它们相减并平方。因为这一个非常接近它的目标，

- ▶ 1:53:36 point nine one is almost one subtracting them gives a very small number. So here we would get
0.91 几乎就是 1，相减会得到非常小的数。所以这里我们会得到

- ▶ 1:53:42 like a negative point one and then squaring it just makes sure that regardless of whether we are
比如负 0.1，然后对它平方就确保了不管我们是

- ▶ 1:53:49 more negative or more positive, we always get a positive number. Instead of squaring, we should
更负还是更正，我们总会得到一个正数。除了平方，我们应该

- ▶ 1:53:55 oh, we could also take for example the **absolute value** we need to discard the sign. And so you see
哦，我们也可以取绝对值，我们需要丢弃符号。于是你会看到

absolute value

绝对值 去掉数值符号后得到的非负大小。

数学与微积分

- ▶ 1:54:00 that the **expression** is arranged so that you only get zero exactly when y_{out} is equal to y_{ground}
这个表达式被安排成只有当 y_{out} 等于 y_{ground}

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 1:54:05 truth. When those two are equal, so your prediction is exactly the target, you are going to get
truth。当这两个相等时，也就是你的预测正好是目标时，你会得到

- ▶ 1:54:10 zero. And if your prediction is not the target, you are going to get some other number.
零。而如果你的预测不是目标，你就会得到某个其他的数。

1:54:15 So here, for example, we are way off. And so that's why the **loss** is quite high.
所以这里，举例来说，我们偏差很大。所以这就是为什么损失相当高。

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

1:54:19 And the more off we are, the greater the **loss** will be. So we don't want high **loss**, we want low **loss**.
我们偏差越大，损失就会越大。所以我们不想要高损失，我们想要低损失。

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

1:54:27 And so the final **loss** here will be just the sum of all of these numbers. So you see that this
所以这里最终的损失就是所有这些数字的总和。所以看到这个

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

1:54:34 should be zero roughly plus zero roughly, but plus seven. So **loss** should be about seven here.
大概应该是零加零，再加上七。所以这里损失大约应该是七。

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

1:54:44 And now we want to **minimize the loss**. We want the **loss** to be low, because if **loss** is low,
现在我们要最小化损失。我们希望损失很低，因为如果损失很低，

minimize the loss

最小化损失 通过调整模型参数使损失数值尽可能小。

训练与优化

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

1:54:51 then every one of the predictions is equal to its target. So the **loss**, the lowest it can be is zero,
那么每一个预测值都等于它的目标值。所以损失最小可以是零，

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 1:54:58 and the greater it is, the worse off the `neural net` is predicting.
而损失越大，神经网络预测得就越差。

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 1:55:04 So now, of course, if we do `loss.backward()`,
那么现在，当然，如果我们调用 `loss.backward()`,

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

梯度下降更新参数 01:55

- ▶ 1:55:07 something magical happened when I hit enter. And the magical thing, of course,
当我按下回车时，神奇的事情发生了。当然，这个神奇的事情
- ▶ 1:55:12 that happened is that we can look at `n.layers[0].neurons[0]`, at say, like the first
就是我们可以查看 `n.layers[0].neurons[0]`，比如说，像第一
- ▶ 1:55:17 `layer`, the neurons at zero, because remember that `MLP` has the layers, which is a list. And each
`layer`
层中索引为零的神经元，因为记住 MLP 有 layers，它是一个列表。而每一层

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

mlp

多层感知机 由多个全连接层堆叠而成的前馈神经网络。

神经网络基础

1:55:27 has neurons, which is a list. And that gives us an individual **neuron**. And then it's got some
有 neurons，它是一个列表。这样我们就得到一个单独的神经元。然后它有一些

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

1:55:31 **weights**. And so we can, for example, look at $w[0]$. Oops, it's not called **weights**.
权重。所以我们可以，举例来说，查看 $w[0]$ 。哦，它不叫 weights。

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

1:55:41 It's called w . And that's a **value**. But now this **value** also has a **grad** because of the **backward** pass.
它叫 w 。这是一个 Value。但现在这个 Value 也有一个 grad，因为反向传播。

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

1:55:50 And so we see that because this **gradient** here on this particular **weight** of this particular **neuron**
所以我们看到，因为这个特定神经元的这个特定权重上的梯度

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

1:55:55 of this particular **layer** is negative, we see that its influence on the **loss** is also negative.
在这个特定层上是负的，我们看到它对损失的影响也是负的。

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 1:56:01 So slightly increasing this particular `weight` of this `neuron` of this `layer` would make the `loss` go
所以稍微增大这个层中这个神经元的这个特定权重，会让损失

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:56:07 down. And we actually have this information for every single one of our neurons and all of their
下降。而且我们实际上拥有每一个神经元及其所有

- ▶ 1:56:12 `parameters`. Actually, it's worth looking at also the `draw_dot(loss)`, by the way. So previously,
参数的信息。实际上，顺便也值得看看 `draw_dot(loss)`。所以之前，

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

draw_dot

`draw_dot` micrograd 中把 Value 计算图导出为 Graphviz 图形的辅助函数。

代码与工程

- ▶ 1:56:18 we looked at the `draw_dot` of a single `neuron` `forward pass`. And that was already a
我们看过单个神经元前向传播的 `draw_dot`。那已经是一个

draw_dot

`draw_dot` micrograd 中把 Value 计算图导出为 Graphviz 图形的辅助函数。

代码与工程

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:56:22 large `expression`. But what is this `expression`? We actually forwarded every one of those four
大型表达式。但这个表达式是什么？我们实际上对这四个样本中的每一个都做了前向传播。

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

▶ 1:56:28 examples. And then we have the **loss** on top of them with the **mean squared error**. And so this 样本。然后我们在它们之上用均方误差得到了损失。所以这

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

mean squared error

均方误差 把预测值与真实值之差平方后取平均得到的常用损失。

训练与优化

▶ 1:56:33 is a really massive **graph**. Because this **graph** that we've built up now, oh my gosh, this **graph** that 真是一个巨大的计算图。因为我们现在构建出的这个图，天哪，这个图

graph

计算图 由变量和运算节点组成的有向无环图，记录计算过程并支持反向传播。

神经网络基础

▶ 1:56:41 we've built up now, which is kind of excessive, it's excessive because it has four forward passes 我们现在构建出的这个图，有点过于庞大了，它庞大是因为它包含了四次前向传播

▶ 1:56:46 of a **neural net** for every one of the examples. And then it has the **loss** on top. And it ends 对每一个样本都要做一次神经网络的前向传播。然后上面还有损失。它最后

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

▶ 1:56:51 with the **value** of the **loss**, which was 7.12. And this **loss** will now back **propagate** through all the 以损失值结束，也就是 7.12。而这个损失现在会反向传播穿过所有

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 1:56:57 forward passes all the way through just every single intermediate `value` of the `neural net`.
前向传播，一路穿过神经网络中每一个中间值。

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 1:57:03 All the way back to, of course, the `parameters` of the `weights`, which are the input. So these
当然会一路回到权重的参数，它们就是输入。所以这些

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

- ▶ 1:57:08 `weight` `parameters` here are inputs to this `neural net`. And these numbers here, these scalars,
权重参数是这个神经网络的输入。而这里的这些数，这些标量，

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

- ▶ 1:57:14 are inputs to the `neural net`. So if we went around here, we will probably find some of these
也是神经网络的输入。所以如果我们在这里四处看看，我们大概会找到其中一些

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 1:57:21 examples, this 1.0, potentially maybe this 1.0, or you know, some of the others. And you'll
样本，这个 1.0，也可能是这个 1.0，或者别的某些。然后你会

▶ 1:57:26 see that they all have **gradients** as well. The thing is, these **gradients** on the input data 看到它们也都有梯度。问题是，输入数据上的这些梯度

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

▶ 1:57:31 are not that useful to us. And that's because the input data seems to be not changeable. It's 对我们来说并不那么有用。那是因为输入数据似乎是不可改变的。它是

▶ 1:57:38 a given to the problem. And so it's a fixed input, we're not going to be changing it or messing 问题给定的条件。所以它是固定输入，我们不会去改变它或者摆弄

▶ 1:57:42 with it, even though we do have **gradients** for it. But some of these **gradients** here 它，即使我们确实有它的梯度。但这里有些梯度

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

▶ 1:57:49 will be for the **neural network parameters**, the Ws and the Bs. And those, of course, 会是神经网络参数的梯度，也就是 W 和 B。而那些，当然，

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

▶ 1:57:53 we want to change. Okay, so now we're going to want some convenience codes to gather up 是我们想要改变的。好，所以现在我们想要一些方便的代码来收集

▶ 1:57:59 all of the **parameters** of the **neural net** so that we can operate on all of them simultaneously. 神经网络的所有参数，这样我们就能同时操作它们。

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 1:58:04 And every one of them, we will **nudge** a tiny amount based on the **gradient** information.
对于它们中的每一个，我们会根据梯度信息轻轻地推动一点点。

nudge

微调 对输入做极小扰动，用来看函数输出如何变化。

数学与微积分

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 1:58:10 So let's collect the **parameters** of the **neural net** all in one array. So let's create a
所以让我们把神经网络的参数都收集到一个数组里。让我们创建一个

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 1:58:15 **parameters** of self that just returns a list, which is a list concatenated with a list of
self 的 parameters 只是返回一个列表，这个列表与一个列表拼接，后面是

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

- ▶ 1:58:25 self.b. So this will just return a list, list plus list just gives you a list.
self.b. 所以这只会返回一个列表，列表加列表得到的就是一个列表。

- ▶ 1:58:32 So that's **parameters** of **neuron**. And I'm calling it this way because also **PyTorch** has a
这就是 neuron 的 parameters。我这么称呼它是因为 PyTorch 也有一个

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:58:37 `parameters` on every single `nn.Module`. And it does exactly what we're doing here. It just 在每个 `nn.Module` 上的 `parameters`。它做的正是我们这里做的事。它只是

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

nn.module

PyTorch 模块基类 PyTorch 中所有神经网络模块的基类，提供 `parameters` 等方法。

代码与工程

- ▶ 1:58:42 returns the `parameter tensors` for us, it's the parameter scalars. Now `layer` is also a module. 为我们返回参数张量，对我们来说就是参数标量。现在 `layer` 也是一个 `module`。

parameter tensors

参数张量 模型中参与训练、需要计算梯度的张量。

神经网络基础

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

- ▶ 1:58:50 So it will have `parameters`. So and basically what we want to do here is something like this, 所以它会有 `parameters`。基本上我们在这里要做的就是类似这样，

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

- ▶ 1:58:57 like, `params` is here and then for `neuron` in `self.neurons`, we want to get `neuron parameters`. 比如，`params` 在这里，然后对于 `self.neurons` 中的每个 `neuron`，我们要获取 `neuron` 的 `parameters`。

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

- ▶ 1:59:10 And we want to `params.extend`. So these are the `parameters` of this `neuron`. And then we 然后我们想执行 `params.extend`。这些就是这个 `neuron` 的参数。然后我们

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 1:59:17 want to put them on top of `params`. So `params.extend(p)`. And then we want to return `params`. 想把它们放到 `params` 上面。所以是 `params.extend(p)`。然后我们想返回 `params`。

- ▶ 1:59:25 So this is way too much code. So actually, there's a way to simplify this, which is return 所以这代码太多了。其实有一种简化方式，就是 `return`

- ▶ 1:59:33 `[p for neuron in self.neurons for p in neuron.parameters()]`
`[p for neuron in self.neurons for p in neuron.parameters()]`

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

- ▶ 1:59:44 So it's a single `list comprehension` in Python, you can sort of nest them like this, and you can 所以这是 Python 中的一个列表推导式，你可以像这样嵌套，然后你就可以

list comprehension

列表推导式 Python 中从可迭代对象生成列表的简洁语法，可嵌套循环。

代码与工程

- ▶ 1:59:51 then create the desired array. So these are identical. We can take this out. 创建出想要的数组。所以这两者是等价的。我们可以把它删掉。

- ▶ 2:00:00 And then let's do the same here. `def parameters(self)` and `return [p for layer in self`
然后我们在这里做同样的事。`def parameters(self)`，然后 `return [p for layer in self`

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

- ▶ 2:00:12 `.layers for p in layer.parameters()`. And that should be good. Now let me pop out this. So
`.layers for p in layer.parameters()`。这样应该就可以了。现在让我把这个弹出来。所以

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

- ▶ 2:00:26 we don't re initialize our network, because we need to re initialize our okay, so unfortunately,
我们不会重新初始化我们的网络，因为我们需要重新初始化我们的.....好吧，所以不幸的是，

- ▶ 2:00:36 we will have to probably initialize network because we just added functionality. Because this
我们可能不得不初始化网络，因为我们刚刚添加了功能。因为这个

- ▶ 2:00:42 class, of course, we want to get all the new `parameters`. But that's not going to work
类，当然，我们想获取所有新的 `parameters`。但这行不通

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

- ▶ 2:00:46 because this is the old class. Okay. So unfortunately, we do have to re initialize the network, which will
因为这是旧的类。好吧。所以不幸的是，我们确实必须重新初始化网络，这会

- ▶ 2:00:53 change some of the numbers. But let me do that so that we pick up the new API, we can now do n
改变一些数字。但让我这么做，这样我们就能使用新的 API，我们现在可以做 n

2:00:58 `n.parameters()`. And these are all the `weights` and biases inside the entire `neural net`. So in total, `n.parameters()`. 这些就是整个神经网络内部所有的权重和偏置。所以总共，

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

2:01:07 this `MLP` has 41 `parameters`. And now we'll be able to change them.
这个 `MLP` 有 41 个参数。现在我们可以改变它们了。

mlp

多层感知机 由多个全连接层堆叠而成的前馈神经网络。

神经网络基础

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

2:01:14 If we recalculate the `loss` here, we see that unfortunately, we have slightly different predictions and slightly different `loss`. But that's okay. Okay, so we see that this
如果我们重新计算这里的损失，会看到不幸的是，我们得到了稍微不同的预测和稍微不同的损失。但没关系。好，我们看到这个

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

2:01:22 `neuron`'s `gradient` is slightly negative. We can also look at its data right now,
神经元的梯度稍微为负。我们也可以现在看看它的数据，

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

▶ 2:01:30 which is point eight five. So this is the current `value` of this `neuron`. And this is its `gradient`
也就是 0.85。所以这是这个神经元当前的值。而这是它的梯度

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

▶ 2:01:36 on the `loss`. So what we want to do now is we want to iterate for every `p` in `n.parameters()`.
关于损失的梯度。所以我们现在要做的是，对 `n.parameters()` 中的每一个 `p` 进行迭代。

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

▶ 2:01:41 So for all the 41 `parameters` of this `neural net`, we actually want to change `p.data` slightly according to the `gradient` information. Okay, so dot dot dot to do here. But this will
所以对于这个神经网络的 41 个参数，我们实际上想根据梯度信息稍微改变 `p.data`。好，所以这里要做的是点点点。但
会

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

▶ 2:01:49 be basically a tiny update in this `gradient` descent scheme. And `gradient` descent, we are
基本上是这个梯度下降方案中的一次微小更新。而在梯度下降中，我们

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 2:01:55 thinking of the **gradient** as a vector pointing in the direction of increased **loss**. And so in **gradient** 把梯度看作指向损失增加方向的向量。所以在梯度

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 2:02:02 descent, we are modifying p.data by a small **step size** in the direction of the **gradient**. So the step 下降中，我们沿着梯度的方向用一个小的步长来修改 p.data。所以这个步

step size

步长 数值梯度估计中用于扰动输入的小量，通常记为 h 。

数学与微积分

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 2:02:09 size as an example could be like a very small number of 0.01 is the **step size** times p. **grad**. 长，举个例子，可以是一个非常小的数，比如 0.01 就是步长乘以 p.grad。

step size

步长 数值梯度估计中用于扰动输入的小量，通常记为 h 。

数学与微积分

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 2:02:20 Right, but we have to think through some of the signs here. So in particular, working with this 对，但我们必须仔细想想这里的一些符号。特别是，结合这个

- ▶ 2:02:27 specific example here, we see that if we just left it like this, then this **neuron's value** 具体例子来看，我们看到如果就这样放着，那么这个神经元的值

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 2:02:33 would be currently increased by a tiny amount of the **gradient**. The **gradient** is negative.
目前会被梯度的一个微小量增加。梯度是负的。

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 2:02:42 So this **value** of this **neuron** would go slightly down. It would become like 0.8, you know,
所以这个神经元的值会稍微下降。它会变成大概 0.8，你知道，

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 2:02:49 4 or something like that. But if this **neuron's value** goes lower, that would actually increase the **loss**.
4 或者类似这样。但如果这个神经元的值变得更低，那实际上会增加损失。

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 2:02:55 That's because the **derivative** of this **neuron** is negative. So increasing this makes the **loss** go
这是因为这个神经元的导数是负的。所以增加它会让损失

derivative

导数 描述函数在某一点的变化率，即输入发生微小变化时输出相应变化的比率。

数学与微积分

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

- ▶ 2:03:00 down. So increasing it is what we want to do instead of decreasing it. So basically what
下降。所以增加它才是我们想做的，而不是减少它。所以基本上

2:03:44 the loss. But then we want to decrease it. So we actually want to go in the opposite
损失。但我们想减小它。所以我们实际上想往相反的

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

2:03:47 direction. And so you can convince yourself that this is the right thing here with the
方向。所以你可以说服自己，这里用这个

2:03:52 negative because we want to minimize the loss. So if we nudge all the parameters by a tiny

负号是对的，因为我们想最小化损失。所以如果我们把所有参数微调一点点，

minimize the loss

最小化损失 通过调整模型参数使损失数值尽可能小。

训练与优化

nudge

微调 对输入做极小扰动，用来看函数输出如何变化。

数学与微积分

2:03:57 amount, then we'll see that this neuron's data will have changed a little bit. So now this neuron's data

那么我们会看到这个神经元的数据会有点变化。所以现在这个神经元的数据

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

2:04:06 is a tiny amount greater value. So 0.8541 to 0.857. And that's a good thing because
会稍微大一点。所以从0.8541到0.857。这是好事，因为

value

Value 对象 `micrograd` 中表示标量及其梯度、参与计算图的基本类。

代码与工程

2:04:15 slightly increasing this neuron's data makes the loss go down according to the gradient.

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

▶ 2:04:22 And so the correct thing has happened, signwise. And so now what we would expect, of course, 所以从符号上看，正确的变化发生了。那么现在我们当然会预期，

▶ 2:04:28 is that because we've changed all these **parameters**, we expect that the **loss** should have gone down a bit.

因为我们已经改变了所有这些参数，我们预期损失应该会下降一点。

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

▶ 2:04:35 So we want to reevaluate the **loss**. Let me basically, this is just a data definition 所以我们想重新计算损失。让我基本来说，这只是一个数据定义

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

▶ 2:04:41 that hasn't changed. But the **forward pass** here of the network, we can recalculate. 它没有变。但网络的这个前向传播，我们可以重新计算。

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

▶ 2:04:49 And actually let me do it outside here so that we can compare the two **loss** values. 实际上让我在外面做，这样我们可以比较两个损失值。

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

▶ 2:04:54 So here, if I recalculate the **loss**, we'd expect the new **loss** now to be slightly lower than this 所以在这里，如果我重新计算损失，我们预期新的损失现在会比这个

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

▶ 2:05:00 number. So hopefully what we're getting now is a tiny bit lower than 4.84, 4.36. 数略低一点。所以希望我们现在得到的是比4.84略低一点，4.36。

- ▶ 2:05:08 Okay. And remember, the way we've arranged this is that low **loss** means that our predictions are good. 记住，我们这样安排的意思是，低损失意味着我们的预测

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 2:05:13 are matching the **targets**. So our predictions now are probably slightly closer to the **targets**. 与目标匹配。所以我们的预测现在可能稍微更接近目标了。

targets

目标值 训练样本中希望模型输出接近的真实标签或期望值。

训练与优化

- ▶ 2:05:19 And now all we have to do is we have to iterate this process. So again, we've done the **forward pass** 现在我们要做的就是重复这个过程。再说一次，我们已经做了前向传播

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

- ▶ 2:05:25 and this is the **loss**. Now we can call **loss.backward()**. Let me take these out and we can do a **step size**.

这就是损失。现在我们可以调用 `loss.backward()`。我把这些拿掉，然后我们可以做一个步长。

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:05:33 And now we should have a slightly lower **loss** 4.36 goes to 3.9. And okay, so we've done the 现在我们应该有一个略低的损失，4.36 变成 3.9。好的，所以我们已经完成了

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 2:05:41 **forward pass**. Here's the **backward pass**. **Nudge**. And now the **loss** is 3.66, 3.47.
前向传播。这里是反向传播。微调一下。现在损失是 3.66, 3.47。

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:05:52 And you get the idea. We just continue doing this. And this is **gradient** descent. We're just
你明白意思了。我们就继续这样做。这就是梯度下降。我们只是

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 2:05:57 iteratively doing **forward pass**, **backward pass**, update, **forward pass**, **backward pass**, update.
迭代地做前向传播、反向传播、更新，前向传播、反向传播、更新。

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:06:02 And the **neural net** is improving its predictions. So here if we look at y-pred now,
神经网络在改进它的预测。所以这里如果我们现在看 y-pred,

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:06:08 y-pred, we see that this **value** should be getting closer to 1. So this **value** should be getting more
y-pred, 我们看到这个值应该越来越接近 1。所以这个值应该变得更

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 2:06:17 positive. These should be getting more negative. And this one should be also getting more positive.
正。这些应该变得更负。而这个也应该变得更正。

▶ 2:06:21 So if we just iterate this a few more times, actually we may be able to afford to go a bit
所以如果我们再多迭代几次，实际上我们也许可以承受走得更

▶ 2:06:28 faster. Let's try a slightly higher **learning rate**. Oops. Okay. There we go. So now we're at
快一点。让我们试试稍微高一点的学习率。哎呀。好的。成功了。所以现在到了

learning rate

学习率 控制每次参数更新步长大小的超参数，过大会导致训练不稳定。

训练与优化

▶ 2:06:35 0.31. If you go too fast, by the way, if you try to make it too big of a step,
0.31。顺便说一句，如果你走得太快，如果你试图迈太大的步子，

▶ 2:06:42 you may actually overstep its overconfidence. Because again, remember, we don't actually know
你实际上可能会因过度自信而迈过头。因为再说一次，记住，我们实际上并不

▶ 2:06:49 exactly about the **loss function**. The **loss function** has all kinds of structure. And we only know about
确切了解损失函数。损失函数有各种各样的结构。我们只知道

loss function

损失函数 衡量模型预测与目标之间差异、训练中需要最小化的函数。

训练与优化

▶ 2:06:54 the very local dependence of all these **parameters** on the **loss**. But if we step too far, we may step
所有这些参数对损失的非常局部的依赖关系。但如果我们步子迈得太大，我们可能会踏入

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

▶ 2:07:00 into a part of the **loss** that is completely different. And that can destabilize training and
损失函数中一个完全不同的区域。这可能会使训练不稳定，

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 2:07:04 make your **loss** actually blow up even. So the **loss** is now 0.04. So actually, 甚至让你的损失爆炸。所以现在损失是 0.04。所以实际上,

loss

损失 衡量模型预测与目标差距的单一数值, 训练时通常希望它越小越好。

训练与优化

- ▶ 2:07:10 the predictions should be really quite close. Let's take a look. So you see how this is almost 1, 预测应该已经非常接近了。我们来看一下。所以你看这几乎是 1,

- ▶ 2:07:17 almost negative 1, almost 1. We can continue going. So yep, **backward**, update. Oops, there we go. 几乎负 1, 几乎 1。我们可以继续。所以是的, 反向传播, 更新。哎呀, 成功了。

backward

反向传播 从输出向输入传播梯度, 利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:07:26 So we went way too fast. And we actually overstepped. So we got to $2e8$. Where are we now? 所以我们走得太快了。我们实际上迈过头了。所以我们到了 $2e8$ 。我们现在在哪里?

- ▶ 2:07:35 Oops. Okay. $7e-9$. So this is very, very low **loss**. And the predictions are basically perfect. 哎呀。好的。 $7e-9$ 。所以这是非常非常低的损失。预测基本上完美。

loss

损失 衡量模型预测与目标差距的单一数值, 训练时通常希望它越小越好。

训练与优化

- ▶ 2:07:45 So somehow we basically, we were doing way too big updates and we briefly exploded, 所以不知怎么的, 我们基本上做了过大的更新, 一度爆炸,

- ▶ 2:07:50 but then somehow we ended up getting into a really good spot. So usually this **learning rate** and 但后来不知怎么地又进入了一个非常好的位置。所以通常这个学习率以及

learning rate

学习率 控制每次参数更新步长大小的超参数, 过大会导致训练不稳定。

训练与优化

- ▶ 2:07:55 the tuning of it is a subtle art. You want to set your **learning rate**. If it's too low, 它的调参是一门微妙的艺术。你需要设置好学习率。如果太低,

learning rate

学习率 控制每次参数更新步长大小的超参数, 过大会导致训练不稳定。

训练与优化

2:08:00 you're going to take way too long to **converge**. But if it's too high, the whole thing gets 收敛会花非常长的时间。但如果太高，整个训练就会变得

converge

收敛 训练过程中损失逐渐趋于稳定、不再明显下降的状态。

训练与优化

2:08:03 unstable and you might actually even **explode the loss** depending on your **loss function**. 不稳定，根据你的损失函数，你甚至可能让损失爆炸。

explode the loss

损失爆炸 训练中损失值急剧增大到无穷或 NaN，通常由学习率过大或梯度爆炸引起。

训练与优化

loss function

损失函数 衡量模型预测与目标之间差异、训练中需要最小化的函数。

训练与优化

2:08:08 So finding the **step size** to be just right, it's a pretty subtle art sometimes when you're using 所以找到恰到好处的步长，有时在使用

step size

步长 数值梯度估计中用于扰动输入的小量，通常记为 h 。

数学与微积分

2:08:13 sort of vanilla **gradient** descent. But we happen to get into a good spot. We can look at 那种标准梯度下降时，确实是一门相当微妙的艺术。但我们碰巧进入了一个不错的位置。我们可以看看

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

2:08:19 nn.**parameters**. So this is the setting of **weights** and biases that makes our network nn.parameters. 所以这就是让我们的网络

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

- ▶ 2:08:28 predict the desired **targets** very, very close. And basically we've successfully trained a **neural net**.
预测结果非常非常接近目标值的权重和偏置设置。基本上，我们已经成功训练了一个神经网络。

targets

目标值 训练样本中希望模型输出接近的真实标签或期望值。

训练与优化

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:08:38 Okay. Let's make this a tiny bit more respectable and implement an actual training loop and what
好的。让我们把它变得稍微正式一点，实现一个真正的训练循环，看看

- ▶ 2:08:42 that looks like. So this is the data definition that stays. This is the **forward pass**. So
它长什么样。所以这是保持不变的数据定义。这是前向传播。那么

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

- ▶ 2:08:49 for k in range, you know, we're going to take a bunch of steps. First you do the forward
对于 k in range，你知道，我们要走很多步。首先做前向

- ▶ 2:08:58 pass. We validate the **loss**. Let's reinitialize the **neural net** from scratch. And here's the data.
传播。我们验证损失。让我们从头重新初始化神经网络。这是数据。

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:09:08 And we first do **forward pass**, then we do the **backward** pass. And then we do an update. That's
我们先做前向传播，然后做反向传播。然后做一次更新。这就是

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:09:21 **gradient** descent. And then we should be able to iterate this and we should be able to print 梯度下降。然后我们应该能够迭代这个过程，并且能够打印

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 2:09:28 the current step, the current **loss**. Let's just print the sort of number of the **loss**. 当前的步数、当前的损失。我们就打印损失的数值。

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 2:09:36 And that should be it. And then the **learning rate** 0.01 is a little too small. 0.1 we saw is 应该就是这样。然后学习率 0.01 有点太小了。0.1 我们之前看到

learning rate

学习率 控制每次参数更新步长大小的超参数，过大会导致训练不稳定。

训练与优化

- ▶ 2:09:44 like a little bit dangerous if too high. Let's go somewhere in between. And we'll optimize this for 如果太高就有点危险。我们取个中间值。而且我们优化这个模型

- ▶ 2:09:50 not 10 steps, but let's go for say 20 steps. Let me erase all of this junk. And let's run the 不跑 10 步，而是比如说 20 步。让我把这些垃圾都擦掉。然后我们运行

- ▶ 2:10:00 optimization. And you see how we've actually converged slower in a more controlled manner. 优化。你会看到我们实际上收敛得更慢，但方式更可控。

- ▶ 2:10:08 And got to a **loss** that is very low. So I expect the predictions to be quite good. 而且得到了一个非常低的损失。所以我预期预测结果会相当不错。

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 2:10:15 There we go. And that's it. Okay, so this is kind of embarrassing, but we actually have a really 好了。就这样。好吧，这有点尴尬，但我们这里其实有一个非常

- ▶ 2:10:26 terrible bug in here. And it's a subtle bug. And it's a very common bug. And I can't believe 严重的 bug。而且这是一个很隐蔽的 bug。也是一个非常常见的 bug。我简直不敢相信

- ▶ 2:10:33 I've done it for the 20th time in my life, especially on camera. And I could have 我这辈子已经犯了 20 次了，尤其是在镜头前。我本来可以

- ▶ 2:10:38 reshot the whole thing, but I think it's pretty funny. And you get to appreciate a bit what working 把整个视频重拍一遍，但我觉得这挺有趣的。而且你可以稍微体会到，从事
- ▶ 2:10:44 with **neural nets** maybe is like sometimes we are guilty of a common bug. I've actually tweeted 神经网络相关工作时可能是什么样子，有时我们也会犯一些常见的 bug。我其实以前发过推文

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

梯度重置与常见错误 02:10

- ▶ 2:10:52 the most common **neural net** mistakes a long time ago now. And I'm not really going to explain 列出了最常见的神经网络错误，那是很久以前了。我不打算解释

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:11:00 any of these except for we are guilty of number three, you forgot to **zero grad** before 其中的任何一个，除了我们犯了第三条：你忘了在

zero grad

梯度清零 在反向传播前将所有参数的梯度重置为零，防止梯度累加。

训练与优化

- ▶ 2:11:05 dot **backward**. What is that? Basically what's happening, and it's a subtle bug and I'm not sure if you saw dot backward 之前做 zero grad。那是什么意思？基本上发生的情况是——这是一个隐蔽的 bug，我不确定你们是否看出来了——

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:11:12 it is that all of these `weights` here have a dot `data` and a dot `grad`. And dot `grad` starts at zero. 就是这里所有这些权重都有一个 dot data 和一个 dot grad。而 dot grad 一开始是零。

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 2:11:22 And then we do `backward` and we fill in the `gradients`. And then we do an update on the data, 然后我们执行 backward，填充梯度。接着我们对 data 做一次更新，

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 2:11:27 but we don't flush the `grad`. It stays there. So when we do the second `forward pass` and we 但我们没有清空 grad。它留在那里。所以当我们做第二次前向传播，并且我们

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

- ▶ 2:11:34 do `backward` again, remember that all the `backward` operations do a `plus equals` on the `grad`. 再次执行 backward 时，记住所有反向操作都会对 grad 做 plus equals (+=)。

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

plus equals

加等于 (+=) 代码中在原值上累加的写法，用于实现梯度累加，避免覆盖已有梯度。

代码与工程

▶ 2:11:39 And so these **gradients** just add up and they never get reset to zero. So basically we didn't **zero grad**.

因此这些梯度就不断累加，永远不会被重置为零。所以基本上我们没有执行 zero grad。

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

zero grad

梯度清零 在反向传播前将所有参数的梯度重置为零，防止梯度累加。

训练与优化

▶ 2:11:47 So here's how we **zero grad** before **backward**. We need to iterate over all the **parameters**.

那么，在 backward 之前进行 zero grad 的方法是这样。我们需要遍历所有参数。

zero grad

梯度清零 在反向传播前将所有参数的梯度重置为零，防止梯度累加。

训练与优化

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 2:11:54 And we need to make sure that $p \cdot \text{grad}$ is set to zero. We need to reset it to zero, 并且需要确保 $p \cdot \text{grad}$ 被设为零。我们需要把它重置为零，

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

▶ 2:12:00 just like it is in the **constructor**. So remember all the way here for all these **value nodes**, 就像构造函数里那样。记住，一直到这里，对于所有这些 value 节点，

constructor

构造函数 创建对象时自动调用的初始化方法，用于设置实例属性。

代码与工程

value node

值节点 micrograd 中存储数据 data 和梯度 grad 的计算图节点。

代码与工程

- ▶ 2:12:05 `grad` is reset to zero. And then all these `backward` passes do a `plus equals` on that `grad`.
grad 都被重置为零。然后所有这些 backward pass 都会对那个 grad 做 plus equals。

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:12:11 But we need to make sure that we reset these grads to zero so that when we do `backward`,
但我们需要确保把这些 grad 重置为零，这样当我们执行 backward 时，

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:12:17 all of them start at zero and the actual `backward` pass accumulates the `loss` derivatives into
它们都从零开始，而真正的反向传播会把损失导数累加进

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 2:12:24 the grads. So this is `zero grad` in `PyTorch`. And we will get a slightly different optimization.
.....梯度。所以这就是 PyTorch 里的 `zero_grad`。这样我们会得到一个稍微不同的优化过程。

zero grad

梯度清零 在反向传播前将所有参数的梯度重置为零，防止梯度累加。

训练与优化

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

- ▶ 2:12:33 Let's reset the `neural net`. The data is the same. This is now, I think, correct. And we get a much
让我们重置神经网络。数据还是一样的。现在这个，我觉得，是正确的。而我们得到的下降要

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:12:39 more slower descent. We still end up with pretty good results and we can continue this a bit more 慢得多。我们最终仍然得到相当不错的结果，而且我们还可以再继续多做一点
- ▶ 2:12:48 to get down lower and lower and lower. Yeah. So the only reason that the previous thing worked, 把损失降得越来越低。对。所以之前那套之所以还能跑通，
- ▶ 2:12:57 it's extremely buggy. The only reason that worked is that this is a very, very simple problem. 其实是因为它 bug 极其严重。它之所以能跑通，唯一的原因就是这个问题非常非常简单。
- ▶ 2:13:05 And it's very easy for this **neural net** to fit this data. And so the grads ended up accumulating 而这个神经网络非常容易拟合这份数据。于是梯度最终就被累积了起来

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:13:12 and it effectively gave us a massive **step size** and it made us **converge** extremely fast. 这实际上给了我们一个巨大的步长，让我们收敛得极快。

step size

步长 数值梯度估计中用于扰动输入的小量，通常记为 h 。

数学与微积分

converge

收敛 训练过程中损失逐渐趋于稳定、不再明显下降的状态。

训练与优化

- ▶ 2:13:19 But basically now we have to do more steps to get to very low values of **loss** 但基本上现在我们必须走更多步，才能把损失降到很低

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 2:13:24 and get y_{pred} to be really good. We can try to step a bit greater. Yeah. We're going to get closer 让 y_{pred} 变得真的很好。我们可以试着把步长调大一点。对。我们会越来越
- ▶ 2:13:35 and closer to one minus one and one. So working with **neural nets** is sometimes tricky because 接近 1、-1、1。所以和神经网络打交道有时是很棘手的，因为

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:13:45 you may have lots of bugs in the code and your network might actually work, 你的代码里可能有一大堆 bug，而你的网络居然还能正常工作，

- ▶ 2:13:49 just like ours worked. But chances are is that if we had a more complex problem, 就像我们刚才那样。但很可能情况是，如果我们面对的是一个更复杂的问题，
- ▶ 2:13:54 then actually this bug would have made us not optimize the **loss** very well. And we were only 那么这个 bug 实际上会让我们没法把损失优化得很好。而我们只是

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 2:13:58 able to get away with it because the problem is very simple. So let's now bring everything 因为问题非常简单才侥幸混了过去。那么现在，让我们把所有东西
- ▶ 2:14:04 together and summarize what we learned. What are **neural nets**? **Neural nets** are these mathematical 汇总起来，总结一下我们学到了什么。什么是神经网络？神经网络就是那些数学

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:14:09 expressions, fairly simple mathematical expressions in the case of **multi-layer perceptron**, 表达式，就多层感知机而言，是相当简单的数学表达式，

multi-layer perceptron

多层感知机 由多个全连接层堆叠而成的前馈神经网络，是最基础的深度网络结构。

神经网络基础

总结与现代神经网络 02:14

- ▶ 2:14:15 that take input as the data and they take input, the **weights** and the **parameters** of the **neural net**. 它们以数据作为输入，同时也以神经网络的权重和参数作为输入。

weight

权重 神经网络中与输入相乘的可学习参数，决定输入对神经元输出的影响。

神经网络基础

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

- ▶ 2:14:21 Mathematical **expression** for the **forward pass**, followed by a **loss function**.
一个用于前向传播的数学表达式，后面再接一个损失函数。

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

- ▶ 2:14:25 And the **loss function** tries to measure the accuracy of the predictions. And usually the
损失函数试图衡量预测的准确程度。通常

loss function

损失函数 衡量模型预测与目标之间差异、训练中需要最小化的函数。

训练与优化

- ▶ 2:14:30 **loss** will be low when your predictions are matching your **targets** or where the network is basically
当你的预测与目标相匹配时，损失会很低，或者说网络基本上.....

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

targets

目标值 训练样本中希望模型输出接近的真实标签或期望值。

训练与优化

- ▶ 2:14:35 behaving well. So we manipulate the **loss function** so that when the **loss** is low,
表现良好。所以我们操纵损失函数，使得当损失很低时，

loss function

损失函数 衡量模型预测与目标之间差异、训练中需要最小化的函数。

训练与优化

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

2:14:40 the network is doing what you wanted to do on your problem. And then we backward the loss, 网络就在你的问题上做你期望它做的事。然后我们对损失执行反向传播,

backward

反向传播 从输出向输入传播梯度, 利用链式法则计算每个变量的梯度。

反向传播与梯度

loss

损失 衡量模型预测与目标差距的单一数值, 训练时通常希望它越小越好。

训练与优化

2:14:46 use backpropagation to get the gradient, and then we know how to tune all the parameters to 用反向传播得到梯度, 然后我们就知道如何调整所有参数, 以

backpropagation

反向传播 从输出节点开始, 沿计算图反向递归应用链式法则, 计算每个中间节点和输入对输出的导数。

反向传播与梯度

gradient

梯度 损失函数对参数的变化率, 指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

2:14:50 decrease the loss locally. But then we have to iterate that process many times in what's 局部地降低损失。但之后我们必须多次迭代这个过程, 也就是所谓的

loss

损失 衡量模型预测与目标差距的单一数值, 训练时通常希望它越小越好。

训练与优化

2:14:55 called the gradient descent. So we simply follow the gradient information and that 梯度下降。所以我们只需沿着梯度信息走, 这就会

gradient

梯度 损失函数对参数的变化率, 指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

2:14:59 minimizes the loss and the loss is arranged so that when the loss is minimized, the network 最小化损失, 而损失被设计成: 当损失被最小化时, 网络

loss

损失 衡量模型预测与目标差距的单一数值, 训练时通常希望它越小越好。

训练与优化

2:15:04 is doing what you wanted to do. And yeah, so we just have a blob of neural stuff and we can make 就在做你希望它做的事。对, 所以我们只是有一团神经物质, 就可以让它

- ▶ 2:15:11 it do arbitrary things. And that's what gives **neural nets** their power. It's, you know, this is a 做任意的事情。这就是神经网络威力的来源。你知道，这是一个

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:15:16 very tiny network with 41 **parameters**. But you can build significantly more complicated **neural nets** 非常小的网络，只有41个参数。但你可以构建复杂得多的神经网络，

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:15:22 with billions at this point, almost trillions of **parameters**. And it's a massive blob of neural 如今有数十亿、甚至接近数万亿个参数。它是一大团神经

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

- ▶ 2:15:28 tissue, simulated neural tissue, roughly speaking. And you can make it do extremely complex problems.

组织，粗略地说，是模拟的神经组织。而且你可以让它解决极其复杂的问题。

- ▶ 2:15:35 And these **neural nets** then have all kinds of very fascinating **emergent properties** when you try to 然后这些神经网络会具有各种非常迷人的涌现特性，尤其是当你试图

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

emergent properties

涌现特性 模型规模和数据量增大后表现出的、并非由单个组件直接设计出的能力。

神经网络基础

- ▶ 2:15:42 make them do significantly hard problems. As in the case of GPT, for example, we have massive 让它们解决相当困难的问题时。比如在 GPT 的例子中，我们有海量

- ▶ 2:15:48 amounts of text from the internet. And we're trying to get a **neural net** to predict, to take
来自互联网的文本。我们试图让神经网络去预测，去拿

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:15:52 like a few words and try to predict the next word in a sequence. That's the learning problem.
比如几个词，然后尝试预测序列中的下一个词。这就是学习问题。

- ▶ 2:15:57 And it turns out that when you train this on all of the internet, the **neural net** actually has like
事实证明，当你在整个互联网上训练它时，神经网络确实会具有

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:16:01 really remarkable **emergent properties**. But that **neural net** would have hundreds of billions of
非常显著的涌现特性。但那个神经网络会有数千亿个

emergent properties

涌现特性 模型规模和数据量增大后表现出的、并非由单个组件直接设计出的能力。

神经网络基础

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:16:05 **parameters**. But it works on fundamentally these exact same principles. The **neural net**, of course,
参数。但它从根本上遵循完全相同的原理。当然，这个神经网络

parameters

参数 神经网络中可学习的变量，例如权重和偏置，训练时通过梯度更新。

神经网络基础

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:16:11 will be a bit more complex. But otherwise, the evaluation of the **gradient** is there and will be
会稍微复杂一些。但除此之外，梯度的计算是存在的，并且将会

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 2:16:17 identical. And the **gradient descent** would be there and would be basically identical. But 完全相同。梯度下降也会存在，并且基本上完全相同。但

gradient

梯度 损失函数对参数的变化率，指示参数应朝哪个方向调整以降低损失。

反向传播与梯度

- ▶ 2:16:22 people usually use slightly different updates. This is a very simple **stochastic gradient descent** 人们通常使用稍微不同的更新。这是一个非常简单的随机梯度下降

stochastic gradient descent

随机梯度下降 一种优化算法，每次使用一小批数据计算梯度并更新参数。

训练与优化

- ▶ 2:16:27 update. And the **loss function** would not be **mean squared error**, they would be using something 更新。损失函数不会是均方误差，他们会使用一种

loss function

损失函数 衡量模型预测与目标之间差异、训练中需要最小化的函数。

训练与优化

mean squared error

均方误差 把预测值与真实值之差平方后取平均得到的常用损失。

训练与优化

- ▶ 2:16:32 called the **cross entropy loss** for predicting the next token. So there's a few more details, 称为交叉熵损失的东西来预测下一个词元。所以还有一些细节，

cross entropy loss

交叉熵损失 用于分类任务的损失函数，衡量预测概率分布与真实分布的差异。

训练与优化

- ▶ 2:16:36 but fundamentally, the **neural network** setup and **neural network** training is identical and 但从根本上说，神经网络的设置和神经网络的训练是相同的，并且

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- ▶ 2:16:40 pervasive. And now you understand intuitively how that works under the hood. In the beginning 普遍适用。现在你直观地理解了它底层是如何工作的。在这个视频的开头

- ▶ 2:16:46 of this video, I told you that by the end of it, you would understand everything in `micrograd`
我告诉过你，到视频结束时，你会理解 `micrograd` 中的所有内容

micrograd

`micrograd` 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 2:16:50 and then we'd slowly build it up. Let me briefly prove that to you. So I'm going to step through
然后我们会慢慢地构建它。让我简要地向你证明这一点。所以我要逐步讲解

- ▶ 2:16:54 all the code that is in `micrograd` as of today. Actually, potentially some of the code will
目前 `micrograd` 中的所有代码。实际上，有些代码可能会

micrograd

`micrograd` 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 2:16:59 change by the time you watch this video because I intend to continue developing `micrograd`. But
在你观看这个视频时发生变化，因为我打算继续开发 `micrograd`。但是

micrograd

`micrograd` 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 2:17:03 let's look at what we have so far at least in it that `.py` is empty. When you go to `engine.py`,
让我们看看目前我们有什么，至少在其中那个 `.py` 是空的。当你打开 `engine.py` 时，

- ▶ 2:17:08 that has the `value`, everything here you should mostly recognize. So we have the data that
那里有 `Value`，这里的大部分内容你应该都能认出来。所以我们有 `data` 这个

value

`Value` 对象 `micrograd` 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 2:17:13 `grad` attributes with the `backward` function, we have the previous set of `children` and the
带有 `backward` 函数的 `grad` 属性，我们有之前的一组子节点和

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:17:17 operation that produced this `value`. We have addition multiplication and raising to a `scalar` power.
产生这个值的操作。我们有加法、乘法和标量幂运算。

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

scalar

标量 单个数值，区别于向量、矩阵或更高维张量。

数学与微积分

- ▶ 2:17:25 We have the `relu` nonlinearity, which is slightly different type of nonlinearity than `tanh` that we
我们有 relu 非线性函数，它是一种与 tanh 略有不同的非线性函数，我们

relu

ReLU 修正线性单元激活函数，负值置零，正值保持不变。

神经网络基础

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 2:17:29 used in this video. Both of them are nonlinearity and notably `tanh` is not actually present in
在这个视频中使用的。它们都是非线性函数，值得注意的是 tanh 实际上并不存在于

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 2:17:34 `micrograd` as of right now, but I intend to add it later. We have the `backward`, which is
目前的 micrograd 中，但我打算稍后添加它。我们有 backward，它是

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:17:39 identical and then all of these other operations, which are built up on top of
相同的，然后所有这些其他操作，它们都构建在

- ▶ 2:17:44 operations here. So `value` should be very recognizable except for the nonlinearity used in this video.
这里的操作之上。所以 Value 应该非常容易识别，除了这个视频中使用的非线性函数。

value

Value 对象 micrograd 中表示标量及其梯度、参与计算图的基本类。

代码与工程

- ▶ 2:17:50 There's no massive difference between `relu` and `tanh` and sigmoid and these other nonlinearity.
relu、tanh、sigmoid 和其他非线性函数之间没有巨大的区别。

relu

ReLU 修正线性单元激活函数，负值置零，正值保持不变。

神经网络基础

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 2:17:55 They're all roughly equivalent and can be used in MLPs. So I use `tanh` because it's a bit smoother
它们大致等价，都可以在 MLP 中使用。所以我使用 tanh，因为它稍微平滑一些

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 2:18:00 and because it's a little bit more complicated than `ReLU` and therefore it stressed a little bit
因为它比 ReLU 稍微复杂一点，所以它稍微强调了一点

relu

ReLU 修正线性单元激活函数，负值置零，正值保持不变。

神经网络基础

- ▶ 2:18:04 more the `local gradients` and working with those derivatives, which I thought would be useful.
更多的是局部梯度和处理这些导数，我认为这会很有用。

local gradient

局部梯度 某个运算节点对其输入变量的导数，只由该节点自身的运算决定。

反向传播与梯度

- ▶ 2:18:10 And in that repo is the `neural networks` library, as I mentioned. So you should recognize identical
在那个仓库里是神经网络库，正如我提到的。所以你应该能认出完全相同的

neural network

神经网络 由多层神经元组成、通过权重和激活函数对输入进行非线性变换的数学模型。

神经网络基础

- ▶ 2:18:15 implementation of `Neuron`, `layer` and `MLP`. Notably, or not so much, we have a class module here.
Neuron、Layer 和 MLP 的实现。值得注意的是，或者也没那么重要，我们这里有一个类模块。

neuron

神经元 神经网络的基本单元，对输入做加权求和并加上偏置后经过激活函数。

神经网络基础

layer

层 一组独立计算的神经元，通常将同一维度的输入映射到多个输出。

神经网络基础

- ▶ 2:18:21 There is a parent class of all these modules. I did that because there's an `nn.Module`
它是所有这些模块的父类。我这样做是因为有一个 `nn.Module`

nn.module

PyTorch 模块基类 PyTorch 中所有神经网络模块的基类，提供 `parameters` 等方法。

代码与工程

- ▶ 2:18:26 class in `PyTorch` and so this exactly matches that API. And `nn.Module` in `PyTorch` has also a
类在 PyTorch 中，所以这完全匹配那个 API。而且 PyTorch 中的 `nn.Module` 还有一个

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

nn.module

PyTorch 模块基类 PyTorch 中所有神经网络模块的基类，提供 `parameters` 等方法。

代码与工程

- ▶ 2:18:32 `zero_grad` which I refactored out here. So that's the end of `micrograd` really. Then there's a test
`zero_grad`，我在这里把它重构出来了。所以这就是 `micrograd` 的结尾了。然后有一个测试

zero_grad

梯度清零 PyTorch 中用于将模型参数的梯度置零的方法，通常在反向传播前调用，避免梯度累积。

训练与优化

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 2:18:40 which you'll see basically creates two chunks of code, one in `micrograd` and one in `PyTorch`, 你会看到它基本上创建了两块代码，一块用 `micrograd`，一块用 `PyTorch`，

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

- ▶ 2:18:46 and we'll make sure that the forward and the `backward` paths agree identically. 我们会确保前向和反向路径完全一致。

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:18:50 For a slightly less complicated `expression`, a slightly more complicated `expression`, 对于一个稍微不那么复杂的表达式，一个稍微更复杂的表达式，

expression

表达式 由多个运算组合而成的计算式，在计算图中对应多个节点。

代码与工程

- ▶ 2:18:54 everything agrees. So we agree with `PyTorch` on all of these operations. 一切都一致。所以我们在所有这些操作上都与 `PyTorch` 一致。

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

- ▶ 2:18:59 And finally, there's a `demo.ipynb` here and it's a bit more complicated binary classification demo 最后，这里有一个 `demo.ipynb`，它是一个稍微更复杂的二分类演示

- ▶ 2:19:04 than the one I covered in this lecture. So we only had a tiny data set of four examples. 比我在这节课上讲的要复杂一点。我们之前只有四个例子的小数据集。

- ▶ 2:19:09 Here, we have a bit more complicated example with lots of blue points and lots of red points 这里，我们有一个稍微更复杂的例子，有很多蓝点和很多红点

- ▶ 2:19:14 and we're trying to again build a **binary classifier** to distinguish two dimensional
我们再次尝试构建一个二分类器来区分二维

binary classifier

二分类器 将输入分为两个类别的模型。

神经网络基础

- ▶ 2:19:18 points as red or blue. It's a bit more complicated **MLP** here with it's a bigger **MLP**.
点为红色或蓝色。这里是一个稍微更复杂的 MLP，它是一个更大的 MLP。

mlp

多层感知机 由多个全连接层堆叠而成的前馈神经网络。

神经网络基础

- ▶ 2:19:24 The **loss** is a bit more complicated because it supports batches. So because our data set was so
损失函数稍微更复杂，因为它支持批次。因为我们的数据集非常

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 2:19:30 tiny, we always did a **forward pass** on the entire data set of four examples. But when your data set
小，我们总是在整个四个例子的数据集上做前向传播。但当你的数据集

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

- ▶ 2:19:35 is like a million examples, what we usually do in practice is we basically pick out some random
有一百万个例子时，我们通常在实践中做的是，我们基本上挑出一些随机

- ▶ 2:19:41 subset. We call that a **batch** and then we only process the **batch** forward, **backward** and update.
子集。我们称之为一个批次，然后我们只对这个批次进行前向、反向和更新。

batch

批次 训练时一次前向和反向传播所使用的样本子集，通常从数据集中随机抽取。

训练与优化

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:19:46 So we don't have to forward the entire training set. So this supports batching because there's
这样我们就不必对整个训练集做前向传播。所以它支持批处理，因为这里有

- ▶ 2:19:51 a lot more examples here. We do a **forward pass**. The **loss** is slightly more different. This is a 多得多的样本。我们做一次前向传播。这里的损失略有不同。这是一个

forward pass

前向传播 按计算图从输入计算到输出，得到预测值或损失。

神经网络基础

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 2:19:57 max margin **loss** that I implement here. The one that we used was the **mean squared error loss** 我在这里实现的最大间隔损失。我们之前用的是均方误差损失

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

mean squared error

均方误差 把预测值与真实值之差平方后取平均得到的常用损失。

训练与优化

- ▶ 2:20:02 because it's the simplest one. There's also the **binary cross entropy loss**. All of them can be 因为它是最简单的。还有二元交叉熵损失。它们都可以

binary cross entropy

二元交叉熵 针对二分类任务的损失函数，衡量预测概率分布与真实标签之间的差异。

训练与优化

loss

损失 衡量模型预测与目标差距的单一数值，训练时通常希望它越小越好。

训练与优化

- ▶ 2:20:07 used for binary classification and don't make too much of a difference in the simple examples 用来做二分类，而且在我们目前看过的这些简单例子里差别并不大

- ▶ 2:20:11 that we looked at so far. There's something called **L2 regularization** used here. This has to do 这里用到了一个叫 L2 正则化的东西。它

L2 regularization

L2 正则化 在损失中加入权重平方和的惩罚项，抑制权重过大，从而缓解过拟合。

训练与优化

- ▶ 2:20:17 with generalization of the **neural net** and controls the **overfitting** in machine learning setting, 与神经网络的泛化能力有关，用来控制机器学习场景中的过拟合，

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

overfitting

过拟合 模型在训练数据上表现很好但在新数据上变差的现象，通常因模型过于复杂或数据太少。

训练与优化

- ▶ 2:20:22 but I did not cover these concepts in this video potentially later. And the training loop you 但我在这个视频里没有讲这些概念，也许以后会讲。而训练循环，你们

- ▶ 2:20:27 should recognize. So forward, **backward**, with **zero grad** and update and so on. You'll notice 应该能认出来。就是前向、反向、清零梯度、更新参数等等。你会注意到

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

zero grad

梯度清零 在反向传播前将所有参数的梯度重置为零，防止梯度累加。

训练与优化

- ▶ 2:20:35 that in the update here, the **learning rate** is scaled as a function of number of iterations 在这里的更新中，学习率是按迭代次数的函数进行缩放的

learning rate

学习率 控制每次参数更新步长大小的超参数，过大会导致训练不稳定。

训练与优化

- ▶ 2:20:39 and it shrinks. And this is something called **learning rate** decay. So in the beginning, 而且它会逐渐变小。这就是所谓的学习率衰减。所以在开始的时候，

learning rate

学习率 控制每次参数更新步长大小的超参数，过大会导致训练不稳定。

训练与优化

- ▶ 2:20:44 you have a high **learning rate** and as the network sort of stabilizes near the end, 学习率比较高，而当网络在接近结束时逐渐稳定下来，

learning rate

学习率 控制每次参数更新步长大小的超参数，过大会导致训练不稳定。

训练与优化

- ▶ 2:20:49 you bring down the `learning rate` to get some of the fine details in the end. And in the end, 你就把学习率降下来，以便在最后抓住一些精细的细节。最后，

learning rate

学习率 控制每次参数更新步长大小的超参数，过大会导致训练不稳定。

训练与优化

- ▶ 2:20:54 we see the decision surface of the `neural net` and we see that it learns to separate out the red 我们看到神经网络的决策面，看到它学会了把红色区域

neural net

神经网络 由多层可学习参数组成的函数逼近器，用于从输入预测输出。

神经网络基础

- ▶ 2:20:58 and the blue area based on the data points. So that's the slightly more complicated example 和蓝色区域根据数据点区分开来。这就是那个稍微复杂一点的例子

- ▶ 2:21:03 in the `demo.ipynb` that you're free to go over. But yeah, as of today, that is , 在 `demo.ipynb` 里，你们可以自己去过一遍。不过，到今天为止，这就是

- ▶ 2:21:09 `micrograd` . I also wanted to show you a little bit of real stuff so that you get to see how this `micrograd`. 我还想给你们看一点真东西，让你们看看这

micrograd

`micrograd` 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 2:21:13 is actually implemented in the production grade library like `PyTorch` . So in particular, I wanted 在生产级库（比如 `PyTorch`）里究竟是怎样实现的。具体来说，我想

pytorch

`PyTorch` 主流的深度学习框架，`micrograd` 的接口设计大致仿照它。

代码与工程

- ▶ 2:21:18 to show I wanted to find and show you the `backward` pass for `tanh` in `PyTorch` . So here in `micrograd` ,

找到并给你们展示 `PyTorch` 里 `tanh` 的反向传播。在 `micrograd` 里，

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 2:21:24 we see that the `backward` pass for `tanh` is 1 minus t square, where t is the output of the `tanh` of x
我们看到 `tanh` 的反向传播是 1 减 t 的平方，其中 t 是 `tanh(x)` 的输出

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 2:21:33 times the local `grad`, which is the `chain rule`. So we're looking for something that looks like
乘以局部梯度，这就是链式法则。所以我们要找的东西看起来像

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

chain rule

链式法则 微积分中计算复合函数导数的法则，即外层函数的导数乘以内层函数的导数。

数学与微积分

- ▶ 2:21:37 Now, I went to `PyTorch`, which has an open source GitHub code base. And I looked through a lot
现在，我去了PyTorch，它有一个开源的GitHub代码库。我翻阅了很多

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

- ▶ 2:21:45 of its code. And honestly, I spent about 15 minutes and I couldn't find `tanh`. And that's because
它的代码。老实说，我花了大约15分钟，也没找到tanh。那是因为

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 2:21:52 these libraries, unfortunately, they grow in size and entropy. And if you just search for `tanh`,
这些库不幸的是，它们在规模和熵上不断增长。如果你只是搜索tanh，

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 2:21:57 you get apparently 2,800 results and 406 files. So I don't know what these files are doing,
显然会得到2,800个结果和406个文件。所以我不知道这些文件在做什么，

- ▶ 2:22:05 honestly. And why there are so many mentions of `tanh`. But unfortunately, these libraries are quite 老实说。以及为什么有这么地方提到tanh。但不幸的是，这些库相当

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 2:22:11 complex. They're meant to be used, not really inspected. Eventually, I did stumble on someone 复杂。它们是拿来用的，而不是真的用来检查的。最终，我偶然发现有人

- ▶ 2:22:18 who tries to change the `tanh` `backward` code for some reason. And someone here pointed to the CPU

出于某种原因试图修改tanh反向传播代码。这里有人指向了CPU

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:22:24 kernel and the `CUDA kernel` for `tanh` `backward`. So this, so basically depends on if you're 核函数和tanh反向传播的CUDA核函数。所以这，所以基本上取决于你是

cuda kernel

CUDA核函数 在NVIDIA GPU上并行执行的函数，PyTorch用其实现张量运算。

代码与工程

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 2:22:29 using `PyTorch` on a CPU device or on a GPU, which these are different devices and I haven't 在CPU设备上还是GPU上使用PyTorch，这是不同的设备，我还没有

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

▶ 2:22:34 covered this. But this is the `tanh` `backward` kernel for CPU. And the reason it's so large is that 讲过这个。但这是CPU的tanh反向传播核函数。它之所以这么大，是因为

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 2:22:43 number one, this is like if you're using a complex type, which we haven't even talked about, 第一，这就像如果你使用复数类型，我们甚至还没讨论过，

▶ 2:22:47 if you're using a specific data type of `bfloat16`, which we haven't talked about. 如果你使用特定的bfloat16数据类型，我们还没讨论过。

bfloat16

bfloat16 一种16位浮点数格式，保留与FP32相同的指数范围但降低尾数精度，常用于加速训练。

训练与优化

▶ 2:22:52 And then if you're not, then this is the kernel and deep here, we see something that 然后如果你不是，那么这就是核函数，而在这里深入看，我们看到一些

▶ 2:22:57 resembles our `backward` pass. So they have a times one minus b squared. So this b here must be the 类似于我们的反向传播。所以它们有a乘以1减去b的平方。所以这里的b一定是

backward

反向传播 从输出向输入传播梯度，利用链式法则计算每个变量的梯度。

反向传播与梯度

▶ 2:23:06 output of the `tanh` and this is the out.`grad`. So here we found it deep inside `PyTorch` on this tanh的输出，而这是out.grad。所以在这里我们在PyTorch深处找到了它，在这个

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

grad

梯度 计算图中某个变量累积的导数，表示最终输出对该变量的敏感度。

反向传播与梯度

- ▶ 2:23:14 location for some reason inside binary ops kernel when `tanh` is not actually a binary op.
位置，出于某种原因它在二元运算核函数里，而tanh实际上并不是二元运算。

tanh

双曲正切函数 把输入平滑地压缩到 $(-1, 1)$ 区间的函数，常用作神经网络的激活函数。

数学与微积分

- ▶ 2:23:20 And then this is the GPU kernel. We're not complex. We're here and here we go with one line of code.
然后这是GPU核函数。我们不是复数。我们在这里，然后这里我们用一行代码就搞定了。

- ▶ 2:23:30 So we did find it, but basically unfortunately these code bases are very large and
所以我们确实找到了它，但基本上不幸的是，这些代码库非常大，而

- ▶ 2:23:36 `micrograd` is very, very simple. But if you actually want to use real stuff,
micrograd非常非常简单。但如果你真的想用真实的东西，

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 2:23:40 finding the code for it, you'll actually find that difficult. I also wanted to show you
去找它的代码，你其实会发现挺难的。我还想给你展示

- ▶ 2:23:45 example here where `PyTorch` is showing you how you can register a new type of function that you
这里的一个例子，PyTorch 向你展示了如何注册一种新的函数类型，你可以

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

- ▶ 2:23:50 want to add to `PyTorch` as a `Lego building block`. So here if you want to, for example, add a
把它作为一块乐高积木加到 PyTorch 里。所以这里，如果你想，比如说，加一个

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

lego building block

乐高积木式模块 指可以任意拼装复用的基础算子单元，用它们逐层搭建出更大的计算图结构。

代码与工程

- ▶ 2:23:55 **Legendre polynomial** three, here's how you can do it. You will register it as a class that 三阶勒让德多项式, 这里是做法。你会把它注册成一个类,

legendre polynomial

勒让德多项式 一类在区间 $[-1,1]$ 上正交的多项式序列, 常用于数值计算, 也常被当作自定义算子的教学示例。

数学与微积分

- ▶ 2:24:03 subclasses **torch.autograd.Function**. And then you have to tell **PyTorch** how to forward 它继承 torch.autograd.Function。然后你必须告诉 PyTorch 如何前向

torch.autograd.function

torch.autograd.Function PyTorch 中用于自定义可自动求导算子的基类, 通过定义 forward 与 backward 让新函数接入自动微分系统。

代码与工程

pytorch

PyTorch 主流的深度学习框架, micrograd 的接口设计大致仿照它。

代码与工程

- ▶ 2:24:08 your new function and how to **backward** through it. So as long as you can do the forward 你的新函数, 以及如何对它做反向。所以只要你能完成这个

backward

反向传播 从输出向输入传播梯度, 利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:24:14 pass of this little function piece that you want to add, and as long as you know the **local derivative**, 你想加入的小函数片段的前向传播, 并且只要你知道局部导数,

local derivative

局部导数 计算图中某个运算节点对其直接输出变量的导数, 不涉及图的其他部分。

反向传播与梯度

- ▶ 2:24:18 the **local gradients** which are implemented in the **backward**, **PyTorch** will be able to back 也就是在 backward 里实现的局部梯度, PyTorch 就能够反向

local gradient

局部梯度 某个运算节点对其输入变量的导数, 只由该节点自身的运算决定。

反向传播与梯度

backward

反向传播 从输出向输入传播梯度, 利用链式法则计算每个变量的梯度。

反向传播与梯度

- ▶ 2:24:22 **propagate** through your function. And then you can use this as a Lego block in a larger Lego castle
传播穿过你的函数。然后你就可以把它当作一块乐高积木，放进一座更大的乐高城堡里，

propagate

传播 梯度沿计算图从输出向输入传递的过程。

反向传播与梯度

- ▶ 2:24:27 of all the different Lego blocks that **PyTorch** already has. And so that's the only thing
和 PyTorch 已有的各种乐高积木一起。所以这就是你唯一

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

- ▶ 2:24:32 you have to tell **PyTorch** and everything would just work. And you can register new types of
需要告诉 PyTorch 的东西，一切就能正常运作。你可以用这种方式

pytorch

PyTorch 主流的深度学习框架，micrograd 的接口设计大致仿照它。

代码与工程

- ▶ 2:24:35 functions in this way following this example. And that is everything that I wanted to cover
按照这个例子注册新的函数类型。这就是我想在这节课里

- ▶ 2:24:40 in this lecture. So I hope you enjoyed building out **micrograd** with me. I hope you find it interesting,
覆盖的全部内容。希望你们喜欢和我一起构建 micrograd。希望你们觉得它有趣、

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 2:24:45 insightful, and yeah, I will post a lot of the links that are related to this video in the video
有启发，然后，我会把很多与这个视频相关的链接放在视频

- ▶ 2:24:51 description below. I will also probably post a link to a discussion forum or discussion group
下方的描述里。我大概还会发一个讨论论坛或讨论组的链接，

- ▶ 2:24:57 where you can ask questions related to this video. And then I can answer or someone else can
你们可以在那里问与这个视频相关的问题。然后我可以回答，或者其他入也可以

- ▶ 2:25:01 answer your questions. And I may also do a follow up video that answers some of the most
回答你们的问题。我也可能会做一个后续视频，来回答一些最

- ▶ 2:25:06 common questions. But for now that's it. I hope you enjoyed it. If you did, then please like and
常见的问题。但目前为止就到这里。希望你们喜欢。如果喜欢，请点赞并

- ▶ 2:25:11 subscribe so that YouTube knows to feature this video to more people. And that's it for now. I'll 订阅，这样 YouTube 就知道要把这个视频推荐给更多人。现在就到这里。我
- ▶ 2:25:16 see you later. Now here's the problem. We know DL by... wait, what is the problem? And that's 们回头见。现在问题来了。我们知道深度学习已经.....等等，问题是什么？就这些
- ▶ 2:25:32 everything I wanted to cover in this lecture. So I hope you enjoyed us building out micrograd. 这就是我在这节课里想讲的全部内容。希望你们喜欢我们一起把 micrograd 构建出来的过程。

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 2:25:38 Micrograd ? Okay, now let's do the exact same thing for multiply because we can't do something like Micrograd? 好，现在我们用完全一样的方式来实现乘法，因为我们没法做类似

micrograd

micrograd 一个轻量级自动微分库，用 Python 从零实现标量计算图与反向传播。

代码与工程

- ▶ 2:25:45 eight times two. Oops. I know what happened there. 8 乘以 2 这样的运算。哎呀，我知道刚才是怎么回事。

点击任意句子从该处播放 · 空格 播放/暂停 · T 中文 · C 连播 · ↑ ↓ 上下句 · Esc 停止
字幕由 ASR 转录并经 AI 校正重译，术语释义仅供参考。